
lichee Pi 荔枝派全流程解读

Documentation

Release 1.0

Lichee

Feb 18, 2018

1	荔枝派概览	3
1.1	荔枝派资源汇总	3
1.2	开箱清单	3
1.3	开箱指南	7
1.4	开箱经历	12
2	硬件设计全解析	15
2.1	主控芯片概览	15
2.2	电源电路设计	21
2.3	内存存储部分设计	28
2.4	多媒体类接口设计	32
2.5	其它接口引出	41
2.6	PCB参数确认及布局	42
2.7	DDR部分Layout	48
2.8	电源层分割	51
2.9	TF卡及其它接口Layout	54
3	生产测试全解析	59
3.1	样板焊接备料	59
3.2	样板元件焊接	62
4	Linux系统适配全解析	65
4.1	使用git进行项目管理	65
4.2	开发环境搭建	73
4.3	u-boot-sunxi走读	77
4.4	tf镜像烧录解读	79
4.5	linux-sunxi简明编译指南	81
4.6	LCD的参数适配	81
5	安卓系统适配全解析	85
5.1	安卓编译简记	85
5.2	codec适配及Audio系统解析	95
5.3	RTL8723BU wifi模块适配	95
5.4	usb适配简记	97
5.5	UART适配	103
6	Zero Linux系统适配过程记录	111

6.1	camdriod编译过程走读	111
-----	--------------------------	-----

荔枝派是一款开源的低成本Cortex-A8 评估板，可以供嵌入式工程师学习较为复杂的Cortex-A系列内核处理器的硬件设计，linux移植与开发，安卓移植与开发。

荔枝派一代软硬件及产测由作者一人在业余时间完成，本电子书将由作者与荔枝派众筹支持者公共编辑。想加入荔枝派合作编辑的朋友请在合作编辑留言墙下留言，我会将你加入编辑队伍。荔枝派二代计划由爱好者合作完成软硬件设计。

联系我们： [Q群](#) | [github](#) | [论坛](#) | [淘宝](#) | [邮箱](#) | _____ 文档来源： [看云](#)



1.1 荔枝派资源汇总

github项目主页: <https://github.com/Zepan/ilichee>

- 包含前期连载帖及相关资源下载链接

sdk及镜像文件下载链接:

- 链接: <http://pan.baidu.com/s/1c4gztvE>

相关github项目主页:

- <https://github.com/linux-sunxi/linux-sunxi.git>
- <https://github.com/linux-sunxi/u-boot-sunxi.git>
- <https://github.com/linux-sunxi/sunxi-tools.git>

linux-sunxi 百科:

- <http://linux-sunxi.org>

需要购买荔枝派及其配件的可以到淘宝店选购:

- <https://shop152705481.taobao.com/category-1266972912.htm>

1.2 开箱清单

荔枝派主板及相关配件如下, 请对照自己的购买清单清点:

- 荔枝派主板
 - 分为板载wifi板和板载USB版, 二者互斥, 所以选择wifi版的板子上是没有焊USB母座的, 请知悉。
 - 主板上默认没有焊接双排插针, 请自己动手焊接

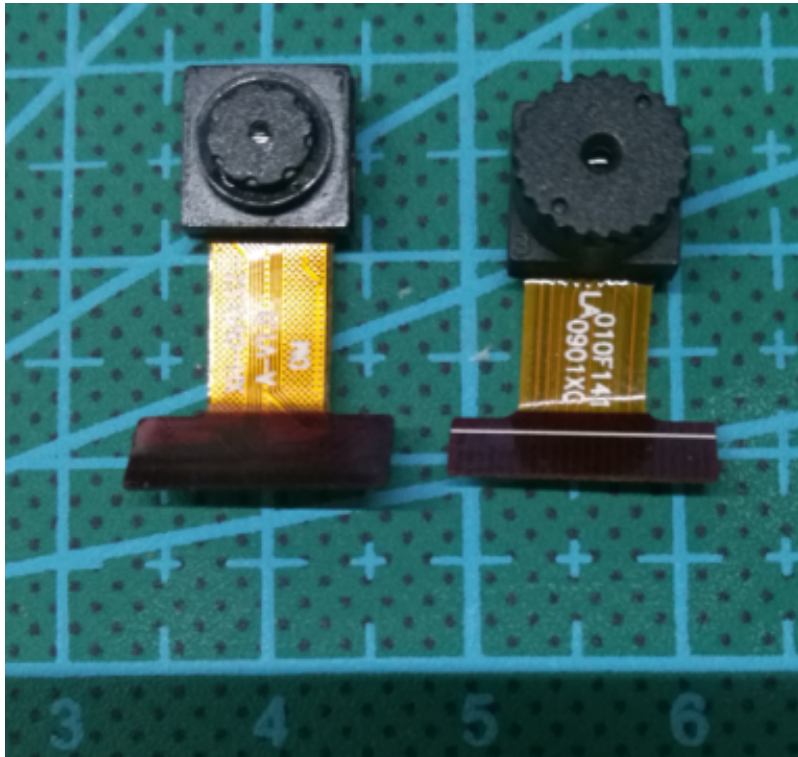
* 此处应有对比图片，请好心人补充。。

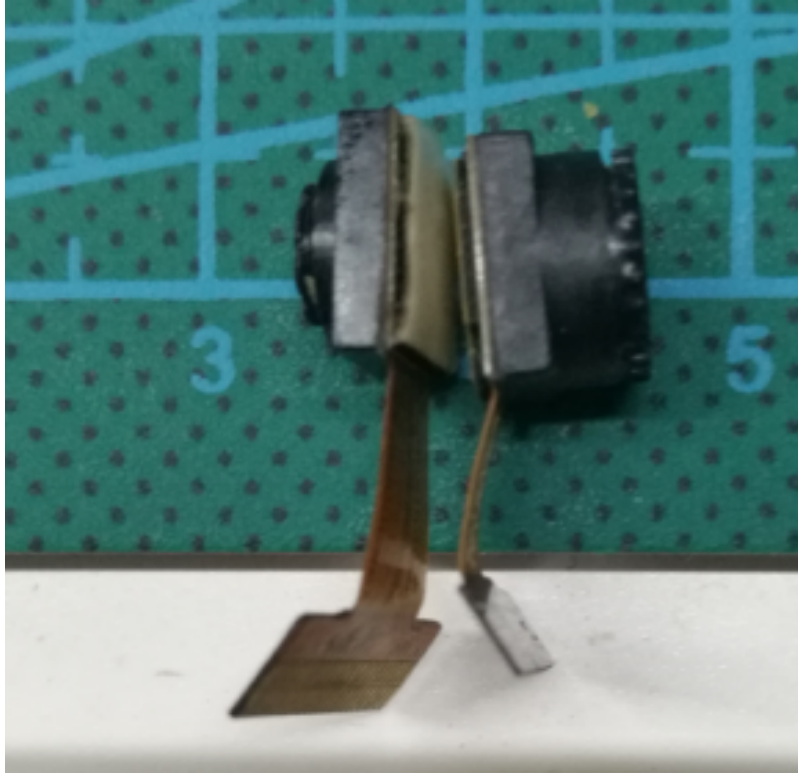
- 板载wifi

- 默认板载wifi模块是RTL8723BU模块，支持WIFI+BT4.0双模
- 天线接口是IPEX座子，配有一根PCB软天线或者铜棒天线
- 天线座前面的pi型匹配电路有一个电容是NC的，不是掉了，请知悉。
- 此处应有图片，请好心人补充。。

- 摄像头模组

- 摄像头模组有OV7670（30W）和OV2640（200W）两种，外型上来说，OV7670摄像头更薄，OV2640摄像头更厚





- 两种摄像头的供电电源不同（2640的一路电源是1.3V，而7670的一路是1.8V），虽然实测并不会对摄像头造成永久性损伤，但还是建议在没有配置好正确的电源前不要插上摄像头
- 摄像头模组很小，请收货时注意不要乱丢包装，可能会被卡在包装盒中



- 摄像头插入方向：FPC座为下接，摄像头朝正面插入

- **LCD屏**

- 目前配件中的屏幕是5寸和6寸的40P RGB屏，分辨率800*480
- 由于近期屏幕价格飞涨，配备的是拆机屏，屏幕边框及排线上可能附着屏蔽用铝箔纸，请知悉。<发货的是品相较好无锈痕的，群主还剩下一堆战斗橙色的屏幕，价格好商量>
- 屏幕插入方向：FPC座同样是下接的，屏幕朝正面插入

- **VGA转接板**

- vga转接板与荔枝派主板使用FPC软排线连接，FPC座子都是下接的。
- 连接完成后，使用VGA线接到显示器即可显示，注意屏幕分辨率等参数需要在fex中设置

- **亚克力外壳**

- 亚克力外壳使用4组螺柱螺母固定板子
- * > 此处应有图片

- **OTG转接头**

- 用于将OTG口转为 USB-A母口

- **USB HUB+网口模块**
 - 可以拓展3个USB口和一个以太网口，免驱

需要另配的配件

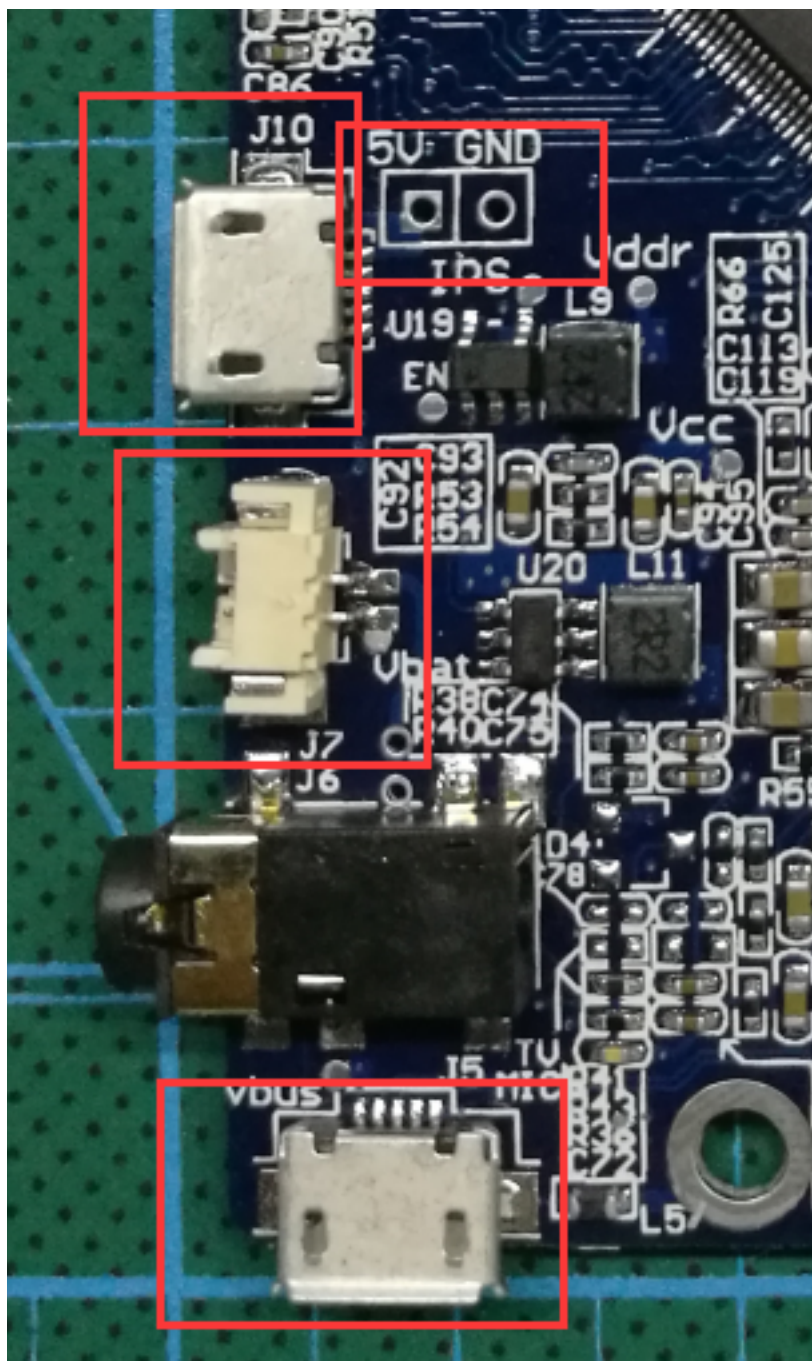
- **tf卡**
 - 系统是存储在tf中的，不插卡是不能启动系统的
- **耳机**
 - 荔枝派使用的耳机是美式耳机标准，不同标准的耳机可能mic不会工作，详见荔枝派 [原理图](#)。

1.3 开箱指南

1.3.1 供电接口

荔枝派有多种供电方式，如下图所示

- J10: 供电micro usb 口（调试时推荐使用此口供电）
- J7: 锂电池供电接口
- J5: otg 供电口
- J10右边: 2.54插针的5V供电口



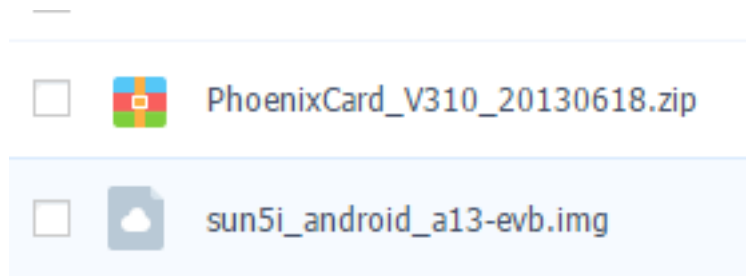
1.3.2 上电测试步骤

1. 准备一张2G以上容量的tf卡
2. 焊接双排针，其中调试串口为UART1
3. 不接串口，不插卡，使用左侧J10上电，观察全彩LED

Note: 如果LED上电后闪烁一次则表示CPU正常复位，否则请跳转到文末的故障处理

4. 到网盘下载测试用安卓镜像: 链接: <http://pan.baidu.com/s/1c4gztvE>

PhoenixCard为烧录工具, 在其中选择卡启动, 然后指定下面的镜像文件即可。



5. 烧录好镜像后, 将tf卡插入背面卡槽, 有液晶屏或者VGA转接板的可以接上, 没有的可以接上串口
6. 同样使用J10上电, 观察屏幕或者串口输出

Note: 正常来说可以进入到安卓界面, 串口里可以进入到安卓终端, 如果出现问题则查看文末故障处理



1.3.3 debian烧录指南

搬运自 [tf镜像烧录解读](#)。

```

1  # tf镜像烧录解读
2
3  ## .背景
4  本文所属目录层次为:
5
6      -> 2.系统移植
7          -> 1.基于linux-sunxi SDK的系统移植
8              -> 3. tf镜像烧录解读
9
10  时隔多日, 连载帖重新开更了~
11  现在部分朋友已经收到了荔枝派, 这里先对荔枝派的镜像烧录进行解读
12  (交流QQ群: 573832310)
13
14  ## 一.系统镜像组成
15
16  TF卡或者其他存储介质中的系统镜像组成如下:
17
18  1. boot部分, 含boot0和boot1/uboot
19  2. linux内核部分, 含启动参数和fex配置
20  3. linux根文件系统
21  > \**回想下在启动过程中这三个部分是如何串联起来的?
22
23  ## 二.烧录boot部分镜像
24  将前面编译出的u-boot-sunxi-with-spl.bin烧录到tf 8KB偏移处
25
26  (sdb换成你自己的tf卡读卡器设备名)
27
28  ```shell
29  sudo dd if=u-boot-sunxi-with-spl.bin of=/dev/sdb bs=1024 seek=8
30  sudo sync
31  ```
32
33  > \**学习下dd命令的详细用法
34
35  ## 三.烧录linux内核镜像
36  需要准备uImage, script.bin, boot.scr三个文件
37  > \**回想下这三个文件的作用?
38
39  1. uImage
40      1. 使用linux-sunxi编译后可得到uImage
41  2. script.bin
42      1. 使用下面命令将fex转为bin
43      2. `fex2bin a13-lichee.fex script.bin`
44      3. 需要根据使用的摄像头型号修改对应的fex字段
45  3. boot.scr
46      1. 使用下面命令将boot.cmd转为boot.scr
47      2. `mkimage -C none -A arm -T script -d boot.cmd boot.scr`
48  > \**查看fex文件和boot.cmd文件, 理解各字段含义
49
50  这三个文件在uboot启动时会在第一分区搜索, 所以需要新建一个TF卡分区 (FAT格式), 需要使用fdisk命令。
51  > \**请自行学习fdisk命令
52  如果原来tf卡已经有分区表, 请先清除分区。
53  烧录相关的脚本已上传至“**资源文件**”目录, 可以查看其中的脚本命令。
54  一般来说, 第一分区设为16MB大小就足够了。

```


然后格式化为FAT格式:

```
`sudo mkfs.vfat /dev/sdb1`
```

最后放入前面提到的三个文件即可。

四.烧录linux根文件系统

linux内核在启动时候会根据启动参数中的root参数来获得根文件系统分区号，然后加载之。

一般就按顺序把根文件系统设为第二分区，使用EXT3或者EXT4文件系统。

整个根文件系统有两种打包方式，dd方式和tar方式。

dd方式是直接备份整个分区信息，tar方式是直接打包整个分区。

dd出来的分区镜像可以直接使用mount命令在本机上加载查看，烧写速度也稍快些，所以荔枝派使用dd出来的镜像发布。

不过由于dd是备份了整个分区信息，所以连分区大小都是一致的，如果tf较大，可以自行扩容分区和文件系统

> * 自学linux下扩容分区和文件系统的方法

```
```shell
```

```
dd if=fs2.img of=/dev/sdb2 bs=64K
```

```
sync
```

```
```
```

五.启动尝试

按上述方法准备好tf卡后，就可以尝试启动了！

把tf卡插入背面卡槽，有屏幕的插入屏幕（注意方向），有摄像头的插入摄像头（注意方向），再连上UART1（默认系统串口），上电。

正常情况就可以启动到debian系统了~

> * 给出的镜像是出厂测试镜像，屏蔽了桌面系统，想开机启动桌面系统的可以尝试修改下

提供的镜像有两个用户lichee和root，密码均为lichee。

在“**资源文件**”目录下，有烧写相关的文件和脚本（脚本还未整理，比较乱，不过也可以参考使用）

1.3.4 常见故障处理

1. 不插卡上电，LED不闪烁

- 表明CPU未正常复位，主要原因可能是CPU虚焊 或者 PMU虚焊
- 尝试用手指用力按压CPU或PMU，上电，若LED可以闪烁 则是相应元件虚焊，给予补焊即可
 - 可以用手指按压CPU不同角落来判断虚焊点
- 若仍然不闪烁，测量下板上DC2和Vcc测试点的电压，正常应该为1.2V及3.4V，若不正常，则补焊PMU
- 主要手段是补焊CPU和PMU，如果尝试后都不行，则联系群主更换。

2. 待反馈添加

1.4 开箱经历

1.4.1 谁都会有个第一次

wifi/显示屏/摄像头版到货，怀着激动地心情打开，最快速的插上屏幕，插上摄像头，找了个手机充电器电源上电，RGBLED一闪，我还以为会有什么其他的呢，然而并没有。

1.4.2 安卓镜像烧写

首先进行的是安卓镜像的傻瓜是烧写，目的检测收到的荔枝皮一切都正常。烧写群主提供的镜像文件安4.1版本，启动成功，然而wifi打不开。以为是wifi有问题呢，群里一问，原来wifi驱动不对头，群里还有共享wifi驱动补丁，所幸下载写来，突然感觉我等之菜竟然不会装驱动。好吧！群主的话把所有的坑都自己经历一遍这就是在进步，怀揣这句话，打开百度。。。经过各种搜索，，，一切的线索都指向了adb调试。给电脑装好adb驱动，荔枝派开机连上数据线，电脑设备管理器应该有adb设备了，好吧 有点嗦，进入cmd调试查找设备（自行百度），然后链接adb shell寻找驱动所在地，，这么多文件夹，，，皇天不负，，笨办法也能找到。替换文件，wifi就这么打开了。好累一晚上就干了点这个。装上哔哩哔哩看了一集海贼王睡觉。

1.4.3 Linux镜像烧录

打开群主debian烧录指南，自行学习dd命令，其实就看了群主提供命令那句话的意思。

sunxi boot镜像烧写

```
sudo dd if=u-boot-sunxi-with-spl.bin of=/dev/sdb bs=1024 seek=8
sudo sync
```

bs代表一次写入多大的块，（blocksize的缩写），if参数为下载的镜像的路径（应该是input file缩写），of后参数为设备地址（应该是output file的缩写），sync是同步数据的意思，加了sync，那么dd写的东西会直接写入你的/dev/sdb设备中，不加的话可能会先缓存一下。加了sync保障数据完整性 命令准备好了，然而还不知道sd卡挂载名 好吧 继续命令：ls /dev/sd* 插不插sd卡可以看出有什么新设备，df -h 命令也可以，可以看看有什么不同。找到设备写入上面的 u-boot-sunxi-with-spl.bin

SD卡分区

下一步就是linux内核镜像了，需要给sd分区，自行学习linux分区命令 fdisk，另外参考了群主的shell命令脚本，虽然没有备注，帮助还是很大的，首先是clear_partion.sh，清除sd卡分区。然后是 write_partion.sh，新建两个分区 第一分区16M，第二分区系统默认。write_mkfs.sh，格式化第一分区为fat，格式化地二分区为ext4。

内核镜像

分区准备好了就要往里写文件啦，，，文件也需要自行转化下，呃，fex2bin，怎么找不到呢，怎么找不到呢，，，继续百度，，原来还得编译sunxi-tools，幸好之前把这些文件都准备好了，，直接make得到fex2bin，用命令 ./fex2bin a13-lichee.fex script.bin 得到script.bin二进制配置文件。摄像头字段先不改了，主要还不知道改哪里。按群主提示：mkimage -C none -A arm -T script -d boot.cmd boot.scr 得到boot.scr 好吧这三个文件终于全了，还是根据群主的shell命令脚本提示把这三个文件搞进去。

根文件系统烧写

```
dd if=fs2.img of=/dev/sdb2 bs=64K
sync
```

这个命令敲完回车后可能要等五六分钟，，不要以为死机了啊，我差点就等不及关了。

启动啦

sd卡现在准备好了，插卡准备启动吧，插卡上电，，，屏幕出现一大堆信息，看不过来，不过最后貌似启动成功了，，wifi OK！后边还有两个错误USBA ERR！/OTG detect ERR！好吧 linux下的串口我还不会用，，，度了后知道有个minicom的东西。好吧继续学习怎么配置minicom，这东东竟然需要root权限，我的是usb转串口

```
Welcome to minicom 2.7
OPTI+-----+
Comp| A - Serial Device : /dev/ttyUSB0 |
Port| B - Lockfile Location : /var/lock |
    | C - Callin Program :             |
Pres| D - Callout Program :             |
    | E - Bps/Par/Bits : 115200 8N1    |
    | F - Hardware Flow Control : No   |
    | G - Software Flow Control : No   |
    |                                 |
    | Change which setting?           |
```

也就这样了。重启系统打印了好长一串东西，，，群文件有正常启动的log信息。启动完了，启动完了，还能干啥？？？启动完成后，按一下键盘回车，荔枝派串口打印：root@Lichee:~# 到此成功了。。。下一步就是进入桌面系统了。。。我也没进去呢，哈哈哈

2.1 主控芯片概览

2.1.1 .背景

本文所属目录层次为:

-> 1. 硬件设计

-> 1. 原理图设计

-> 1. 主控芯片概览

主要介绍A13主控芯片的基本特性，引脚分布情况，并进行初步的引脚复用功能选择。

2.1.2 一.A13基本特性

A13是全志在2012年推出的Cortex-A8内核的微处理器，采用55nm工艺，主要是面向平板电脑的解决方案，目前最高支持到安卓4.2。

下面简要摘抄翻译datasheet记录的特性，欢迎对下面特性的补充说明或吐槽~

1.CPU

ARM **Cortex-A8**内核，32KB ICache,32KB DCache, 256KB L2 Cache

无聊地和一些常用芯片做对比(~~~~)~

| 芯片 | 内核 | 核心数 | 主频 | L1(KB) | L2(KB) |
|-----------|------------|-----|--------|-----------|--------|
| STM32F779 | Cortex-M7 | 1 | 0.2GHz | 16+16 | n/a |
| S3C2440 | ARM920T | 1 | 0.4GHz | 32+32 | n/a |
| MT7620 | MIPS24KEc | 1 | 0.6GHz | 64+32 | n/a |
| BCM2835 | ARM1176JZ | 1 | 0.7GHz | 16+16 | 128 |
| 全志A13 | Cortex-A8 | 1 | 1GHz | 32+32 | 256 |
| 全志H3 | Cortex-A7 | 4 | 1.5GHz | (32+32)*4 | 512 |
| RK3288 | Cortex-A17 | 4 | 1.8GHz | (32+32)*4 | 1024 |

2.GPU

Mali400, 支持Open GL ES 1.1/2.0 和open VG1.1

3.VPU

解码: 支持 VP6/8,AVS,H.264,H.263, MPEG-1/2/4等, **1920x1080@30fps**

编码: 支持H.264编码, **1920x1080@30fps**

4.图像输入输出

并行**RGB**接口, 最大1024x600貌似?待后期测试。

//R8还有一个CVBS(AV)输出,A13疑似也有 (Pin99, 可疑的NC)

摄像头接口**CSI**

5.存储

16位 DDR2/DDR3控制器, 最高533MHz, 最大**512MB** 8位Nand Flash控制器

6.外设

//注意以下有很多被复用的。。

USB2.0 OTGx1, USB2.0 Hostx1

SD3.0控制器x3, 支持UHS-1,eMMC 4.3

UARTx4

SPIx3

I2Cx3

红外发射接收 (CIR)

6bit LRADC (键盘)

电阻屏控制器, 支持两点触摸

内置24bit音频编码器, 支持双通道耳机和单通道mic

PWM控制器 (屏幕背光)

7.系统

8通道普通DMA, 8通道专用DMA

片上**48KB SRAM**（可以当单片机玩了，科科）

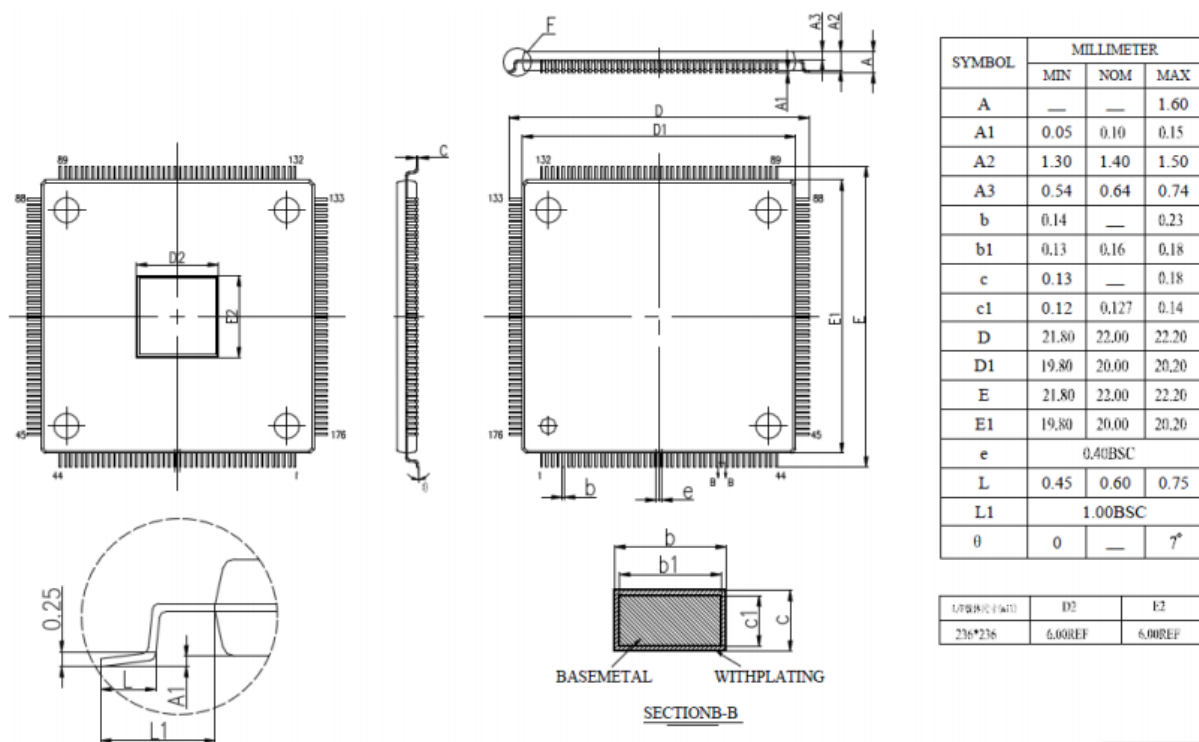
6个异步计数器, 2个同步计数器, 1个看门狗, 1个AVS计数器

支持一些硬件加密算法DES/3DES/AES,摘要算法SHA-1, MD5

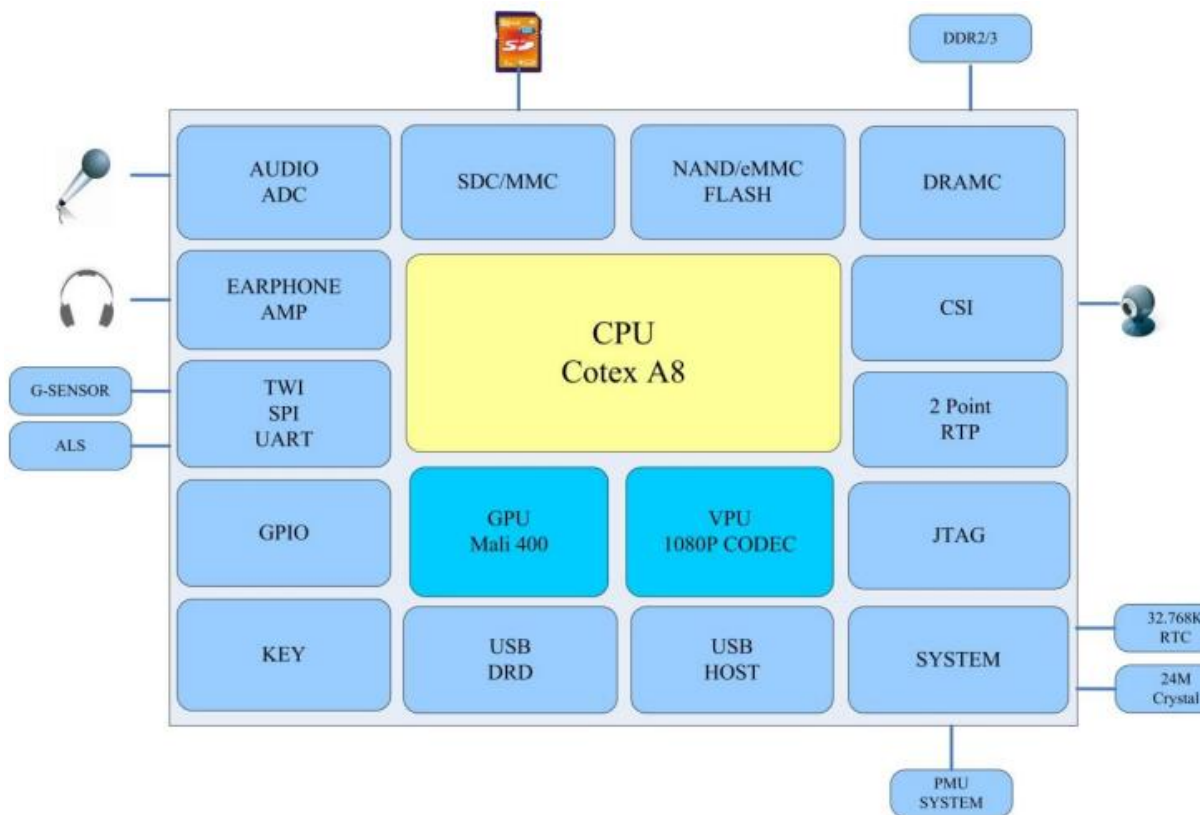
128-bit芯片ID

8.封装

eLQFP176, 喜闻乐见少数可以手焊的cortex-a芯片 但是尺寸达到了22mm见方。。



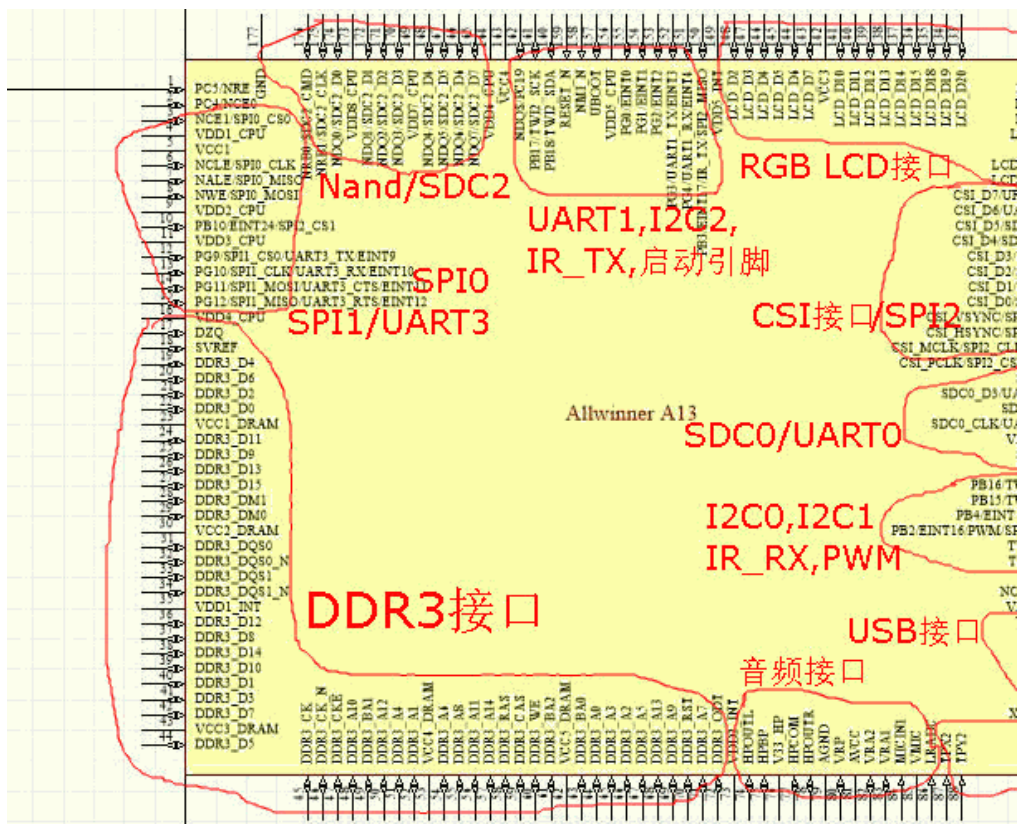
9.系统功能框图



如下是典型应用框图

2.1.3 二.A13引脚分布

为方便查找引脚，我将A13的引脚整理到xls里，可以从下面链接里下载查阅 [引脚查找](#) 直观起见，我直接在原理图上标出了引脚功能分布图（其中‘/’表示复用）



这样看起来是不是感觉很清晰了呢？

在这里主要确定下启动存储接口：

虽然A13芯片硬件支持从SDC0/2,Nand,SPI0,USB启动，但实际上在SDK里支持得比较好的启动方式是SDC0和Nand，其它方式启动将需要修改大量文件，所以选定SDC0作为启动存储接口。

确认了启动接口后，初步的功能引脚分配就出来了：

荔枝板初步功能引脚分配

| 功能 | 端口引脚 | 备注 |
|---------------|---------------------|-----------------------|
| SDC0 | PF0~5 | 系统启动接口，SD卡启动 |
| SDC2 | PC6~11 | 第二SD卡(四线)接口 |
| USB0 | - | USB OTG，可USB升级固件 |
| USB1 | - | USB Host，拟接wifi/BT模块 |
| RGB LCD | PD2~27 | 接RGB LCD显示屏 |
| CSI/SPI2 | PE0~11 | 主功能摄像头接口，不接时可使用SPI2 |
| SPI0 | PC0~3 | - |
| SPI1/UART3 | PG9~12 | 带CTS/RTS的UART3和SPI1复用 |
| UART1 | PG3~4 | 默认的系统调试信息输出口 |
| I2C0 | PB0~1 | 默认接PMU |
| I2C1 | PB15~16 | - |
| I2C2 | PB17~18 | - |
| IR_TX/RX | PB3~4 | 红外发射、接收口 |
| PWM | PB2 | 默认LCD背光控制 |
| HeadPhone,Mic | - | 耳机，麦克音频接口 |
| TPX/Y | - | 触摸屏接口 |
| LRADC | - | 低精度ADC，用于键盘输入 |
| TV_OUT | Pin99 | 疑似CVBS输出 |
| GPIO | PC4~5,PC12~15,PG0~2 | 约9个剩余IO |

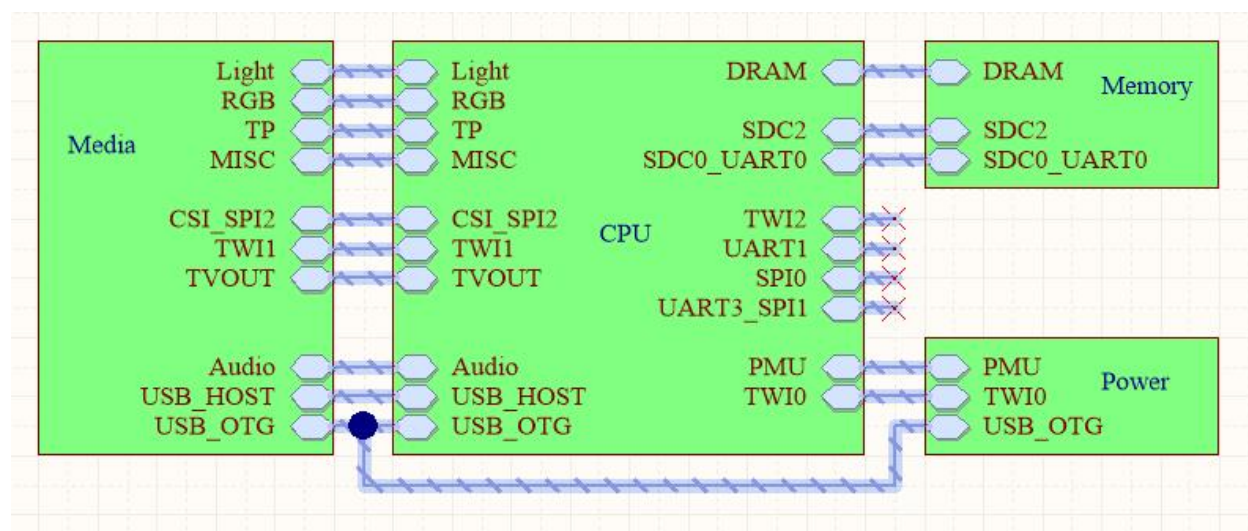
现在已经对A13功能引脚有了初步的认识，下面就可以逐个按照功能模块画原理图啦！

我使用的画板工具是比较简单易上手的altium designer 2016（试用版），虽然在高速电路设计时AD可能不太严谨，但对于1GHz以内的简单板子，AD还是绰绰有余的。

另外，AD16新增的一些特性对于DDR等高速布线很有帮助。

这里首先对整个板子的上层结构进行一下规划：

荔枝板上层原理图规划



如上图所示，整个板子将大致分为4张原理图，CPU，电源，存储，多媒体。

下节将先从电源部分开始画原理图，介绍A13专用PMU芯片AXP209的使用。

2.2 电源电路设计

2.2.1 一. A13电源引脚介绍

这里我们先找A13的主要电源引脚，也就是datasheet中标识为Power的引脚，用excel筛选下可得：

| Name | Pin Designator | Electrical Type |
|-----------|----------------|-----------------|
| VDD1_CPU | 4 | Power |
| VCC1 | 5 | Power |
| VDD2_CPU | 9 | Power |
| VDD3_CPU | 11 | Power |
| VDD4_CPU | 16 | Power |
| VCC1_DRAM | 23 | Power |
| VCC2_DRAM | 30 | Power |
| VDD1_INT | 35 | Power |
| VCC3_DRAM | 43 | Power |
| VCC4_DRAM | 53 | Power |
| VCC5_DRAM | 62 | Power |
| VDD2_INT | 73 | Power |
| V33_HP | 76 | Power |
| AGND | 79 | Power |
| AVCC | 81 | Power |
| VMIC | 85 | Power |
| V33_USB | 97 | Power |
| VDD3_INT | 98 | Power |
| VCC2 | 100 | Power |
| VDD4_INT | 109 | Power |
| VCC3 | 142 | Power |
| VDD5_INT | 149 | Power |
| VDD5_CPU | 156 | Power |
| VCC4 | 163 | Power |
| VDD6_CPU | 164 | Power |
| VDD7_CPU | 169 | Power |
| VDD8_CPU | 173 | Power |
| GND | 177 | Power |

简单说下各电源的用处

| 电源 | 用途 |
|----------|-----------------------|
| VCC | IO口电源 |
| AVCC | 模拟电路电源 |
| VDD_CPU | CPU内核电压 |
| VDD_INT | Interrupt Power 中断电源? |
| VCC_DRAM | 内存电源 |
| V33_HP | 耳机电源 |
| V33_USB | USB电源 |

查阅手册可知它们的参考范围:

| Symbol | Parameter | Min | Typ | Max | Unit |
|------------|--|-------|---------|------|------|
| Ta | Operating Temperature[Commercial] | -20 | — | +70 | °C |
| VCC | DC Supply Voltage for I/O | 1.7 | 1.8~3.3 | 3.6 | V |
| VDD | DC Supply Voltage for Internal Digital Logic | 1.1 | 1.2 | 1.3 | V |
| VCC_ANALOG | DC Supply Voltage for Analog Part | 2.7 | 3.0 | 3.3 | V |
| VCC_DRAM | DC Supply Voltage for DRAM Part | 1.425 | 1.5~1.8 | 1.98 | V |
| VCC_USB | DC Supply Voltage for USB PHY | 3.0 | 3.3 | 3.45 | V |

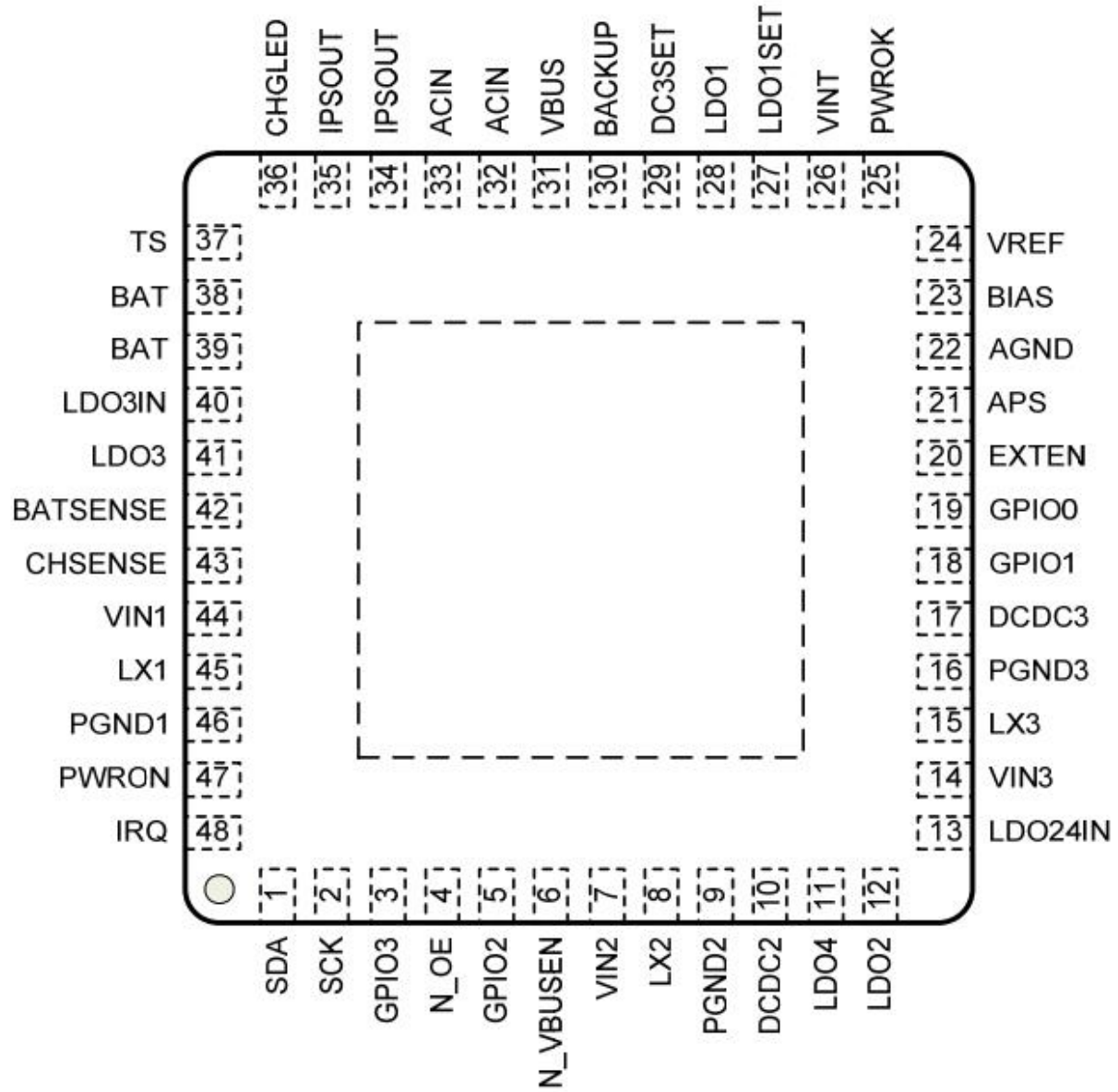
把板子上所有的电源按电压分类:

| 电压(V) | 电源名 | 备注 |
|---------|--------------------|-------------------|
| 5 | DC输入电压 | - |
| 4.2 | 锂电池供电 | - |
| 3.3 | VCC,V33_HP,V33_USB | - |
| 2.7~3.3 | AVCC | 为模拟键盘等供电 |
| 2.8 | 摄像头电源 | 工作电流约20mA |
| 1.3/1.8 | 摄像头内核电压 | - |
| 1.5 | VCC_DRAM | 最大工作电流IDD5B=200mA |
| 1.1~1.4 | VDD_CPU | 不同电压对应不同主频 |
| 1.2 | VDD_INT | - |

可见板子上需要的电压种类比较多, 如何生成、管理这些电源是个问题。

2.2.2 二. AXP209介绍及电源树设计

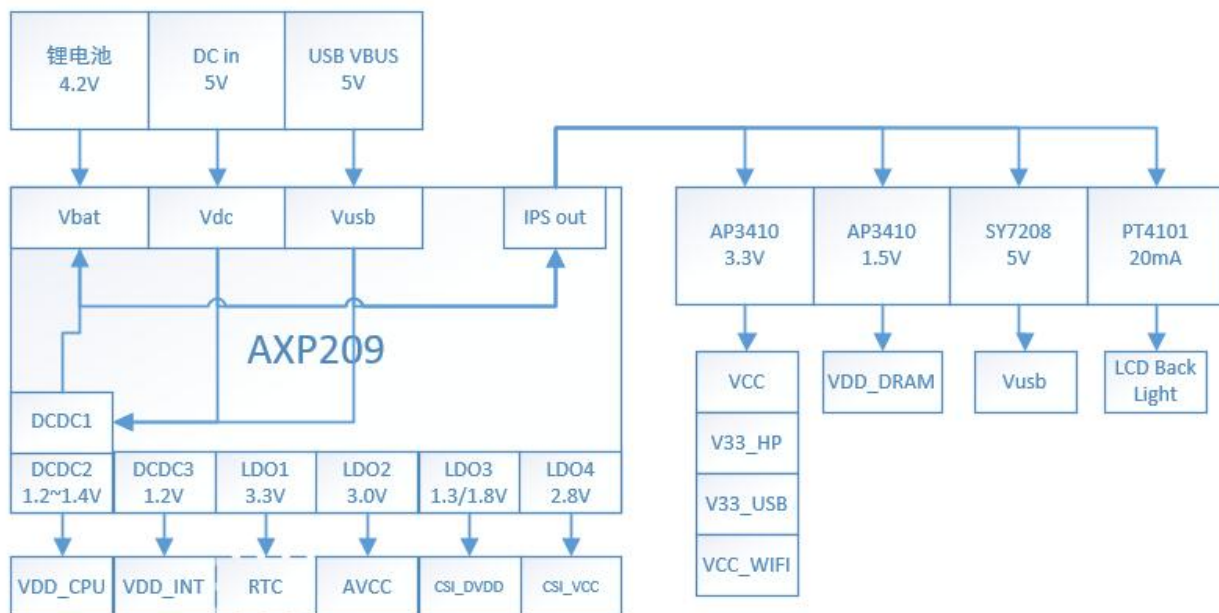
AXP209 是专为全志A10/13/20等主控配备的电源管理芯片 (PMU), 管脚如下图所示:



具体功能可以查看它的datasheet，简而言之，它提供以下功能：

| 功能 | 参数 |
|--------|--------------------------|
| DC-DC2 | 0.7~2.275V可调，1.6A max |
| DC-DC3 | 0.7~3.5V可调，1.2A max |
| LDO1 | 3.3V，30mA |
| LDO2 | 1.8~3.3V可调，200mA max |
| LDO3 | 0.7~3.3V可调，200mA max |
| LDO4 | 1.8~3.3V可调，200mA max |
| LDO5 | 1.8~3.3V可调，50mA max（不常用） |
| 锂电池充电 | 1.8A max，可指示，内建库仑计 |
| 系统管理 | 支持软硬复位/关机等 |
| 管理接口 | I2C |

充分利用AXP209的电源接口，设计电源树如下：

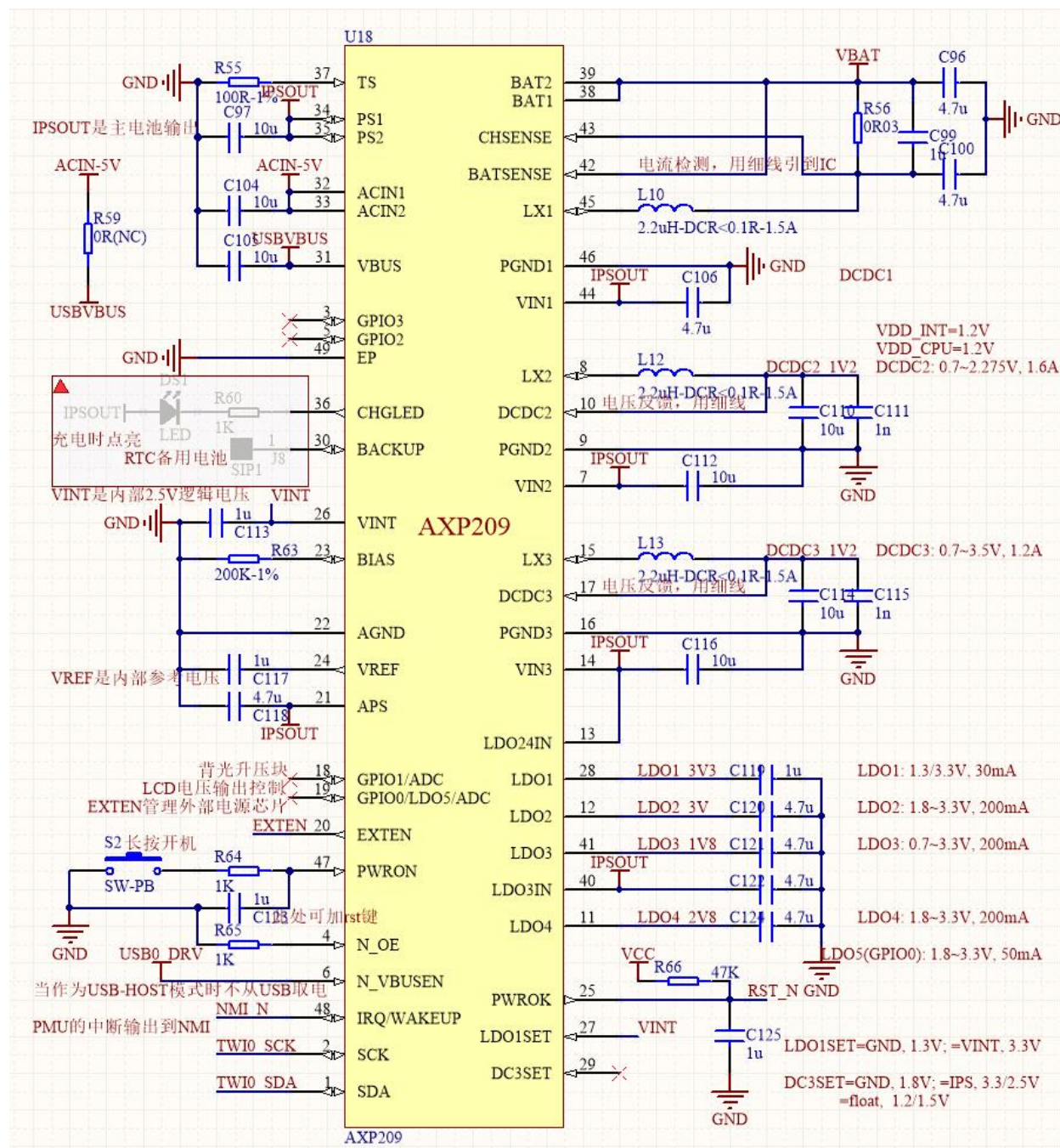


最终还是基本按照官方设计来的，使用了很多外置DCDC。

因为之前将VDD_CPU和VDD_INT合并，DCDC3作为DRAM电源的设定，需要将设置PMU电压调整到 **boot0中进行**，而 **原boot0并没有进行PMU操作就直接进行DRAM的初始化，将会导致boot失败**。

2.2.3 三. 电源部分原理图设计

AXP209 部分设计

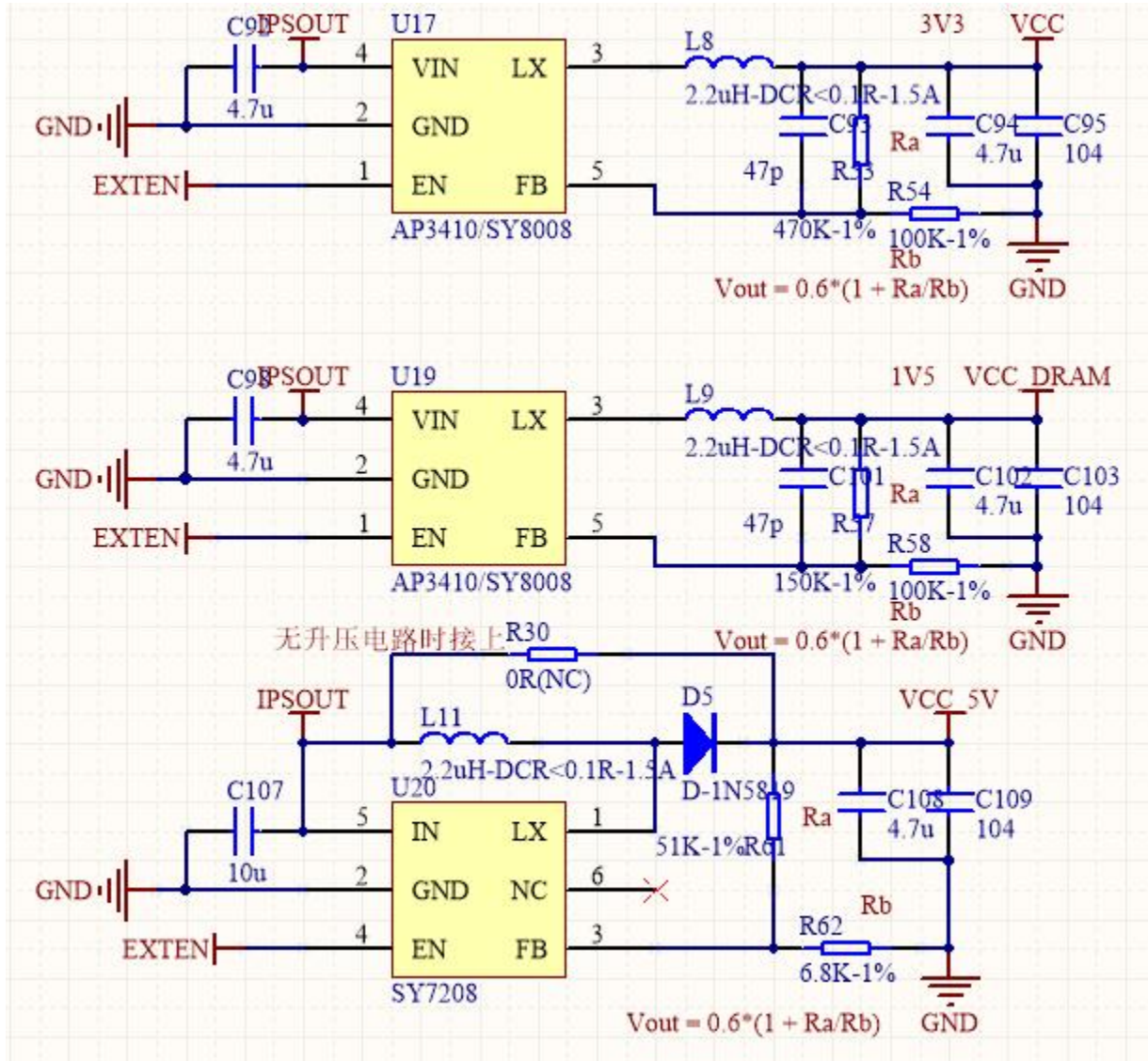


从原理图上按顺序(U字形)分析各个引脚:

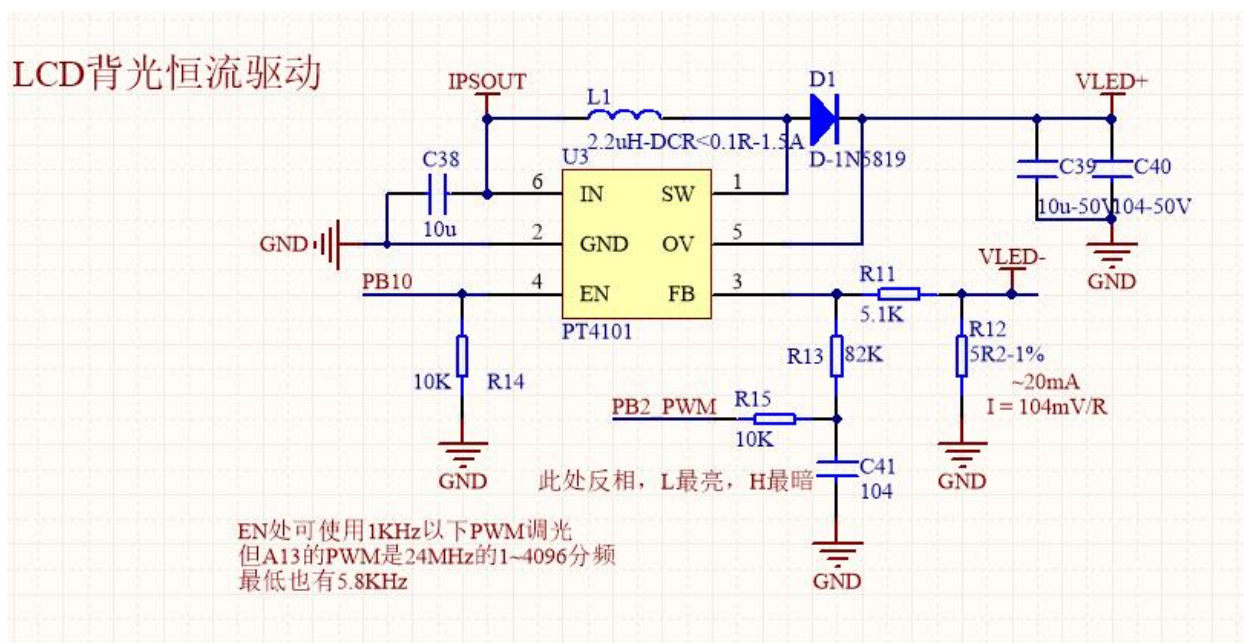
1. **TS**: 温度传感引脚, 可以接热敏电阻, 不用时接普通100欧电阻
2. **PS**, **ACIN**, **VBUS** 分别是IPSOUT输出, DC输入, usb otg的电源输入, 如果没有DC输入, 则直接短接它和USB_VBUS
3. **EP** 就是底部散热焊盘

4. **CHGLED** 是充电指示，**BACKUP**可以接后备电池
5. **VINT** 是内部2.5V逻辑电压，可以用来设置LDO，DCDC的开机默认电压
6. 中间一堆稳压滤波电容保证内部参考电压稳定
7. **EXTEN** 用来管理外部DCDC芯片的使能
8. **POWERON** 开机键，动作逻辑见datasheet
9. **SCK/SDA**，AXP209通过TWI0来被A13管理
10. A13的NMI接AXP209的中断引脚 **IRQ**（电源中断优先级相对最高的），A13的RST引脚接 **PWROK**，即AXP209完成电源系统的启动后 启动A13
11. 往上就是一堆LDO的稳压电容
12. 再往上是3路DCDC的外部电路，都是BUCK降压型DCDC，参数使用datasheet推荐的参数。
13. DCDC1是锂电池管理部分，其中采样电阻 **30毫欧**，一般封装在0805以上。

外部DCDC设计



3.3V输出和1.5V输出均是buck降压电路，5V输出为boost升压电路，按典型应用电路设计即可。



背光电路使用PT4101,可支持2~8颗 LED串联，典型20mA恒流驱动。

这里使用PB10作为使能，PB2的PWM作为调光。

2.3 内存存储部分设计

2.3.1 .背景

本文所属目录层次为：

```
-> 1. 硬件设计
    -> 1. 原理图设计
        -> 3. 内存存储部分设计
```

主要介绍A13的DRAM设计和存储设计

2.3.2 一.A13 DRAM控制器介绍

A13的DRAM控制器支持DDR2和DDR3内存，最高支持到512MB地址空间,最高时钟频率533MHz（也即DDR1066）。支持到16bit的DDR2和DDR3,然而实际在支持列表里只有8bit 256MB 的DDR3支持：

| VENDOR | PART NUMBER | TYPE | | | | | IC SUPPORT | | Process |
|---------|------------------|------|--------------|-----------|-----------|-----------------|------------|--|---------|
| | | | Organization | Speed(MT) | Bus Width | Memory Size(MB) | A13 | | |
| Samsung | K4B2G0846B-HCH9 | DDR3 | 256Mx8 | 1333 | 8 | 256 | SNT | | |
| Hynix | H5TQ2G83CFR-H9C | DDR3 | 256Mx8 | 1333 | 8 | 256 | √ | | 38nm |
| | H5TQ2G83DFR-H9C | DDR3 | 256Mx8 | 1333 | 8 | 256 | √ | | 38nm |
| | H5TQ2G83BFR-H9C | DDR3 | 256Mx8 | 1333 | 8 | 256 | SNT | | 44nm |
| Elpida | EDJ2108BDBG-DJ-F | DDR3 | 256Mx8 | 1333 | 8 | 256 | √ | | 30nm |
| Nanya | NT5CB256M8GN-CG | DDR3 | 256Mx8 | 1333 | 8 | 256 | √ | | 42nm |
| GT | GT8UB256M8BN-BG | DDR3 | 256Mx8 | ? | 8 | 256 | √ | | |
| Elixir | N2CB2G80GN-C6 | DDR3 | 256Mx8 | 1333 | 8 | 256 | √ | | |
| Wins | VP5TQ2G0835L-F9 | DDR3 | 256Mx8 | 1333 | 8 | 256 | √ | | |
| Mezza | Z3P2GF3CKF-GDJ | DDR3 | 256Mx8 | 1333 | 8 | 256 | √ | | |
| Micron | MT41K256M8DA-125 | DDR3 | 256Mx8 | 1600 | 8 | 256 | √ | | 30nm |

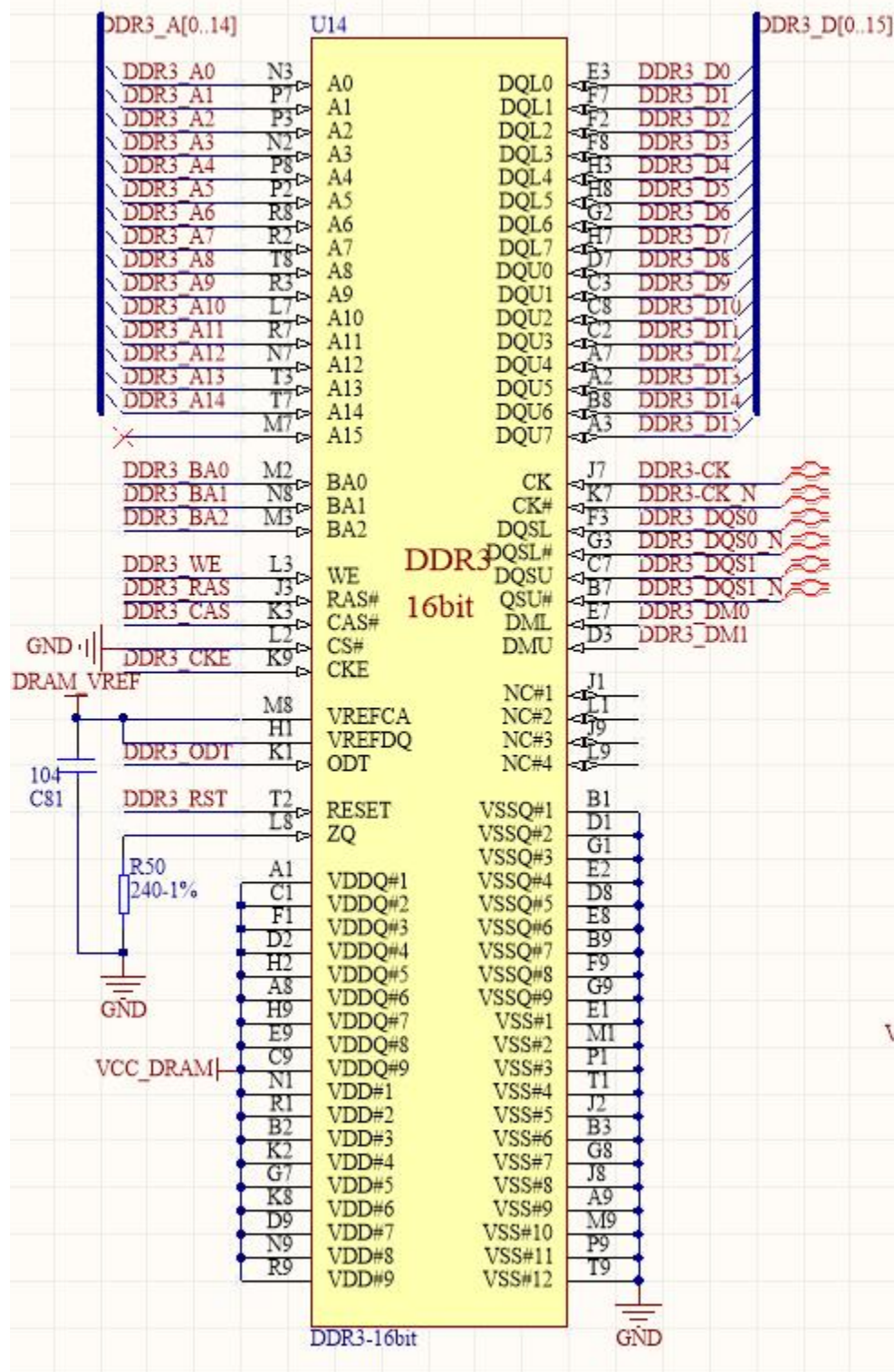
如果使用8bit 256MB的DDR3内存，那么需要两片DDR来组512MB内存，需要使用T型或fly-by拓扑layout，增加了PCB的复杂度，所以在这里冒一点软件上的技术风险，设计使用一片 256M*16bit的DDR3作为内存，这样就可能需要修改boot0的DDR初始化部分代码了。

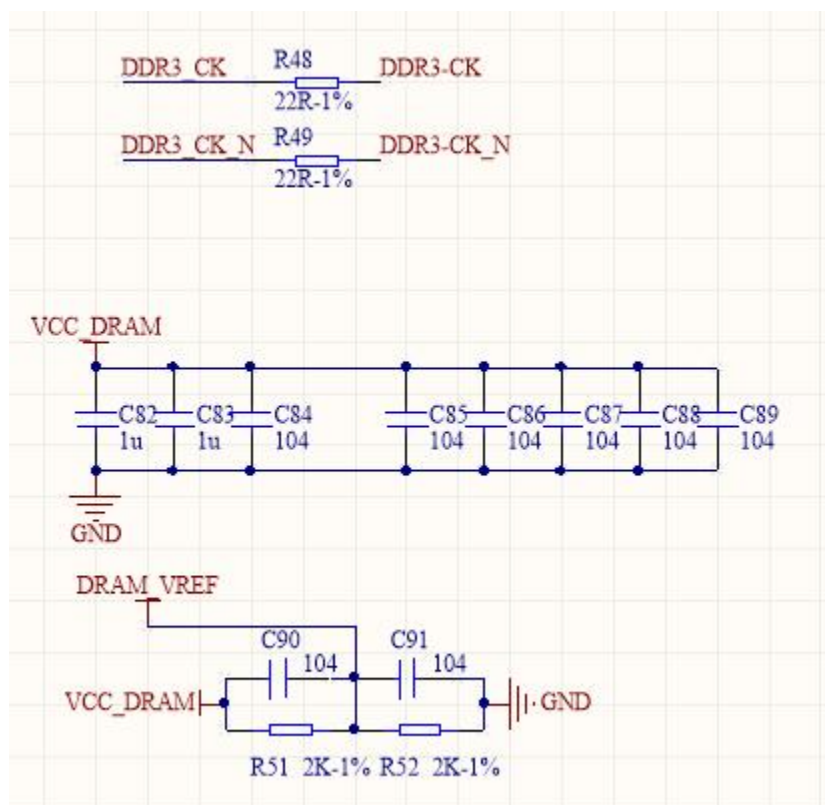
2.3.3 二.DDR3介绍 及 原理图

DDR3内存颗粒在高性能的Cortex-A系列芯片里比较常用，常用速率有800, 1066, 1333, 1600等。DDR3同DDR2一样，可以在时钟的上下沿传递数据，所以实际的时钟频率是400/533/666/800等。DDR3相对DDR2加入了很多新特性，这里就不赘述，可以参见：

<http://blog.csdn.net/shanghaiqianlun/article/details/6976804>

DDR3的连接如下图，引脚主要可分为电源引脚和信号引脚





DDR3电源引脚

1. **VDDQ, VSSQ** 数据线的电源供应, 1.35/1.5V兼容
2. **VDD, VSS** 芯片电源供应, 1.35/1.5V兼容
3. **VREFCA, VREFDQ** 分别是CA和DQ的参考电压, 为VDD/2,单片DDR芯片可以直接由精密电阻分压获得参考电压, 多片DDR则需要专用参考电压生成芯片。
4. **ZQ** DDR3新增的脚, 在这个引脚上接有240欧姆的精密参考电阻, 可以通过片上校准引擎 (On-Die Calibration Engine, ODCE) 来自动校验数据输出驱动器导通电阻与ODT的终端电阻值。

DDR3信号引脚

1. **数据线组** 每组数据线间控制等长, 以DQS,DQSN为参照
 - (a) DQ0~7,DQS0,DQS0N,DM0;
 - (b) DQ8~15,DQS1,DQS1N,DM1;
2. **地址线组**
 - (a) A0~A15
 - (b) BA0~BA2 bank地址
3. **时钟线组**
 - (a) CLK和CLKN, 建议外部串接20~30欧电阻
4. **控制线组**

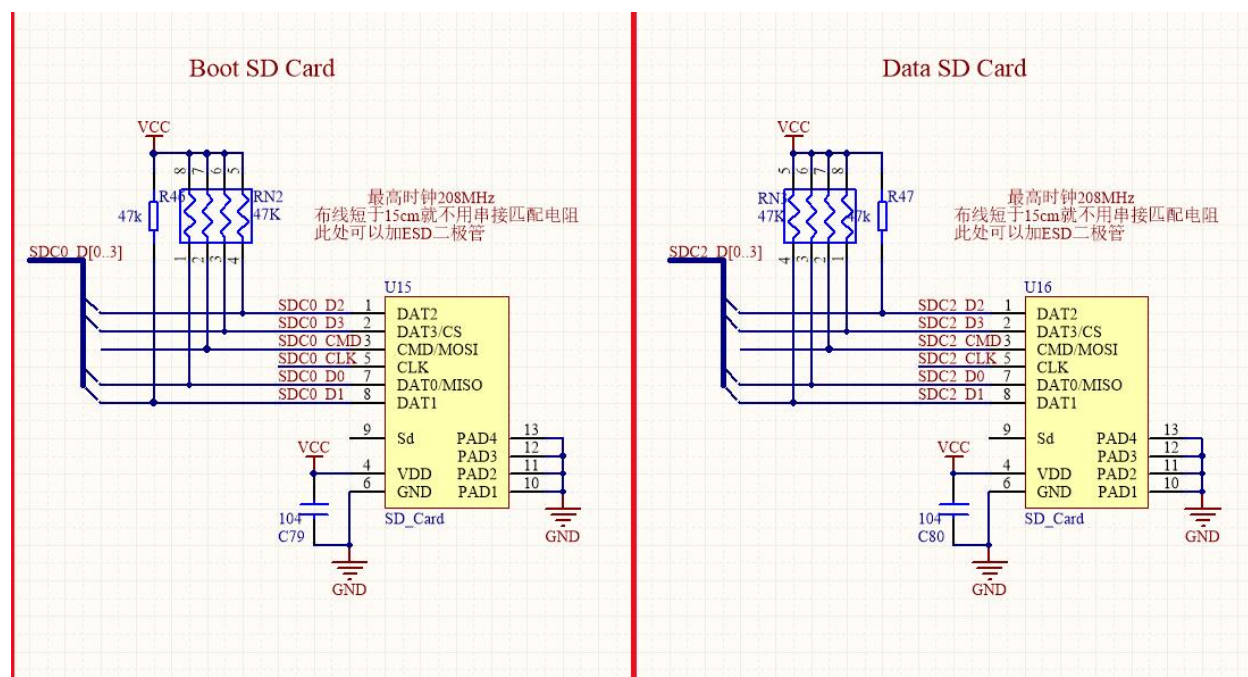
- (a) CS#, 片选, 低有效
- (b) ODT, On Die Termination, 可以省去外接终端电阻
- (c) RAS#, CAS#, WE#, 命令输入

2.3.4 三.A13 SD 控制器介绍

A13的SD控制器 支持SD1.0~3.0,MMC3.3~4.3, 最高可以支持到**UHS-1**(带宽104MB/s, 即时钟速率208MHZ) A13有两个SD接口, **SDC0** 和**SDC2** 其中SDC0是boot流程中的首选接口, 与UART0复用 SDC2也是boot接口之一, 与Nand接口复用, SDC2是8bit的SD接口, 可以连接EMMC。

不过由于eMMC芯片的封装较为精密, 所以本次DIY不会使用。而Nand封装虽然常用, 但是Nand的价格仍然较tf卡贵很多, 所以也不使用。而TF卡既便宜, 烧写方式也比nand什么的简单, 还可以说是家中常备的, 所以本次DIY使用的系统存储介质就选用TF卡。

2.3.5 四.SD卡引脚介绍 及 原理图



TF卡的原理图比较简单, 如上图所示, 主要是4线数据线, 时钟线和命令线。除了时钟线之外的信号线需要加上上拉电阻保证信号稳定。这里使用的自弹式tf卡槽还有个tf检测引脚, 这里暂不连接。

2.4 多媒体类接口设计

2.4.1 .背景

本文所属目录层次为:

- > 1. 硬件设计
 - > 1. 原理图设计
 - > 4. 多媒体类接口设计

主要介绍A13的LCD接口，摄像头接口，音频接口，TV，以及USB Host，OTG接口。

2.4.2 一.A13 LCD/TV 控制器 及 原理图设计

TCON框图

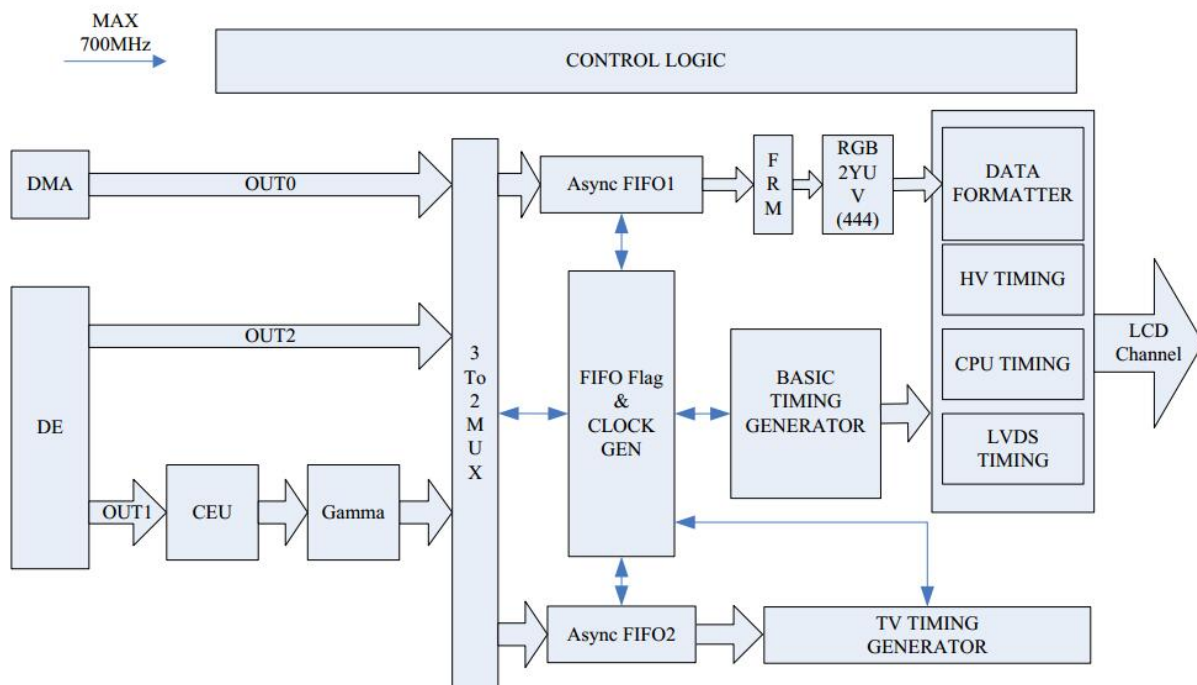


Figure 29-1 LCD/TV Timing Controller Block Diagram

上图是A13的LCD/TV控制器框图，可见在其内部是支持LCD，LVDS，和CVBS的 然而实际的引脚中只有LCD接口被明确标识，还有一个疑似CVBS输出引脚的NC引脚Pin99（该引脚在R8芯片中为TV输出）。所以在设计中引出LCD接口，TV接口也预留引出，待测试是否可用。

LCD接口设计

A13支持多种LCD接口：

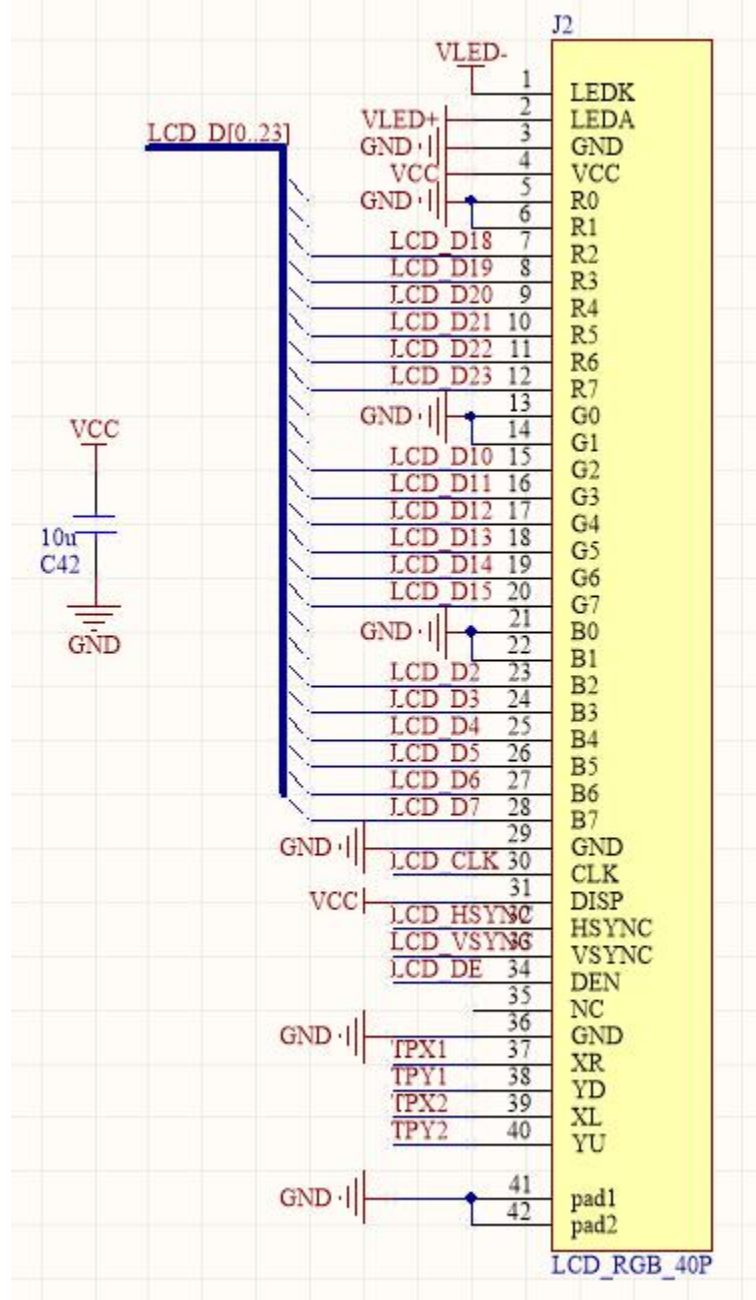
1. RGB接口

- (a) 并行
- (b) 串行
- (c) CCIR656

2. CPU/I80接口

- (a) 并行RGB666
- (b) 并行RGB565
- (c) 串行RGB666
- (d) 串行RGB565

在设计时考虑到LCD屏幕购买的便捷性，选用最通用的**40P 并行RGB LCD**屏幕。



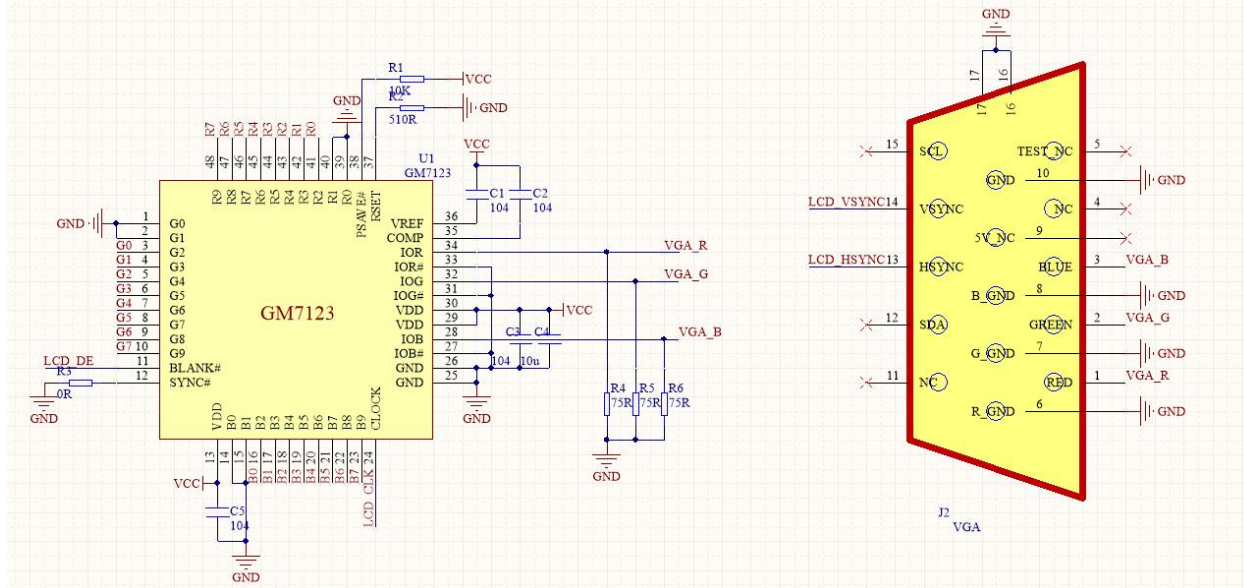
如上图所示，通用40P LCD自带 **led**背光 和 **四线电阻触摸**，非常适合显示及交互。A13也是支持四线电阻触摸功能的，可以进行两点触摸检测。

4.3寸 RGB屏幕和 OV7670摄像头模组



LCD转VGA设计

考虑到有些应用需要使用显示器，所以额外设计了LCD转VGA的转接小板：



GM7123 是一款频率 330MHz 的 3 通道 10 位高速视频 DAC 芯片，兼容 RS-343A/RS-170 标准差分输出，输出电流范围是 2mA~26mA。输入兼容 TTL 电平，内部基准 1.23V，单电源 3.3V 供电，采用 LQFP48 封装。该芯片可应用于：数字视频系统（**1600×1200@100Hz**）；高分辨率彩色图像；数字射频调制；图像处理；仪器和视频信号重建等。

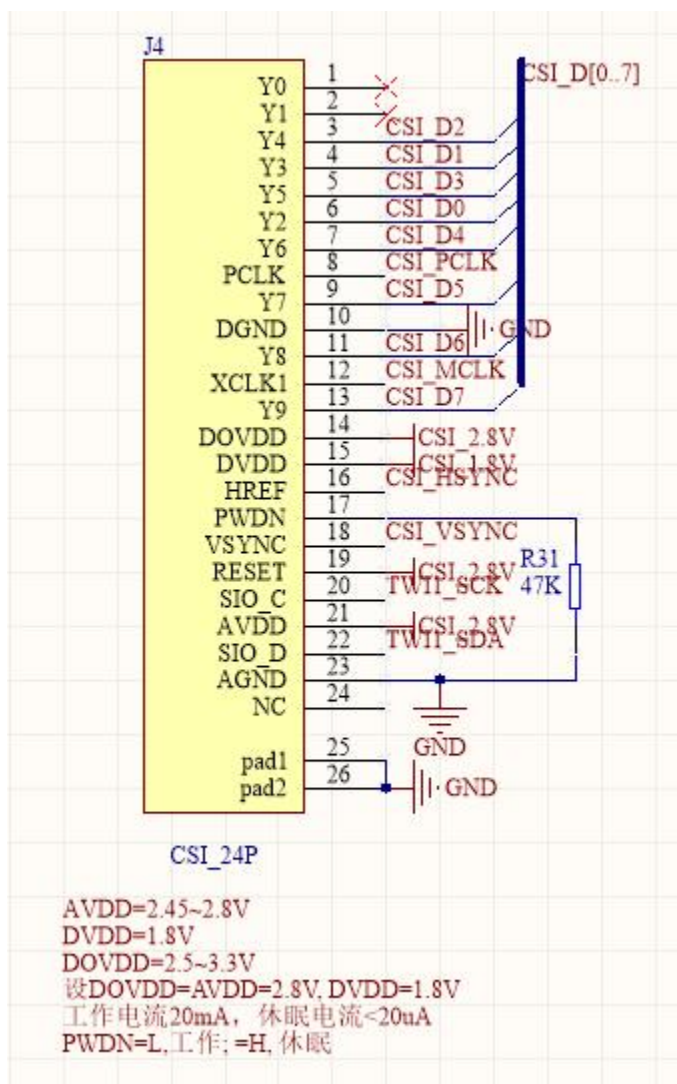
简单来说，**GM7123**就是把数字化的RGB数据转成了VGA信号里模拟的RGB信号，是一个高速DAC转换器。对于分辨率不高，色深较浅的应用，甚至可以使用电阻分压来完成RGB LCD转VGA信号。

2.4.3 二.摄像头接口设计

A13的CMOS Sensor Interface（**CSI**）支持

1. 8位输入数据
2. NTSC/PAL的CCIR656
3. 解析数据到RGB或YCbCr平面
4. etc.

这里选择了常用的 **OV7670**摄像头模组（30W像素）的引脚定义，该引脚定义也适用于高分辨率的**OV2640**模组（200W像素）等，只是核心电压有差别，可以通过调整PMU的LDO输出电压来适配。

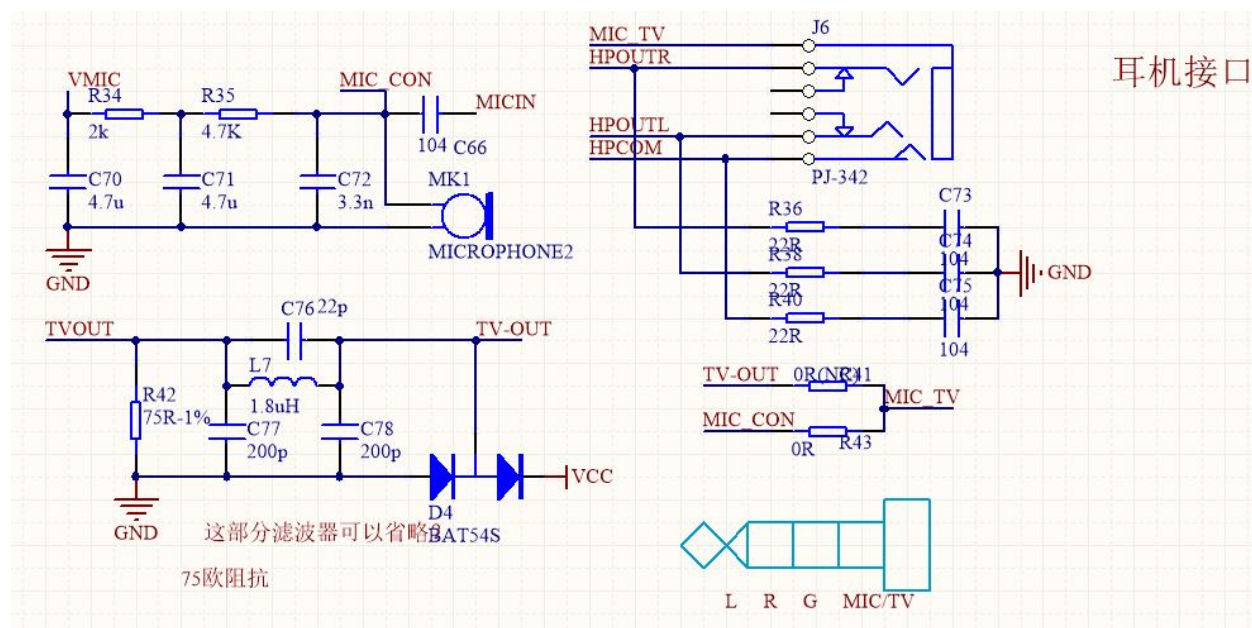


2.4.4 三.音频，TV接口设计

A13自带立体声音频编解码器 以及 耳机功放，具有以下特性：

1. 内置24位DAC
2. 内置24位ADC录音
3. 支持48K,44.1K采样
4. 支持192K, 96K解码
5. 立体声耳机功放支持无电容的耳机模式

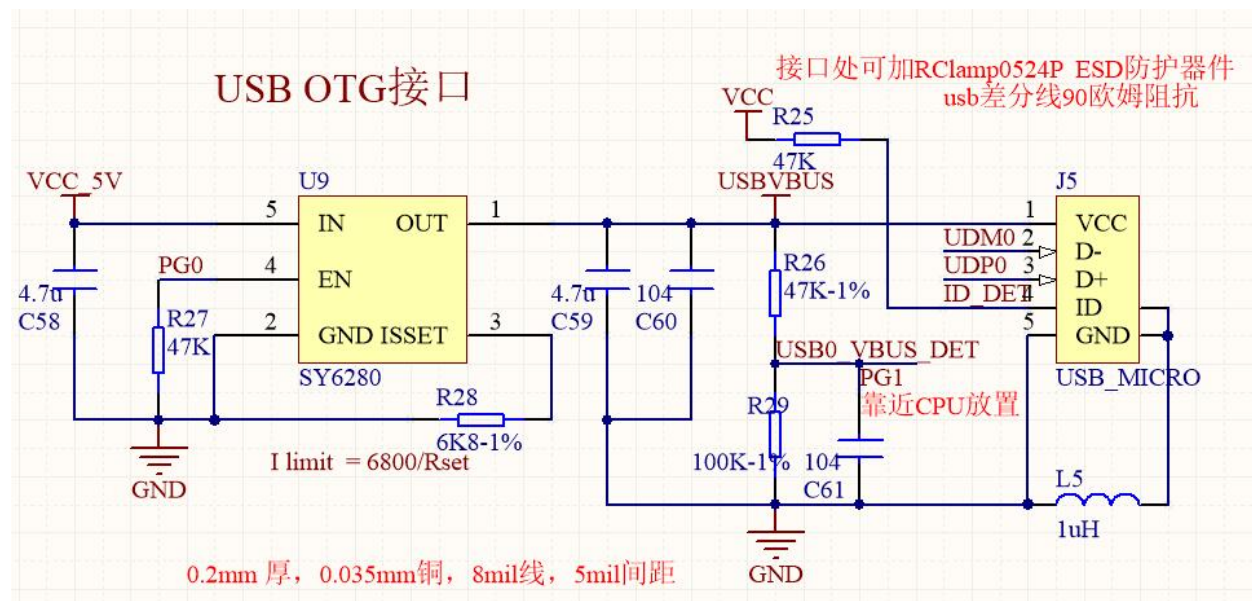
A13的疑似存在的TV接口部分，直接照搬A20的设计，就是一个简易的CVBS视频信号滤波器。麦克信号与TV信号通过一个选择电阻来选通。耳机座的连线使用常用的4段耳机插头的线序。



2.4.5 四.USB接口设计

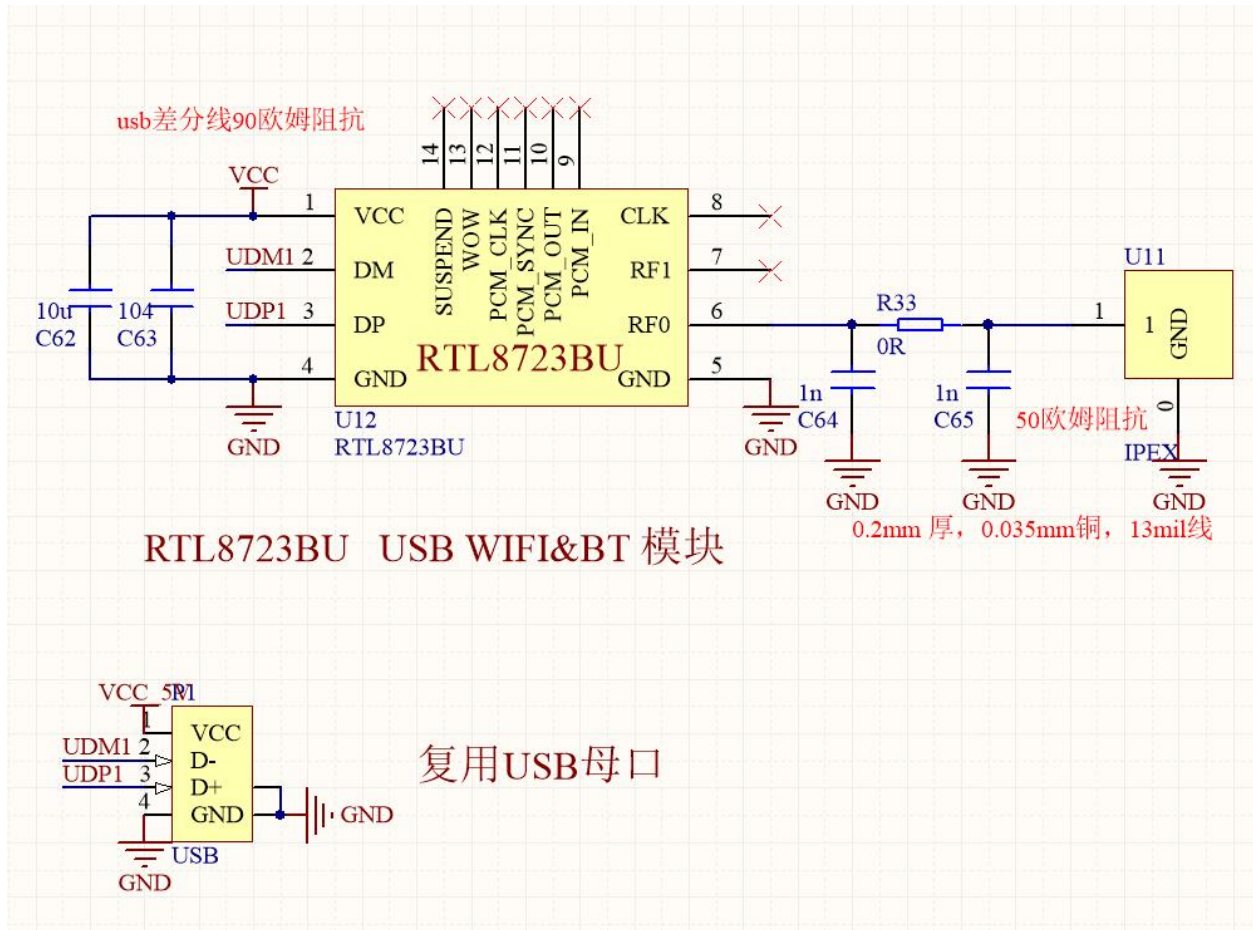
USB OTG接口

USB OTG接口既可以连接到电脑来取电，又可以连接一些外设来扩展功能。为了防止作为主机使用时外部取电过大，造成电源损坏，所以在USB_VBUS和5V电源间加SY6280来限流保护



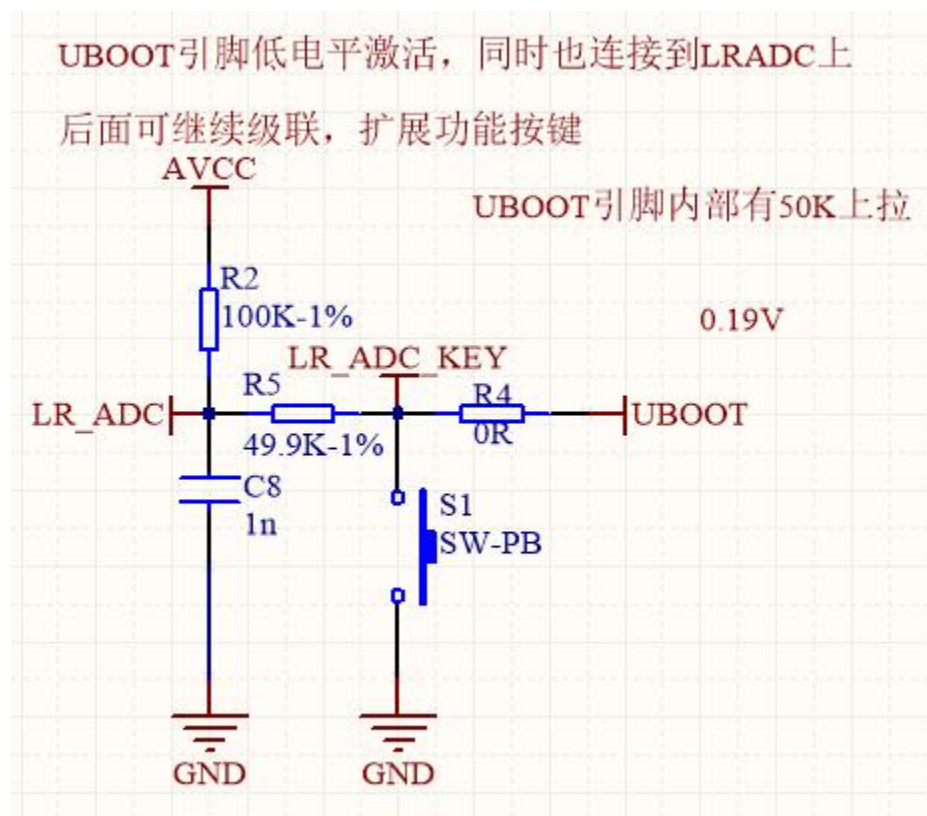
USB HOST接口

为了方便荔枝板的网络通信，这里占用USB HOST口来连接wifi/BT模块；需要接usb外设的可以考虑使用otg口来接外设。同时也留下了USB母口的封装复用，需要外接hub的选择焊接USB母口。

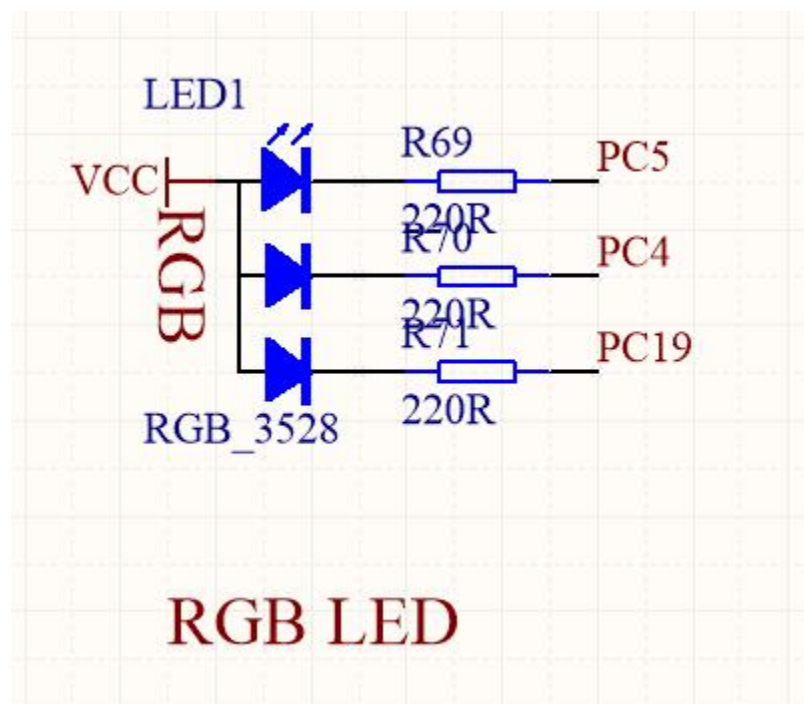


2.4.6 五.其它

LRADC的按键接口，A13的LRADC为6bit精度，可以如下图级联扩展多个按键。下图的按键兼具开机BOOT选择（开机按下进入FEL更新程序），和安卓HOME键功能。



应网友要求，增加了RGB LED方便指示工作状态。



2.5 其它接口引出

2.5.1 .背景

本文所属目录层次为:

```
-> 1. 硬件设计
    -> 1. 原理图设计
        -> 5. 其它接口引出
```

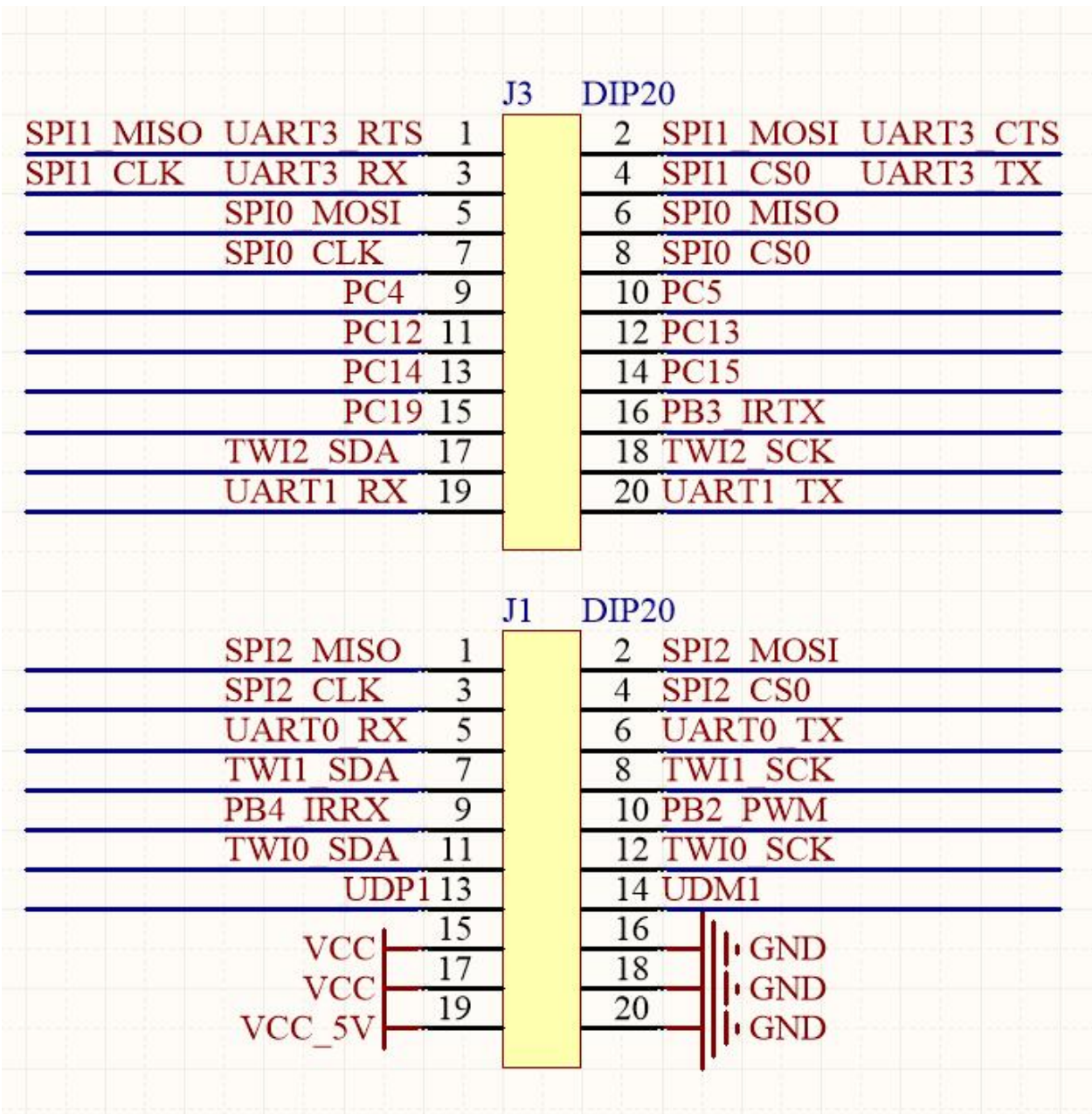
主要介绍A13的一些低速外设脚的引出

2.5.2 一.A13的低速外设接口

连接了前面的那些主要接口后, 还剩余以下这些接口, 通过2.54双排针引出

| 功能 | 端口引脚 | 备注 |
|------------|----------------------|-----------------------|
| SPI1/UART3 | PG9~12 | 带CTS/RTS的UART3和SPI1复用 |
| SPI0 | PC0~3 | - |
| CSI/SPI2 | PE0~11 | 主功能是摄像头接口, 不接时可使用SPI2 |
| UART1 | PG3~4 | 默认的系统调试信息输出口 |
| I2C0 | PB0~1 | 默认接PMU |
| I2C1 | PB15~16 | - |
| I2C2 | PB17~18 | - |
| IR_TX/RX | PB3~4 | 红外发射、接收口 |
| PWM | PB2 | 默认LCD背光控制 |
| GPIO | PC4~5, PC12~15, PC19 | 约7个剩余IO |

1. I2C: 支持10bit地址, 400Kbps
2. SPI: 8*64 Byte FIFO, 主从可设置
3. UART: 64 Byte FIFO, 支持5~8数据位, 可选校验位, 1, 1.5, 2停止位
4. CIR: 双向8*16bit FIFO, 支持常用红外遥控协议



2.6 PCB参数确认及布局

2.6.1 .背景

本文所属目录层次为:

- > 1. 硬件设计
 - > 2. PCB设计
 - > 1. PCB参数确认及布局

主要介绍荔枝板的PCB参数，以及初步布局（交流QQ群：573832310，上车口令：爱荔枝）

2.6.2 一.PCB参数确认

PCB板厂选择

由于是个人DIY项目，资金不是很充裕，所以只能选择一些默认的工艺，和低价便捷的板厂，兴森快捷什么的就不用考虑了哈。

个人最常用的打样板厂是 **嘉立创**，优点

- 速度快，价格低，广东省内包邮
- 可在线下单，甚至有配套APP
- 可同时购买钢网，元件
- 今年（2016）上线了低价样板贴片服务 缺点
- 工艺精度不高，最小6/6mil线宽线距，12/24mil过孔，基本做不了BGA

嘉立创的生产工艺不高，所以也就是平常打些简单的样板比较合适，相对来说包邮挺划算O(∩_∩)O~

由于荔枝板里使用了FBGA96的DDR3芯片，焊盘大小0.4mm(16mil)，焊盘间距0.8mm，所以嘉立创的工艺是无法走通的，这时候就用到备选的PCB板厂 **华强PCB** 优点

- 速度快，价格低
- 可在线下单
- 工艺精度高，最高可达3/3mil线宽线距，8/16mil过孔

缺点

- 在订单高峰期 交货速度可能比较慢
- 不包邮

PCB参数选择

虽然华强PCB的制程精度高，但是额外的精度也是要钱的，所以这里来确定我们所需的最低的PCB精度要求。主要是FBGA96的焊盘对过孔的要求比较高，实测后发现，使用10/18mil的过孔 搭配 5mil的走线可以扇出FBGA96。

然后就是PCB层叠结构。层叠结构主要影响到走线阻抗，但是由于做阻抗的收费太高（500起），所以就选用了默认的层叠结构：

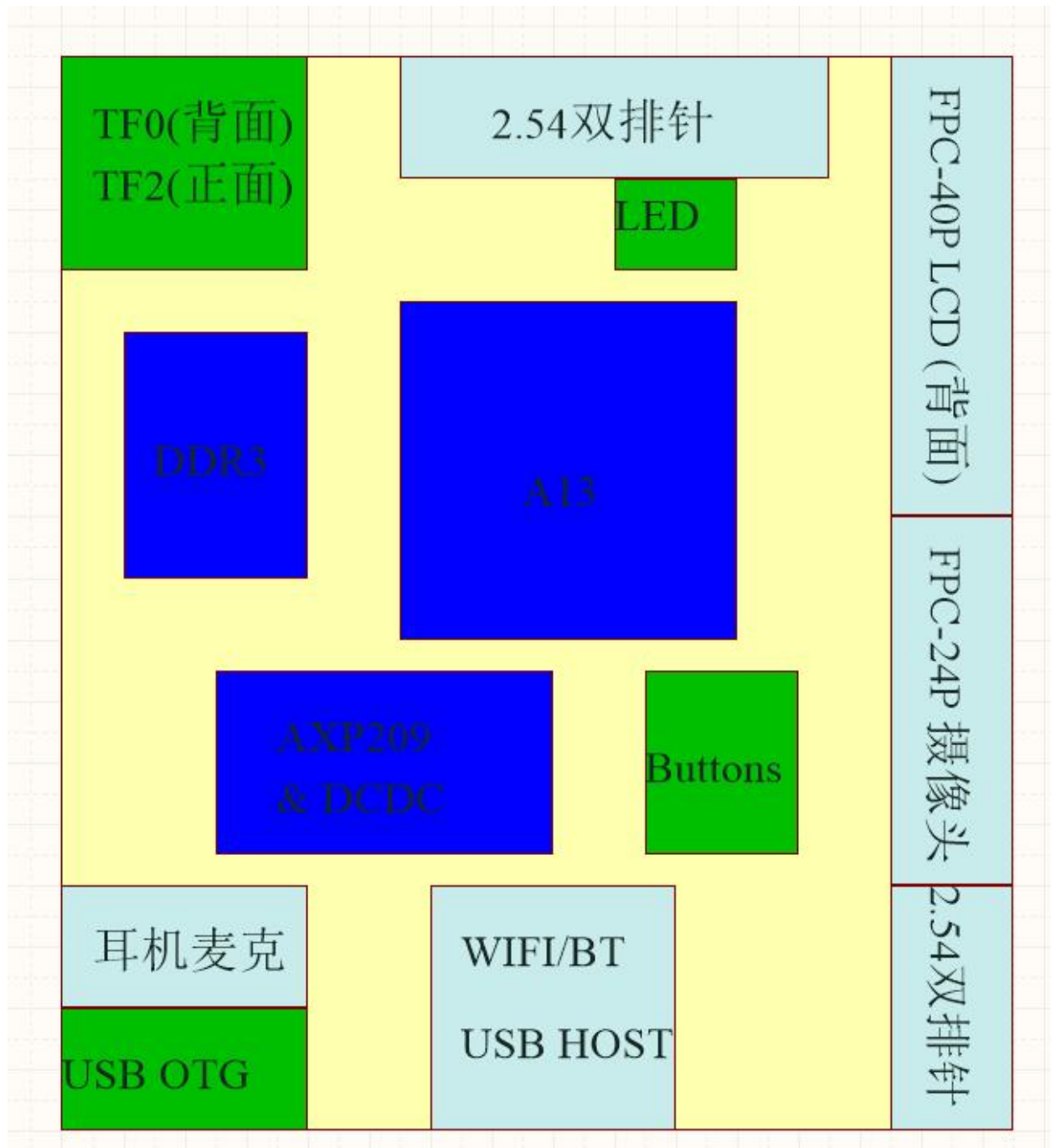


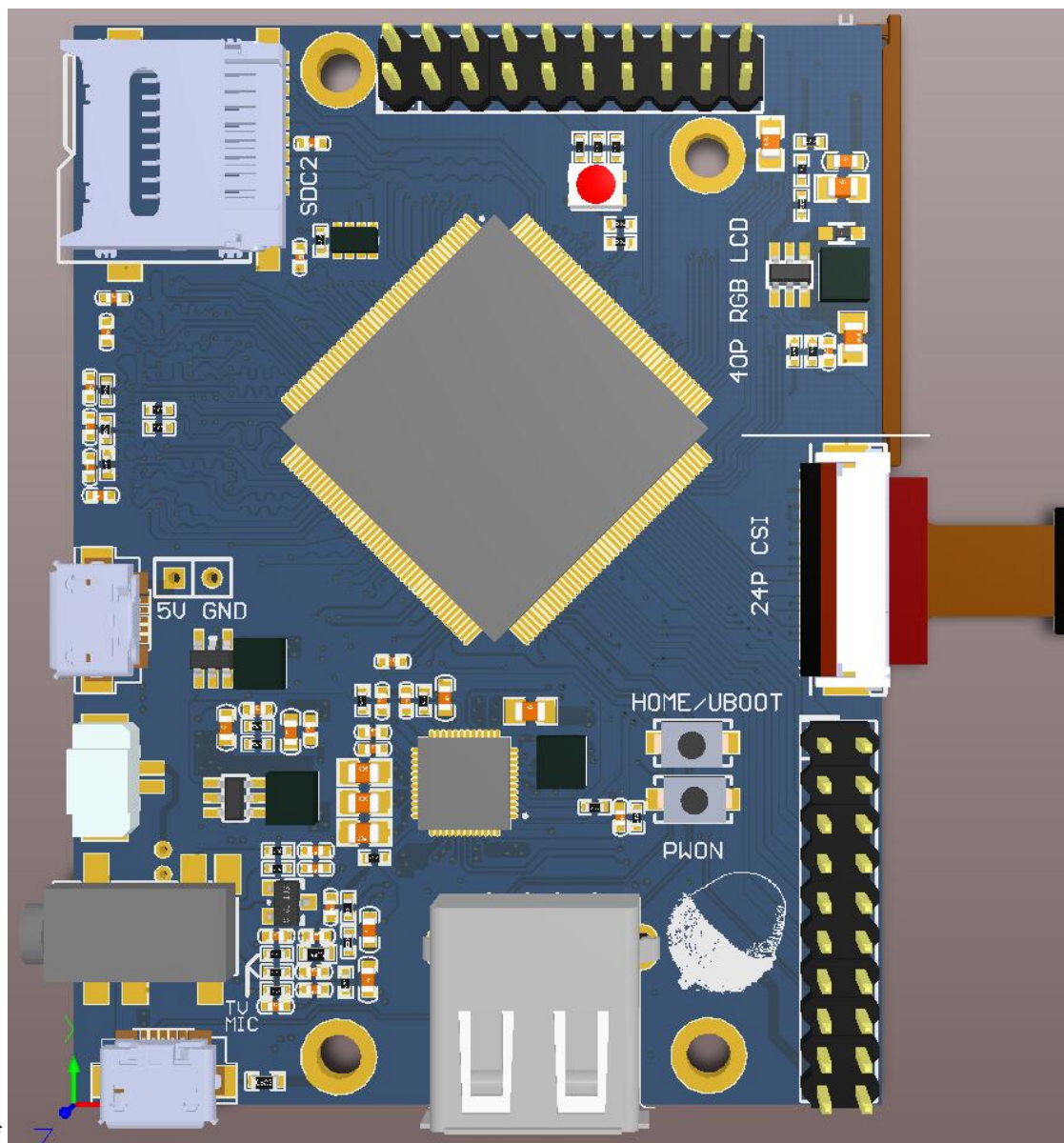
最终确认的PCB参数如下:

| 项目 | 参数 |
|--------|----------------|
| 板厚 | 1.6mm |
| 层数 | 4 |
| 外层铜厚 | 1oz (0.35mm) |
| 内层铜厚 | 0.5oz (0.17mm) |
| PP胶厚 | 0.2mm |
| 芯板厚 | 1.1mm |
| 最小线宽线距 | 4mil |
| 最小过孔 | 10/18mil |

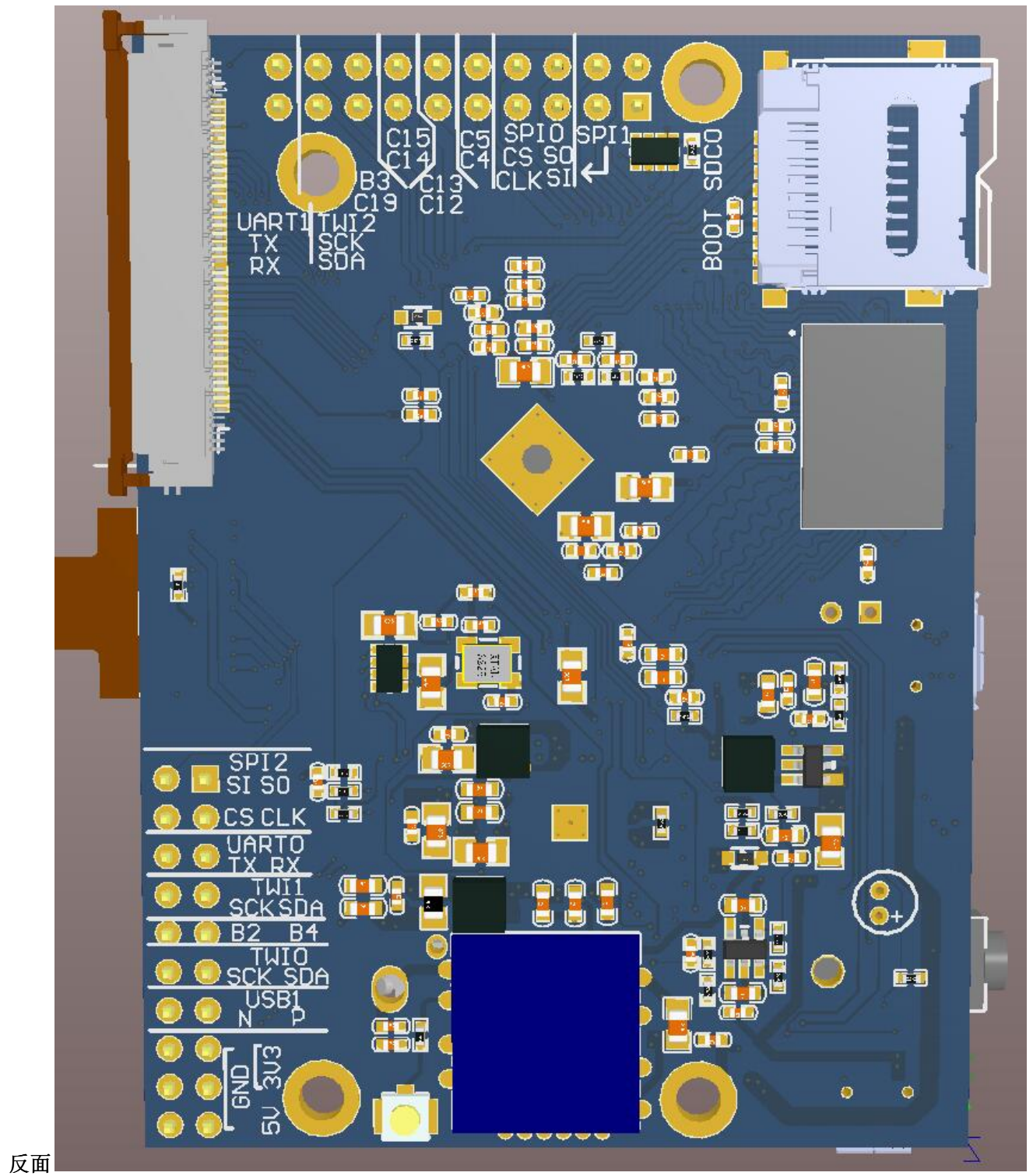
2.6.3 二.PCB 布局

以下是大致的PCB布局设计，但实际上布局是很难一开始就确定的（特别是走线层数少的时候），布局往往是边布线边修改的，和一开始设想的布局相差很大，它是比较方便走线的一个布局。





最终的3d效果图如下：正面 7



2.7 DDR部分Layout

2.7.1 .背景

本文所属目录层次为:

```
-> 1.硬件设计
    -> 2.PCB设计
        -> 2.DDR部分Layout
```

主要介绍荔枝板的DDR布线

2.7.2 一.DDR布线一般规则

阻抗匹配

- DQn/DM/DQS, CLK,传输线阻抗50欧, 差分阻抗100欧
- 线距3W, 或者至少4mil

等长约束

- CKP/N严格等长(<5mil), 长度小于4inch
- DQS以CLK为基准, 误差+-250mil, P/N间严格等长, <5mil
- DQ八根一组与对应DQ严格等长, 误差50mil
- DM与对应DQ严格等长, 误差50mil
- ADDR[0:14]以CLK为基准, 误差100mil。
- 剩余控制信号以CLK为基准, 误差100mil。

差分线约束

- 差分线和其他线距离12mil以上, 长度差<5mil,线间距>4mil
- 蛇形线的平行长度不宜过长, 宜45度走线

电源滤波

- 每个电源管脚放104, 至少有一个10uf
- 每2厘米电源线放一个10uf

2.7.3 二.DDR布线实用规则

阻抗匹配规则

- 阻抗匹配适用于“长”线传输, 一般来说信号线长超过十分之一的波长才需要考虑阻抗匹配。

PP胶介电常数约4.7，电磁场传播速度约 $(1/\sqrt{4.7})=1/2.1$ ，即传播速度为 150mm/ns~6inch/ns
DDR3-1066时钟频率533MHz~0.5GHz，周期约2ns，所以波长约300mm，波长/10约30mm 所以当DDR信号线走线长度小于30mm~1200mil时，可以不考虑阻抗匹配

荔枝板在实际布线时可以目测到走线长度在1000mil以内，所以无需特别进行阻抗匹配。

- 另一方面，可以计算阻抗失配后的波峰波谷电压，看是否在合理范围

荔枝板prepreg为0.2mm(7628)，走线为5mil，计算得阻抗约75~80欧，归一化阻抗约1.5 反射系数 $= (1.5-1)/(1.5+1)=0.2$ VH波峰电压 $= 1.5 \times 1.2 = 1.8V \leq V_{max} 1.8V$ VH波谷电压 $= 1.5 \times 0.8 = 1.2V > 0.75 + 0.1 = 0.85V$ 所以这种程度的阻抗失配还不会直接造成信号传输错误

通过以上规则的取舍，实际应用中可以省下几百块阻抗费 $O(\cap \cap)O\sim$ （阻抗计算器si9000）

等长约束规则

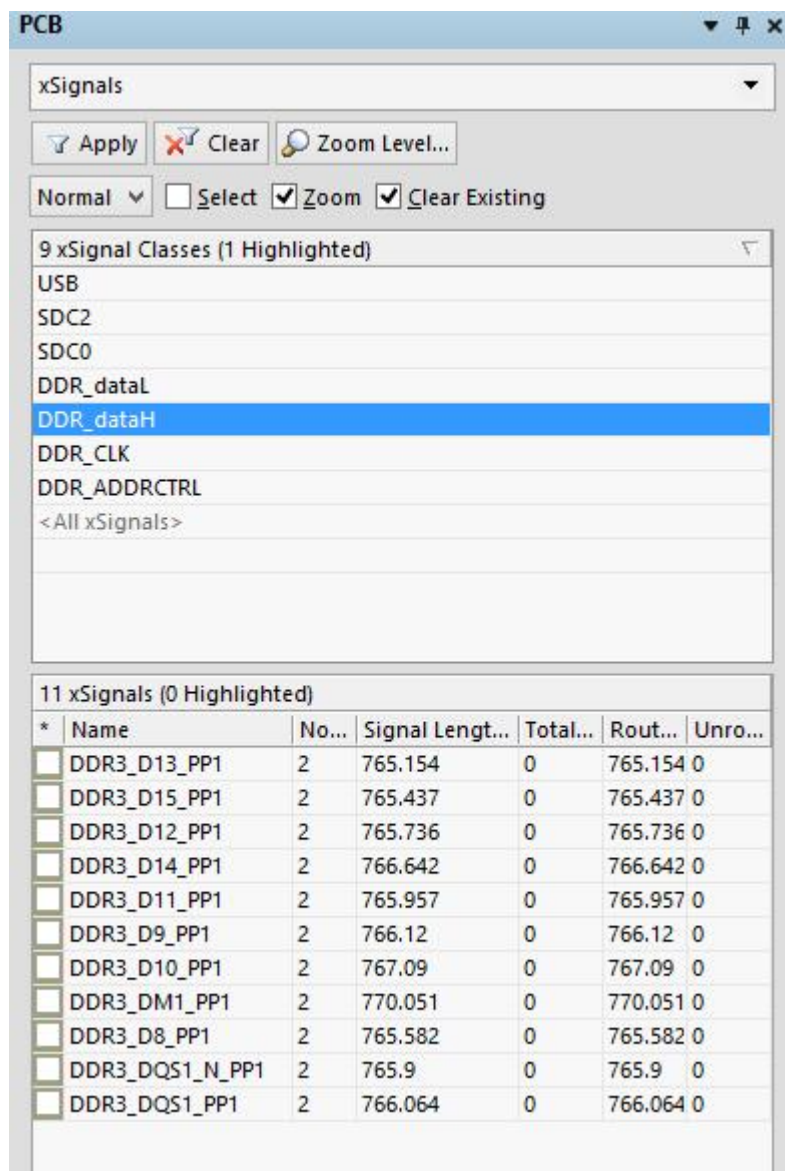
- 等长约束通常是芯片相关的，最好翻阅对应芯片的手册
- 有时候蛇形走线造成的串扰影响甚至会超过不等长的影响，这里也需要取舍

2.7.4 三.DDR布线小技巧

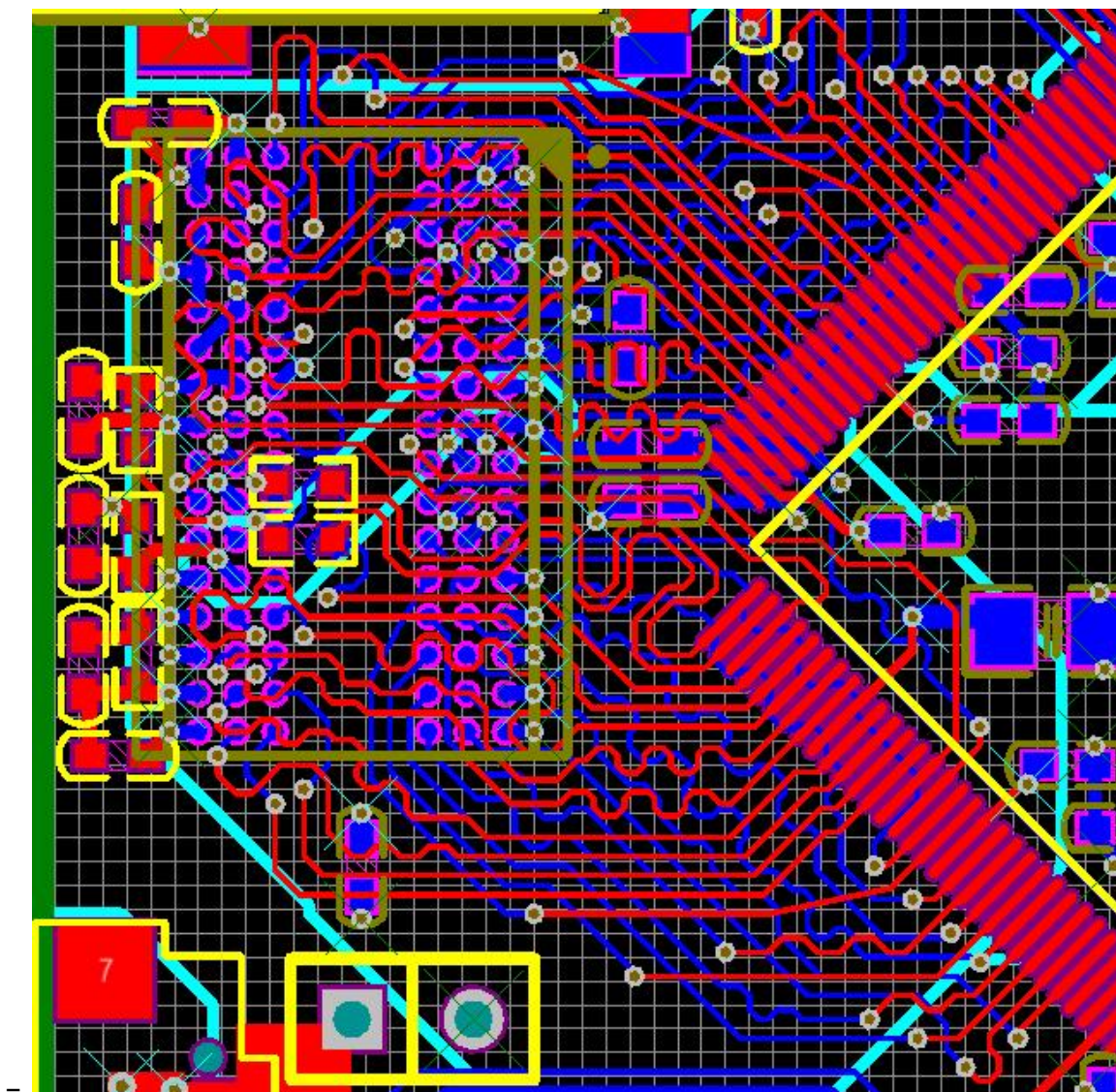
- 放DDR之前先 大致确认下连线最长和最短的信号线，合理摆放使得两个最值之间的差距尽量小
- 放置好DDR后，先打好电源引脚的过孔，放上滤波电容，防止布好线后找不到地方打孔。。
- 走线时先走最长的线和差分线，确定长度基准
- 需要绕等长时，外部的线直接往外扩，让内部线有空间绕等长
- 顶层和底层的走线数目大致相同，可以使得布线面积较小

2.7.5 四.Altium Designer 布DDR指南

- AD15之后的版本具有x-signal功能，类似x-net，可以利用该功能控制等长
 - 首先选中CPU和DDR，然后右键->xsignals->create xsignals between component,即可添加需要的信号线到xsignal
 - 然后可以在rule里设置规则，在右下角的PCB里点开查看信号线长度



- 调整差分线等长，T->I，交互式差分线长度调整；
 - tab键，输入要调整到的长度，以及幅度，间距（建议更大于3W）等
 - 通过1234来微调蛇形形状，逗号，句号 调整幅度
 - 空间有限时，可以手动来调整线长。
 - 调整差分线接近最长的其他走线
- 调整其它线长T->R（交互式线长调整），
 - 按tab设置等长选项（会自动以原理图里的差分线为基准），可以设置蛇形幅度，间距（最好3W以上）
 - 然后按1234等按键微调蛇形形状，调整至等长；按逗号和句号调整幅度；
 - 最终效果



2.8 电源层分割

2.8.1 .背景

本文所属目录层次为：

```
-> 1. 硬件设计
    -> 2. PCB设计
        -> 3. 电源层分割
```

主要介绍荔枝板的电源层分割

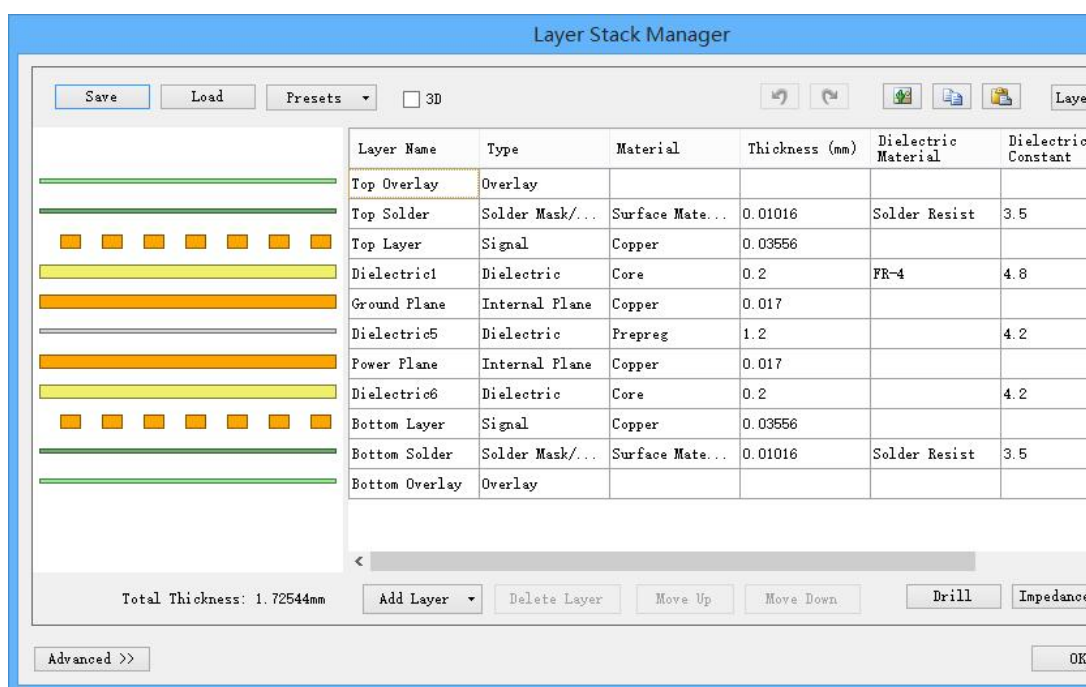
2.8.2 一.多层板的电源层设置

在4层及以上PCB中，通常有两层作为电源层和地层，而不是像双面板那样大面积敷地。这样设计的好处

是:

- 方便走线
 - 只要在需要接地或者接电源的地方打过孔即能联通，再也不用绕线连接了~
- 方便控制阻抗
 - 对于需要控制阻抗的电路来说，两层板的厚度是很难做到对应的阻抗的，4层板一般0.2mm的PP胶就比较容易做到对应阻抗
 - 底下有完整的地参考层容易控制阻抗
- 较大的电源平面保证电压的一致性

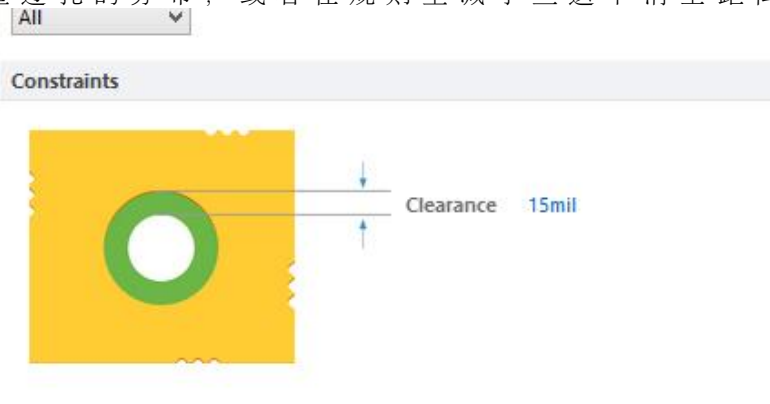
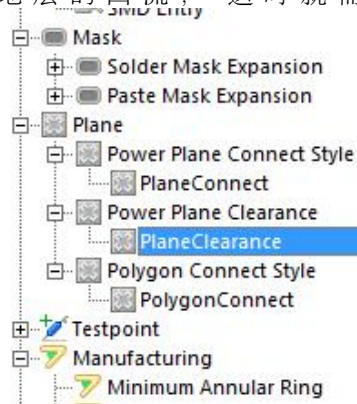
2.8.3 Altium Designer下的电源层设置



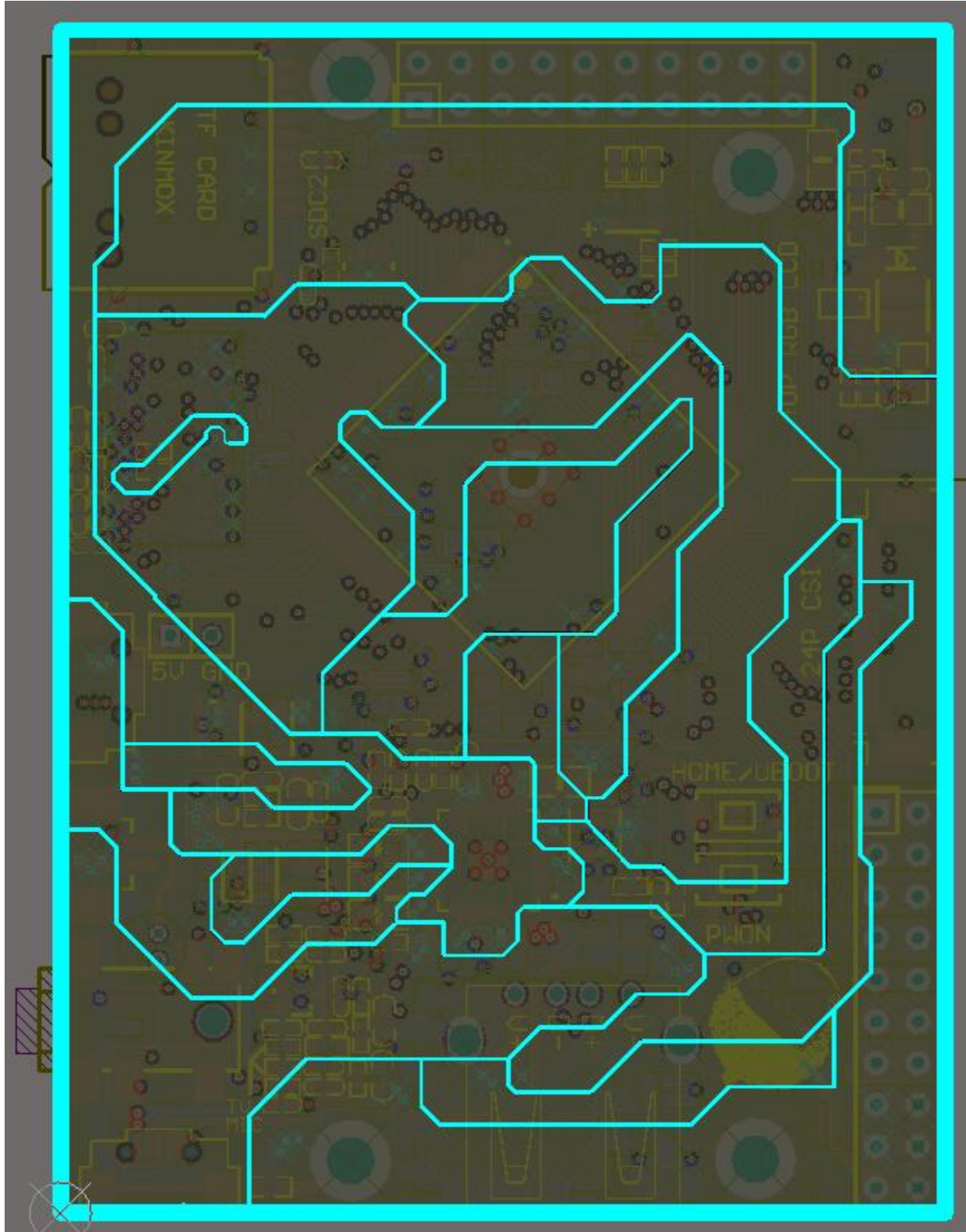
AD中的层叠管理器设置如下:

电源层和地层都是 **Internal Plane**,即负片层,画线的地方是无铜的,所以又被称为电源层分割。

注意打的过孔在负片层会有一段“清空”距离,有时候太密的过孔会截断电源层或地层的回流,这时就需要调整过孔的分布,或者在规则里减小些这个“清空”距离



最后完成的电源层分割如下，可以看到A13由于有众多电源，整个电源平面被分割得很厉害



2.9 TF卡及其余接口Layout

2.9.1 .背景

本文所属目录层次为:

```
-> 1. 硬件设计
    -> 2. PCB设计
        -> 4. TF卡及其余接口Layout
```

主要介绍荔枝板的TF卡及剩余的布线

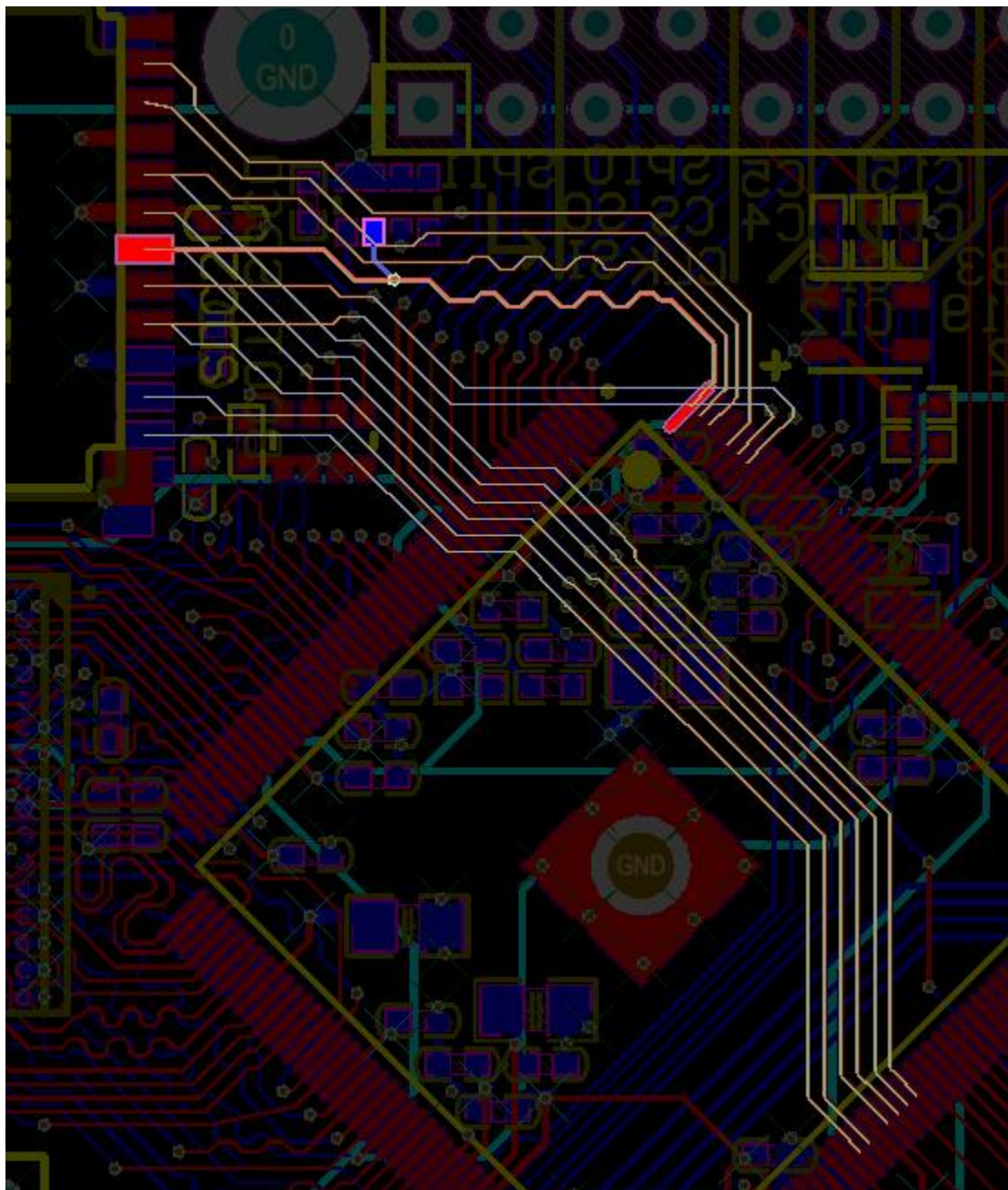
2.9.2 一.TF卡布线

A13支持的UHS-1的SD接口带宽是104MB/s,由于SD接口是4线的,所以时钟频率是208MHz。所以时钟周期约5ns,波长 $150 \times 5 = 750\text{mm}$,波长的 $1/10 = 75\text{mm}$ 而从荔枝板的大致布局上看,走线不会超过40mm,所以TF卡的布线可以不用考虑阻抗匹配。

如果长度过长,则需要控制单端阻抗到50欧姆

所有走线尽量同一层,参考GND平面

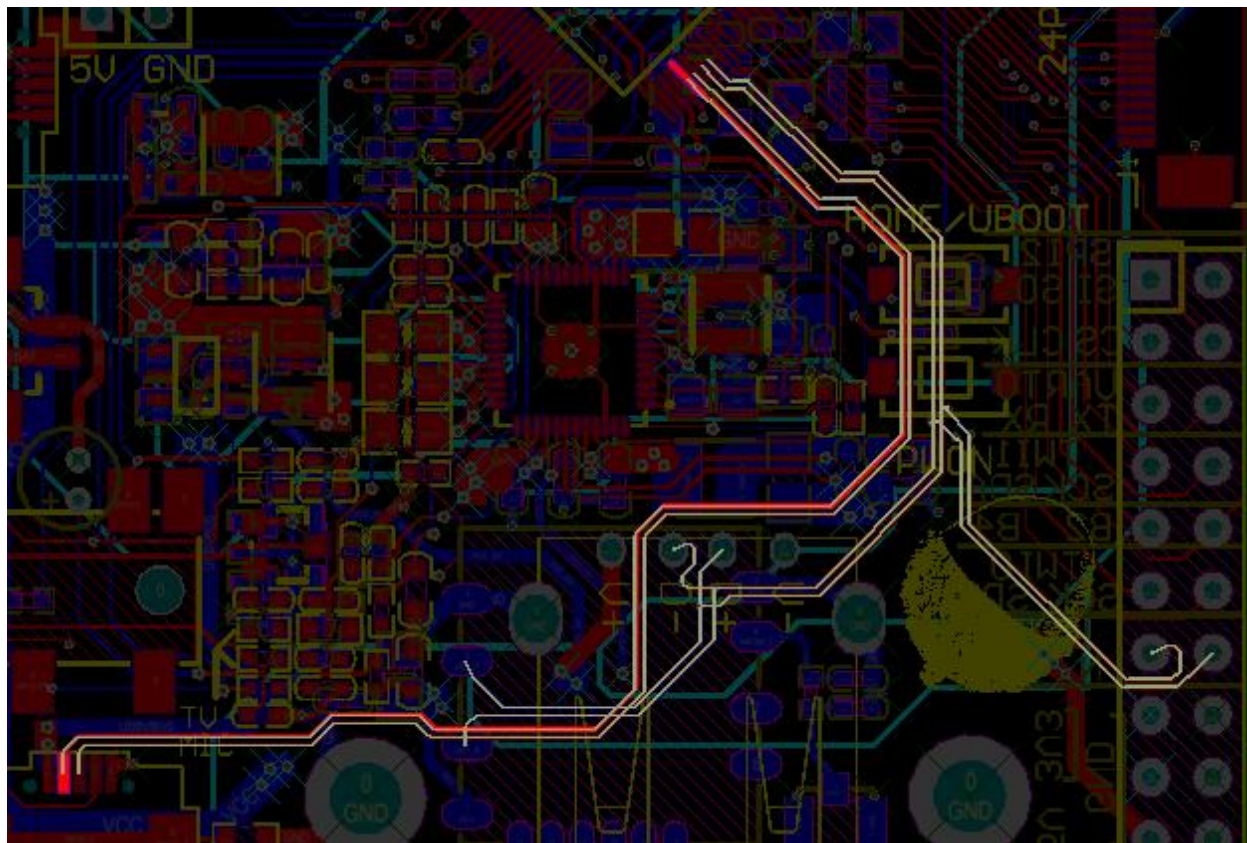
SD信号线的等长控制,相对于DDR3宽松很多,数据线参考时钟线尽量等长,误差在 $\pm 300\text{mil}$ 时钟线和其它线之间距离要满足3W原则



如果需要满足ESD要求，则 需要在TF卡接口处信号线加多个ESD二极管

2.9.3 二.USB布线

usb是差分信号线，USB2.0 的带宽**480Mbps**，比较高速， 需要严格控制等长和阻抗。 USB信号的时钟周期约2ns，波长约 $150 \times 2 = 300\text{mm}$ ，波长的 $1/10 = 30\text{mm}$ 从荔枝板的布局来看，usb0的长度会达到50mm左右，所以对于USB的布线需要考虑阻抗匹配。使用si9000计算可得，差分100欧阻抗时，需要**8mil**走线，**5mil**线距。 USB走线的等长也需要在10mil以内。



如果需要满足ESD要求，则 需要在usb接口处信号线加多个ESD二极管

2.9.4 三.WIFI模块的天线布线

本来想直接在板上直接画上PCB天线，但由于两边有定位孔在，估计效果会不太好，所以就放了个IPEX座子的封装上去。

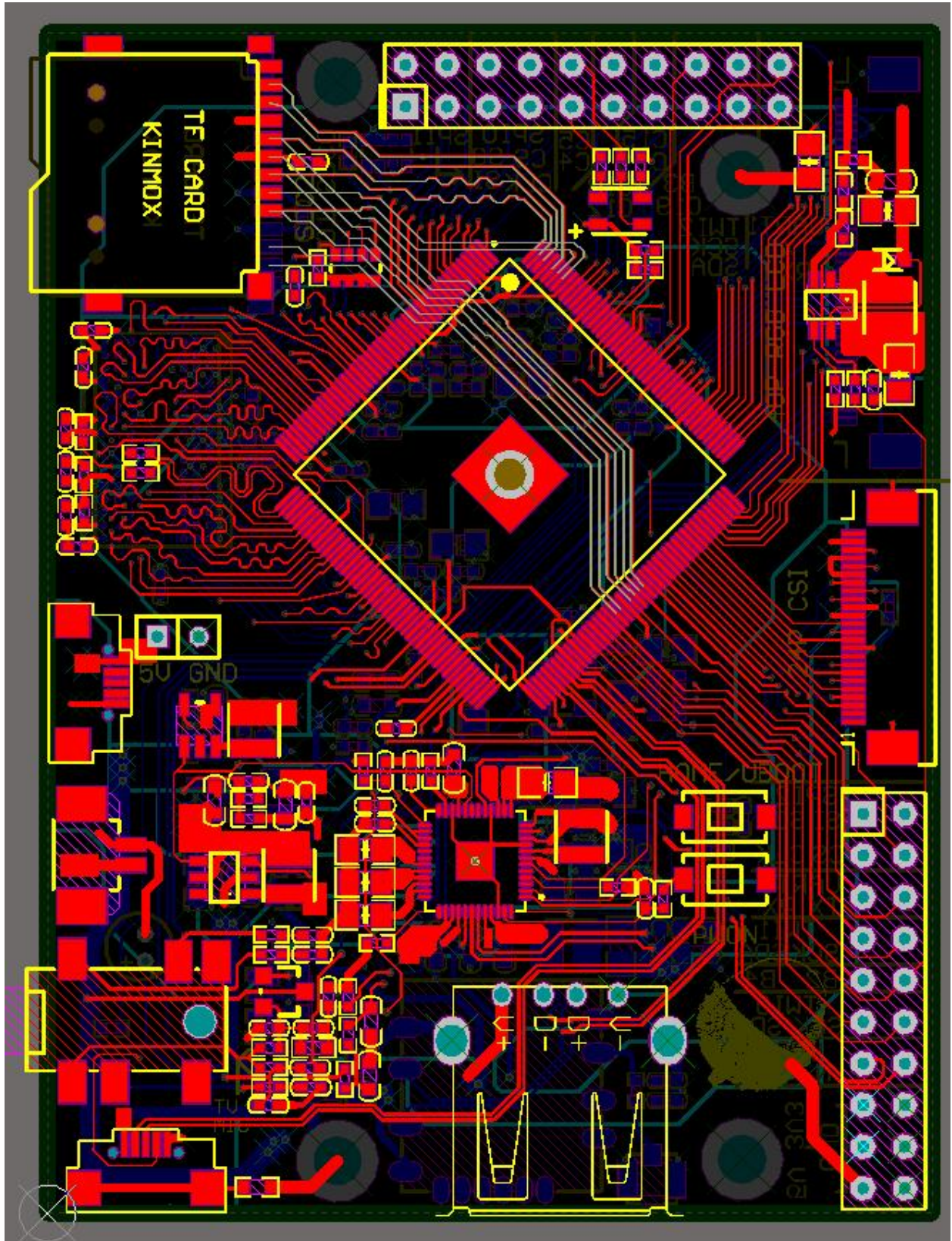
WIFI是2.4G射频信号，周期约0.4ns，波长 $150 \times 0.4 = 60\text{mm}$ ，波长的 $1/10 = 6\text{mm}$ 从wifi模块的布局上看，模块到IPEX座距离差不多超过了6mm，所以需要考虑下阻抗匹配 使用si9000计算阻抗得，单端50欧姆阻抗时，需要13mil走线

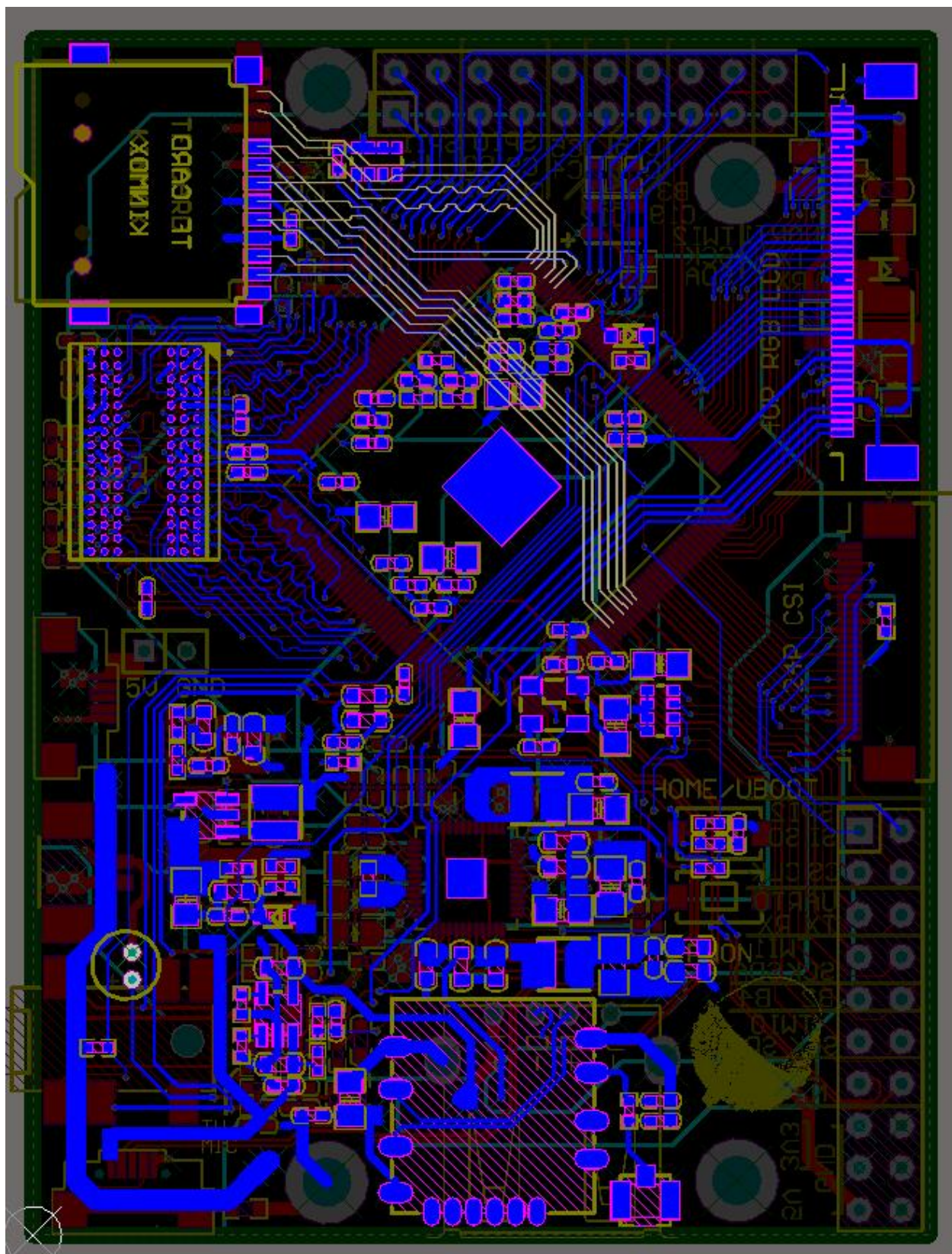
在天线引脚和IPEX座之间还需要加上一个pi型匹配电路，用于额外的阻抗微调。

2.9.5 四.其余走线布线

其余走线，如摄像头，RGB LCD接口等，时钟速率都在100MHZ以下，属于较低速的信号，基本上布通就行。布线时可以适当调整元件布局使得整体更方便布线。

2.9.6 五.最终布线





3.1 样板焊接备料

3.1.1 .背景

本文所属目录层次为:

- > 1. 硬件设计
 - > 3. 样板焊接
 - > 1. 样板焊接备料

主要介绍荔枝板的样板焊接之前的准备工作，PCB打样，bom生成备料，钢网准备。

3.1.2 一.PCB打样

在 之前的PCB参数确认 一节中就选用了华强PCB，在华强PCB的官网直接下单即可

| | |
|--------------|------------------------|
| 订单类型: 样板 | 快递费(元): 8 |
| 发货时间: 正常4-5天 | 网络支付(元): 326.50 |

发票: 普票

抬头: 

快递公司: 优速快递(免费) [查看物流](#)

收货人: 

收货人手机: 

收货人地址: 

工艺详情:

| | | | |
|---------|-------------|------|--------|
| 板子层数 | 4 | 板子厚度 | 1.6 mm |
| 最小线宽/线距 | 4.0/4.0 mil | 过孔 | 0.25mm |
| 铜箔厚度 | 1oz | 内层铜厚 | 0.5oz |
| BGA | 0.35 | 半孔 | 无 |
| 盲埋孔 | 无 | 阻抗 | 无 |
| 阻焊颜色 | 绿色 | 阻焊覆盖 | 过孔盖油 |
| 字符颜色 | 白色 | 焊盘喷镀 | 有铅喷锡 |

订单详情:

| | | | |
|------|------------|------|--------|
| 板子长度 | 7.2 厘米 | 板子宽度 | 5.4 厘米 |
| 订单面积 | 0.03888平方米 | 板子数量 | 10 片 |
| 发货时间 | 正常4-5天 | 板子重量 | 0.1240 |
| 交货形式 | 1 X 1 pcs | 测试方式 | 样品免费测试 |
| 拼版个数 | 1 | 工艺边 | 无 |
| 快递方式 | 优速快递(免费) | 发票 | 普票 |

326.5 = 200样板费 + 50过孔费 + 50BGA费 + 税费 周五下的单, 周一就收到了, 速度还是挺快的~

3.1.3 二.钢网准备

华强PCB上貌似没有钢网选项，就直接在嘉立创上下单了钢网

您的位置：开激光钢网>>查看钢网订单信息

| | | | | | | | | | | | |
|---|-----------------------|-------------|-----------|-------|----|----|---|----|---|------|------|
| 下单日期： | 2016-07-25 17:19:50 | 客户编号： | | | | | | | | | |
| 资料名称： | lichee | pcb文件： | | | | | | | | | |
| 购买数量： | 1 个 | 订单方式： | 钢网单独下单 | | | | | | | | |
| 钢网用途： | 锡膏网 | 订单编号： | | | | | | | | | |
| 规格长： | 47.0 厘米 | 规格宽： | 37.0 厘米 | | | | | | | | |
| 有效面积（长）： | 29.0 厘米 | 有效面积（宽）： | 19.0 厘米 | | | | | | | | |
| MARK要求： | 不需要 | 厚度： | 0 mm | | | | | | | | |
| 快递代收单价： | 90.00 元/个 | 网络支付单价： | 50.00 元/个 | | | | | | | | |
| 抛光工艺： | 打磨抛光 | 抛光工艺费用： | 0 元 | | | | | | | | |
| 快递公司： | 加运美 毛重:1.5KG (运费:8 元) | 付款方式： | 支付宝支付 | | | | | | | | |
| 需要发票： | 机打普票 (累计开票, 1000元起开) | 是否与pcb一起发货： | 否 | | | | | | | | |
| 下单联系人： | | 下单联系人手机： | | | | | | | | | |
| 收货人： | | 收货人手机号码： | | | | | | | | | |
| 收货人地址： | | | | | | | | | | | |
| 送货单及收据： | 需要 | | | | | | | | | | |
| 是否绑定当天其他钢网订单一起发货： | 需要绑定发货 | | | | | | | | | | |
| 收据抬头： | | | | | | | | | | | |
| 是否自动扣款确认订单： | 客户自己确认订单 | | | | | | | | | | |
| 其它备注： | | | | | | | | | | | |
| 本单业务联系人 | 罗先生 | 联系电话 | | | | | | | | | |
| <p>总价格为：63 元</p> <table> <tr> <td>网络支付价</td><td>55</td></tr> <tr> <td>税费</td><td>5</td></tr> <tr> <td>运费</td><td>8</td></tr> <tr> <td>运费税金</td><td>1.00</td></tr> </table> | | | | 网络支付价 | 55 | 税费 | 5 | 运费 | 8 | 运费税金 | 1.00 |
| 网络支付价 | 55 | | | | | | | | | | |
| 税费 | 5 | | | | | | | | | | |
| 运费 | 8 | | | | | | | | | | |
| 运费税金 | 1.00 | | | | | | | | | | |
| 审单结果： | | | | | | | | | | | |

嘉立创的钢网还是比较贴心的，之前PCB打样回来就发现有个小问题，PMU的底部焊盘忘开窗了，钢网回来却发现嘉立创发现了这个问题并自动修正了。

3.1.4 三.bom生成及采购

使用AD的BOM生成工具导出bom到xlsx，然后就可以按照BOM列表采购。电子料样品我一般在优信买，比较全，价格也可以。

在采购的时候发现有些E24的阻值买样品不好买，可以稍微调整下阻值到E12，方便采购。还有一点就是，10uF的贴片电容，特别是小封装，大电压的贴片电容会很贵，所以在之前PCB布板时就应该尽量选择大封装的。

在采购中也发现0.03欧的精密采样电阻比较难买，特别是0805小封装的，大多都是1206，2010的，这也给我们一个提醒，就是在设计PCB的时候一定要确认物料的采购情况。

3.2 样板元件焊接

3.2.1 .背景

本文所属目录层次为：

```
-> 1. 硬件设计
    -> 3. 样板焊接
        -> 2. 样板元件焊接
```

主要介绍荔枝板的样板元件焊接

3.2.2 一. 外发PCB焊接

外发样板焊接是最轻松的方式，目前一款PCB样板焊接费用是300起步，价格是比较合理的，只是来回快递会花费不少时间。最近嘉立创还提供了SMT服务，可以贴常用的阻容元件，工程费只要100，在嘉立创打样的PCB可以试试，不过只能单面贴片。小批量生产时，只要提供好产测设备，SMT厂也会帮你做好PCB的检测（当然是要加钱的），这个在后面的小批量生产一节中再详述。如果有样板焊接或者SMT小批量生产需求的也可以联系我 O(∩_∩)O~

3.2.3 二. 手工样板焊接

有时候由于时间关系，或者个人DIY兴趣，可以进行手工样板焊接，下面介绍常用元器件的手工焊接方法

分立元件及大引脚间距芯片

少量的此类元件可以直接使用镊子夹持摆放，手工使用烙铁焊接 焊小封装元件如0402时，利用放大镜台灯

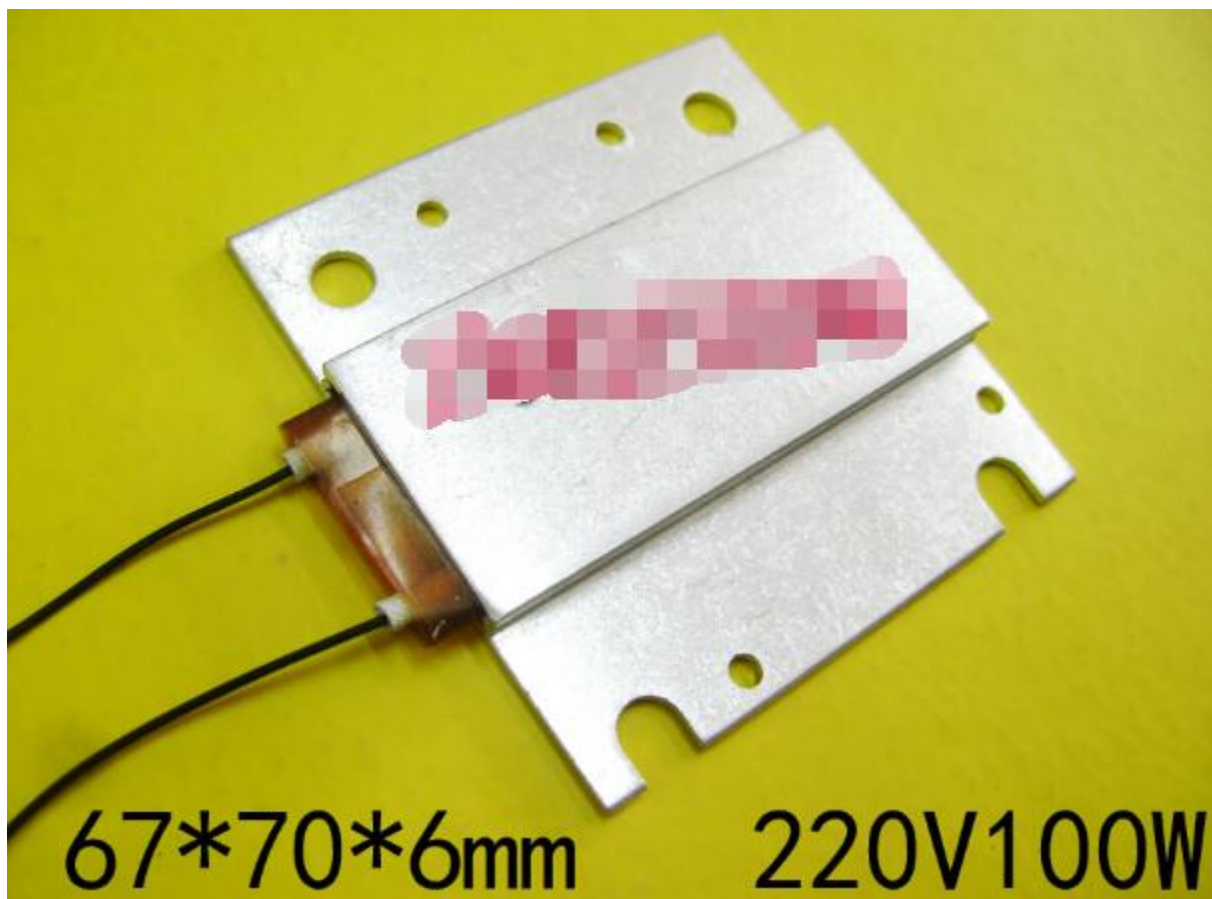


就会轻松很多

但是如果需要焊接很多分立元件的话，手焊就很痛苦了，这时候推荐使用手工贴片焊接 钢网目前大概是40~50元每款，在嘉立创打样的话可以一同购买方便焊接 锡膏建议使用针筒式的，易于保存



加热焊接 可以使用小型的回流焊机，或者直接用热风枪吹。如果是单面元件，强烈推荐使用PTC恒温加热板焊接，比热风枪快很多，价格也在10元以内



密引脚芯片

像A13这样的eLQFP封装芯片，引脚间距特别密（不到0.5mm），那么使用拖焊是最合适的

http://blog.sina.com.cn/s/blog_6566538d0100qjei.html

BGA 芯片焊接

对于像DDR之类的BGA芯片，其实只要涂好助焊膏，摆放齐位置，使用热风枪或者PTC都能很好地焊接上。

http://blog.163.com/mcu_expert/blog/static/1312451532010921112128366/

如果有个数字摄像头观察焊接，那会更直观:

http://v.youku.com/v_show/id_XMTYyNTI5MjQyNA==.html

4.1 使用git进行项目管理

- 一、git与github简介
 - 二、环境配置
 - 三、Git入门准备(以荔枝派库为例)
 - 四、git工作流程简介
 - 五、具体操作
 - 六、更多
-

4.1.1 一、Git与Github简介

* Git是一种版本控制系统,是一种记录若干文件内容变化,以便将来查阅特定版本修订情况的系统。它具有极为丰富的命令集,对内部系统提供了高级操作和完全访问。Git诞生于2005年,由Linux开源社区(特别是Linux的缔造者Linus Torvalds)开发。

Git 的特点: — 支持离线开发, 离线仓库(Repository) — 强大的分支功能, 适合多个独立开发者协作 — 速度块

* Github是一个网站, 给用户提供了git服务。这样你就不用自己部署git系统, 直接用注册个账号, 用他们提供的git服务就可以。所以只要到www.github.com申请一个github帐号, 就可以免费使用git服务。

Github将每一个项目称作一个“仓库”(repository),在仓库里,你可以自由地添加、同步、删除文件,而且可以多人协作对一个仓库中的文件进行修改。横向上, github采用工作流的方式,你的本地仓库由git维护的三棵“树”组成。第一个是你的工作目录,它持有实际文件;第二个是暂存区(Index),它像个缓存区域,临时保存你的改动;最后是HEAD,它指向你最后一次提交的结果;纵向上, github采用主干-分支的

流程控制方式，采用多分支实现单人多作和多人协作，可以方便地找回任何一个修改节点的记录。本文主要介绍如何使用git进行合作项目管理，希望之前没有接触过git和github的朋友可以通过这篇介绍对如何使用git有一定的了解。另外，我们使用命令行的git操作方式，所以没接触过的朋友需要先学会使用简单的命令行操作；如果需要像本文一样进行Markdown文本格式编辑，需要使用相应的Markdown格式；涉及到在vim编辑器中进行代码或文本的修改，需要了解一些vim相关的使用命令。这些我就不再一一叙述，如果还不会的可以自行查阅网上相关资料进行学习。下面正式发车：

4.1.2 二、环境配置

- 第一次接触git和github的朋友，需要先配置环境。首先，到<https://git-scm.com/download>，根据自己的操作系统选择下载对应的客户端（一般linux已经预安装）。下载后安装、打开，进入git（命令行界面）。
- 然后，我在自己电脑的E盘建立了一个专门用于git操作文件的文件夹，这个文件夹就相当于github的本地站点，比如我建了e盘的github/my_site文件夹。在命令行中，去到对应的文件夹：

```
姜朝峰@DESKTOP-J5JHQ29 MINGW64 ~  
$ cd e:/github/my_site
```

于是打开了根目录，输入ls命令，该目录下还没有文件：

```
姜朝峰@DESKTOP-J5JHQ29 MINGW64 /e/github/my_site  
$ ls
```

图例2.2

- 接着，我们要建立本地库和github上对应代码库的连接，对应到群主的github代码库中，就需要进行下面的设置。在本地创建ssh key，建立和github服务器的连接：

```
$ ssh-keygen -t rsa -C "your_email@youremail.com"
```

后面的your_email@youremail.com改为在github上注册的邮箱（如图），之后会要求确认路径和输入密码，我们默认一路回车就行。

图例2.3

成功的话会在“C:\Users\XXX”下生成.ssh文件夹，进去并打开id_rsa.pub，复制里面的key。

图例2.4

回到浏览器打开github，进入Settings(配置)，左边选择“SSH and GPG Keys”，点击“New SSH Key”，随便填一下title，粘贴生成的key。

图例2.5

- 为了验证是否成功，在git bash下输入：
-

```
$ ssh -T git@github.com
```

如果出现下图所示的情况，这就表示已成功连上github。如果要多人协作对这个库进行修改，就需要把合作者的SSH KEY都添加到账户列表中。

图例2.6

- 在我们能用git工作之前，我们需要还做个一次性的用户配置。为了git能跟踪到谁做了修改，我们需要设置用户名和账户。发送下面的命令，相应地替换掉其中的“your_username”和“your_email@server.com”，改成自己的信息：

```
git config --global user.name "your_username"
git config --global user.email your_email@server.com
```

- 设置好之后，再输入初始化命令，就把当前目录作为git根目录，创建了一个本地仓库：

```
git init
```

图例2.7

最后出现“Initialized empty Git repository in E:/github/my_site/.git/”的指令，说明git的初始化配置成功。Git会在my_site文件夹内创建一个名为.git的隐藏文件夹，那就是你的本地代码仓库。如果不进行初始设置，你就无法提交任何东西。设置好之后，下一次打开可以直接工作，不用再设置一次。不过要是换了电脑，那就需要再设置一遍。

我们希望为荔枝Pi的github项目库增加或修改内容，这就涉及到多人一起在github上的开发。一般有三种方法：

- * 第一种：把各位的公钥加到该项目的公钥列表里；
- * 第二种：在github建一个organization，然后建一个team，把大家加到team并把项目放到team下；
- * 第三种：就是你要修改的人fork，然后给群主发pull request，等群主通过。

第一种上面的环境配置中已经谈到，第二种可以自行到github了解，这里推荐第三种方式。

- 到此，就完成了git的环境配置，可以开始具体的工作流程。

4.1.3 三、Git入门准备(以荔枝派为例)

提醒：下面有的操作要在命令行界面，有的要在浏览器中到github主页鼠标操作，注意切换。

1. 复制别人的代码库(fork)

如果想参与到对别人代码库内容的增加或修改中，可以先fork再pull request。fork就是复制别人这个项目代码库到自己帐号下；pull request后面会谈到。比如群主A有一代码库a(也就是荔枝派的github主页)，用浏览器打开：

图例3.1

你叫做B，看到这个觉得不错，所以就fork(点击图中右上角的fork图标)到自己的代码库中，现在暂且称为代码库b。这时，你查看自己的profile下面的就会多出了一个illichee的代码库。a和b的修改互不干预，大家可以随意修改自己的代码库b，群主的a不会受影响。另外，旁边的“Star”表示持续关注别人项目更新，“Watch”则是设置接收邮件提醒。



图例3.2

2. 复制远程代码库到本地(clone)

但现在，代码库b还是存放在github服务器上，你只能在浏览器中修改，不能本地修改。所以你想把这个远程代码库整个下载到自己的电脑上再修改的话，你需要拷贝(clone)它。发送(注意，此时要在根目录下)：

```
git clone git@github.com:your_username/name_of_remote_repository.git
```

得到如图:

图例3.3

果然在这台电脑本地库目录下多了一个“ilichee”的文件夹:

图例3.4

这样你就把代码库**b**都下载到本地了。以后，可以用相同的方法在不同的电脑上、由不同的人一起完成一个代码库。另外，如果已经在本地的项目上工作了，只是想从远程代码库上取得它最新的版本，那我移动到项目的根目录下(注意，不在根目录下无法获取)，并发送:

```
git pull first master
```

3. 推送到远程代码库(push)

在第一次你想推送一个本地代码库到远程代码库时，你需要把它添加到你的项目配置里。像这样做:

```
git remote add origin https://your_username@bitbucket.org/your_username/name_of_
↪remote_repository.git
```

图例3.5

注意这里的“origin”只是一个习惯。它是你的远程代码库的别名，但是你可以用其他任何你喜欢的词。你甚至可以有多个远程代码库，你只需要给它们起不同的别名。之后，推送你的本地代码库的主干分支到你的远程代码库:

```
git push origin master
```

下图是对该库文件修改后提交并推送的示例:

图例3.6

4.同步代码库

假设按照上述的流程复制别人的代码库到自己的GitHub(fork)，别人的代码库已经更新，自己复制的代码库有没有新代码可以提交，这时候就需要同步复制别人的源代码库。进入clone到本地的代码库文件夹中，然后增加源分支地址到你项目远程分支列表中;

```
git remote add source https://github.com/originalL_owner/original_repo.git
```

这里的source是自己给源仓库起的名字，可以起自己喜欢的。

fetch源分支到本地

```
git fetch source
```

然后，切换到本地 master 分支:

```
git checkout master
```

合并两个版本的代码

```
git merge source/master
```


把合并后的代码push到你的Github项目上去

```
git push origin master
```

图例4.1 这样 fork 源仓库、fork 仓库副本 和 local 仓库实现了同步更新

5. 推送请求(pull request)

假设就按照上述流程，初始clone代码库到本地主机上，B就可以尽情修改code(branch、commit、merge、push)，每次 B push 更新，都只会更新自己的代码库b，并不会影响到A的代码库a。如果哪天B觉得自己新增加的内容很不，可以帮助到群主A，想跟A分享，那就可以发一个pull request，问问A要不要这一份。下面我用两个帐号模拟fork和pull request：我用帐号B fork了A的一个代码库a，成为了自己的代码库b，修改提交之后，然后想把改动增加到库a里，于是点击“Create pull request”：

图例2.7

A帐号马上收到了邮件通知，当然github主页也有timeline消息通知。

图例3.8

A收到这则pull request之后，如果得ok，用线上merge，就代码库b合并到代码库a上。

图例3.9

4.1.4 四、Git工作流程简介

- [x] 建立目录
- [x] 加载文件(stage)
- [x] 提交文件(commit)
- [x] 创建分支(branch)
- [x] 合并分支(merge)
- [x] 丢弃分支
- [x] 删除分支(delete)
- [x] 回滚到之前的提交状态(back)
- [x] 复制和推送请求(fork/pull request)
- [x] 推送到远程代码库(push)
- [x] 取得远程代码库的拷贝(pull)
- [x] 更多.....

首先，你需要去github上将别人的代码库复制到自己的库中，也可以先在自己的电脑上建立目录，相当于自己的个人站点，对应着github服务器上的站点。接下来，你可以在这个目录下进行创建新文件、修改文件等操作，修改完之后，就需要加载文件，也就是把修改过的文件正式放进自己的项目里。而提交文件则是把文件上传到github服务器，进行文件同步。接着，我想加入新的代码进行测试，那么就可以创建分支。建立分支是你创建代码的独立版本的动作，独立于你的主干分支，相当于进入了另外一条“时间河流”。默认情况下，每次你提交到Git的文件都会被储存到主干分支。满意的话将新分支和主干合并分支，不满意的话“时光倒流”回到主干，并删除分支。如果发现新增的改动有问题，可以回滚到之前的提交状态。终于，本地的修改结束，想要上传到github，就可以推送到远程代码库；相反，如果想把github仓库里的文件下载到本地，可以取得远程代码库的拷贝。

4.1.5 五、具体操作

下面，我们以windows上的操作步骤为例进行示范（linux类似）。

1. 加载文件

我用vim指令创建并保存了text1.txt和markd.md两个文本文件，但它们还没有放入中。现在加载(stage)所有项目文件仓库(repository)，输入：

```
git add .
```

最后的“.”符号的意思是“所有文件、文件夹和子文件夹”。

图例2.1

假如我们只想要把特定文件添加到源代码控制中去，我们可以指定要添加的文件。比如，我用vim创建了两个markdown文件——README1.md、README2.md，发送：

```
git add README1.md, README2.md
```

图例2.2

2. 提交文件

现在，我们想要提交已加载（staged）的文件。提交文件时，我们需要给这个状态一个备注，所以我们提交我们的文件时，总是附带着有意义的注释，描述了它们现在的状态。比如用“first commit”来作为第一个提交的注释，如下：

```
git commit -m "first commit"
```

图例2.3

但发现至提交了3个文件，检查可知我把其中一个文件README2.md打成了README2.MD，修改后重复上面的操作即可。这样我们就用“first commit”代表这个时间点提交的修改，后面还可以再回滚到这个提交状态。提交之后，如果你想查看现在已加载、未加载的文件及状态，可以用以下命令：

```
git status
```

图例2.4

3. 创建分支

如果要写或者测试新的部分，还不想直接加入到程序中时，就可以创建分支，就像暂时踏入另一条时间的河流一样。建立分支是你创建代码的独立版本，独立于你的主干分支。默认情况下，每次你提交到Git的文件都会被储存到“master（主干）”分支。创建并同时切换到你新建的分支，发送：

```
git checkout -b new_feature
```

图例2.5

或者，你可以先创建一个分支然后手动切换，输入命令：

```
git branch new_featuregit checkout new_feature
```

要看你现在项目下所有的分支，输入如下指令：

```
git branch
```

图例2.6

现在你可以在你的项目上无所顾忌地做任何你想做的：任何时候，你都可以回到你创建分支前的状态。你同时可以有多个分支，甚至可以从一个分支上再创建一个分支。

4. 合并分支

现在我在新的分支new_feature做一些修改，比如删除了README2.md。

```
姜朝峰@DESKTOP-J5JHQ29 MINGW64 /e/github/my_site (new_feature)
$ rm README2.md

姜朝峰@DESKTOP-J5JHQ29 MINGW64 /e/github/my_site (new_feature)
$ git commit -m "third commit"
On branch new_feature
Changes not staged for commit:
  deleted:    README2.md

no changes added to commit

姜朝峰@DESKTOP-J5JHQ29 MINGW64 /e/github/my_site (new_feature)
$ git rm README2.md
rm 'README2.md'

姜朝峰@DESKTOP-J5JHQ29 MINGW64 /e/github/my_site (new_feature)
$ git commit -m "third commit"
[new_feature 8a82d25] third commit
1 file changed, 1 deletion(-)
delete mode 100644 README2.md
```

图例2.7

在这个分支上修改得差不多的时候，如果想要把这个分支加回到主干(master)上，首先需要加载(stage)并且提交(commit)你的文件：

```
git add .git commit -m "third commit"
```

然后你移到主干分支：

```
git checkout master
```

最后像这样合并：

```
git merge new_feature
```

```

姜朝峰@DESKTOP-J5JHQ29 MINGW64 /e/github/my_site (new_feature)
$ rm README2.md
姜朝峰@DESKTOP-J5JHQ29 MINGW64 /e/github/my_site (new_feature)
$ git commit -m "third commit"
On branch new_feature
Changes not staged for commit:
      deleted:    README2.md

no changes added to commit

姜朝峰@DESKTOP-J5JHQ29 MINGW64 /e/github/my_site (new_feature)
$ git rm README2.md
rm 'README2.md'

姜朝峰@DESKTOP-J5JHQ29 MINGW64 /e/github/my_site (new_feature)
$ git commit -m "third commit"
[new_feature 8a82d25] third commit
1 file changed, 1 deletion(-)
delete mode 100644 README2.md

```

图

例2.8

此时，主干分支就和新功能分支会变成一样的了。

5. 丢弃分支

相反，如果你打算丢弃你在分支里做的修改，你首先需要加载你的文件并且在分支里提交：

```
git add .git commit -m "feature to be discarded"
```

然后，你移到主干分支：

```
git checkout master
```

图例2.9

现在，你的代码处于你创建分支之前的状态了。

6. 删除分支

如果你要把你的分支合并到主干分支，从主干（master）分支上发送：

```
git branch -d new_feature
```

图例2.10

发现"new_feature"这个分支确实被删除了。假如修改已经合并了，它只会删除分支。假如分支没有合并，你会得到一个错误信息。删除一个未合并的分支（通常你不想保留的修改），你需要发送一样的命令附带一个大写D。意思是“强制删除分支，无论如何我不想要它了”。

```
git branch -D new_feature
```


7. 回滚到之前的提交状态

在某些时候，你可能想要回到之前的代码版本。首先，你需要找到你想回到哪个版本。要看所有的完成了的提交，发送：

```
git log
```

图例2.11

这会输出你的提交的历史记录。

如果你想回到“second commit”这个提交，简单地用提交的ID做签出(checkout)（可以只用到ID开头的9个字符）

```
git checkout cc87a2d42
```

图例2.12

你也可以签出到一个新的分支，像这样：

```
git checkout -b my_previous_version cc87a2d42
```

但是，要注意保持分支的清晰，太多的分支会导致整个仓库的混乱，让整个项目失去控制。

4.1.6 六、更多

- 如果对命令行的git使用方式很不适应，还可以下载git的**GUI版本**，比如GitHub Desktop和Gitbox(见网站<https://git-scm.com/downloads/guis>)，GUI版本的git确实要比命令行更直观更容易入门。
- 当然，还有比这些更多的**Git**的相关知识，本文也是我参考网上的资料进行整理、修改和实践得到的。如果有疑惑或者需要更进一步了解的地方，可以自己搜索“**Git**”相关资料或者和我联系，推荐廖雪峰的官方网站/Git教程，相信你会获得更多收获。

4.2 开发环境搭建

4.2.1 .背景

本文所属目录层次为：

```
-> 2. 系统移植
    -> 1. 基于linux-sunxi SDK的系统移植
        -> 1. 开发环境搭建
```

主要介绍linux-sunxi SDK的开发环境搭建 本系列教程使用启发式编写，在写明操作步骤后会附加一些思考题供读者思考，以*号标记思考题的程度 一星：刚接触linux，纯新手需要思考、搜索解决的问题 二星：熟悉linux使用的朋友可以了解的一些问题，如底层问题，或者sdk相关问题 三星：非常熟悉linux，可以一起探讨的问题

4.2.2 一.准备开发机

与普通单片机开发使用Windows下的可视化IDE，如MDK，IAR等不同，Linux下开发嵌入式程序多使用一套工具链开发，如arm-linux-gnueabi-，arm-linux-gnueabi-，arm-none-linux-gnueabi-等

Note: * 工具链的具体定义？前面列出的工具链名字的含义？‘.’分割的各字段的含义？

所以我们需要使用有个Linux开发机来进行开发。具体到全志这颗芯片的SDK，其适合的开发环境是64位linux系统，推荐Ubuntu 1404 64bit系统（桌面版，服务器版均可），预留足够大的硬盘（如果要编译安卓的话，有100G以上最好，单linux的话20G至少）

Note: ** 为什么全志的SDK需要64位系统开发？为什么推荐Ubuntu 1404 64bit系统？

首先我们下载Ubuntu 1404 64bit系统的镜像安装包，对于没有多余实体机的用户可以再下载VMWare等虚拟机软件

Note: * 在哪里可以下载？

然后在实体机或者虚拟机上安装好Ubuntu系统，这样开发机就准备完成了

Note: * 如何安装Ubuntu系统？

实体机上开发可以直接打开终端，虚拟机的话可以在桌面里开终端，或者使用Xshell等终端软件ssh连接。

Note: * 什么叫ssh连接？

Note: * 如何设置虚拟机才能让本机在终端里连接上虚拟机？

4.2.3 二.准备SDK及相关软件

开源的linux-sunxi比较适合初学者学习，所以我们先开始试用linux-sunxi这个sdk 开发主要用到下面几个工程，请先下载到本地(网速慢的话可以从百度网盘下: <http://pan.baidu.com/s/1eSM9uKy>)

```
git clone https://github.com/linux-sunxi/linux-sunxi.git
git clone https://github.com/linux-sunxi/u-boot-sunxi.git
git clone https://github.com/linux-sunxi/sunxi-tools.git
```

<https://github.com/linux-sunxi>下的工程有兴趣也都可以浏览下

Note: * git的基本操作？ 尝试自己建立编辑一个github项目

Note: ** linux-sunxi下的工程大致都是什么功能？

简单说下前面三个工程包含的内容，linux-sunxi是linux内核的工程，u-boot-sunxi就是uboot的工程，sunxi-tools是sunxi芯片相关的一些工具，如fex2bin,bin2fex

Note: * linux内核是什么？ uboot是什么？

因为我们要从头开始移植，所以先从u-boot-sunxi这个工程开始

Tip: ** 可以先浏览下目录，看下应该用什么参数编译

首先我们安装一些依赖包

```
sudo apt-get install git-core gnupg flex bison gperf build-essential zip curl zlib1g-
↳dev libc6-dev \
lib32ncurses5-dev gcc-multilib x11proto-core-dev libx11-dev lib32z1-dev libgl1-mesa-
↳dev \
g++-multilib mingw32 tofrodos python-markdown libxml2-utils
```

```
sudo apt-get install gcc-arm-linux-gnueabi
```

Note: * 什么叫安装依赖？前面安装的这都是啥？

Note: * 安装的时候是不是有个别依赖没装上？怎么解决？

再到本教程所在github工程下的Config_Files里下载荔枝板的配置文件A13-Lichee_defconfig, sun5i-a13-lichee.dts（因为是直播移植，这两个配置文件目前尚未完善，会在以后直播过程中完善），以及修改后的dts的Makefile

拷贝这三个文件到uboot工程的对应位置

```
cp A13-Lichee_defconfig configs/
cp sun5i-a13-lichee.dts arch/arm/dts/
cp Makefile arch/arm/dts/
```

Note: ** 看下这两个文件都是啥配置？

4.2.4 三.初次编译尝试

然后先配置成荔枝派的默认配置

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- A13-Lichee_config
```

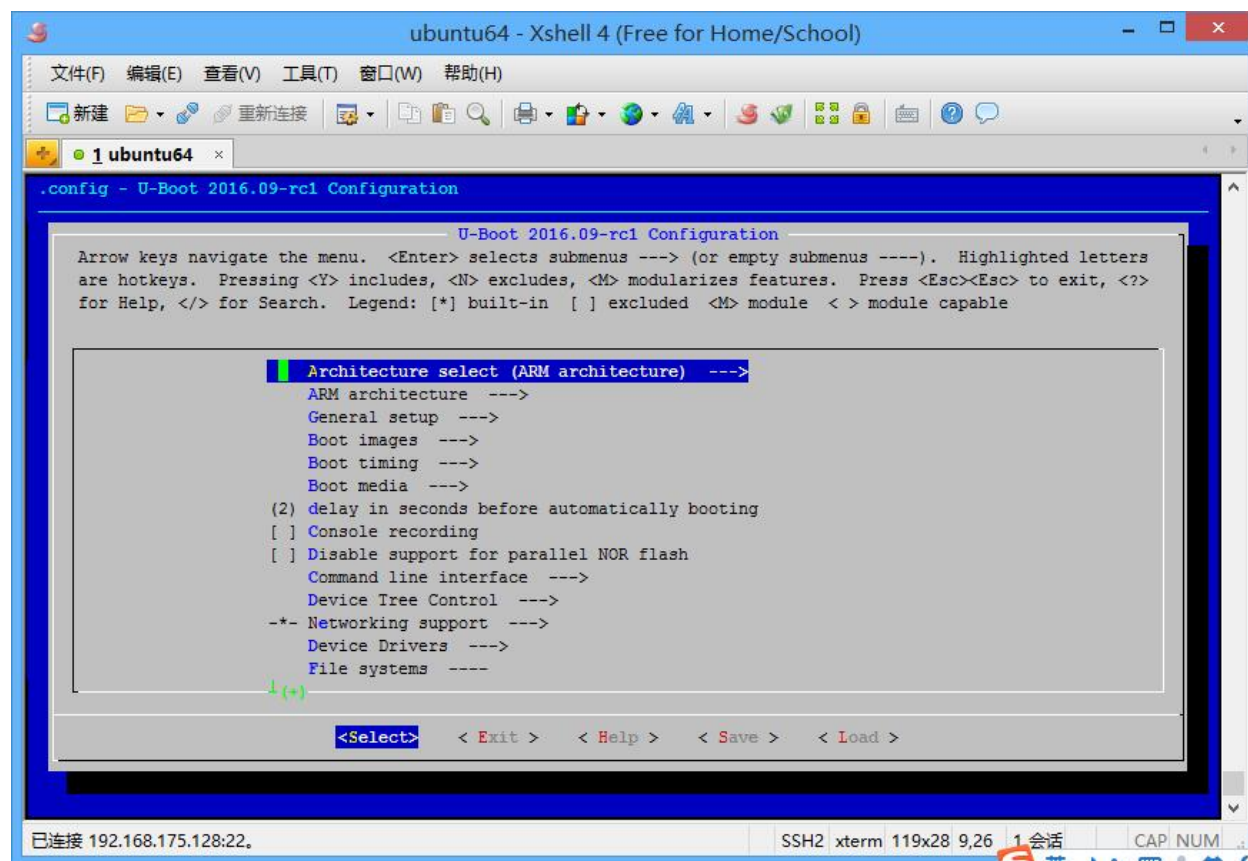
Note: * 这个命令的各部分是什么意思？

Tip: ** 可以浏览下uboot的Make流程

现在可以打开menuconfig配置菜单来看看uboot的一些选项，如果知道那些参数的意思的话也可以试着修改下

```
make ARCH=arm menuconfig
```

Tip: * 熟悉下配置菜单的使用，比如查找选项，查看帮助是按什么键？



浏览好后，先退出配置菜单，使用下面的命令来编译一次试试,注意编译前后目录下多了什么文件(如果提示dtc版本低请自行更新):

```
time make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- 2>&1 | tee build.log
```

Note: * 上面的编译命令是什么意思？

可以看到目录下多了一堆u-boot开头的文件

```
u-boot.cfg u-boot-dtb.bin u-boot.img u-boot.map u-boot.srec
↪ u-boot.sym
u-boot.bin u-boot.dtb u-boot-dtb.img u-boot.lds u-boot-nodtb.bin u-boot-sunxi-
↪ with-spl.bin
```

Note: ** 这堆文件各是什么用途的文件？

我们这里需要的文件是u-boot-sunxi-with-spl.bin，如果你手上有荔枝派的话，可以使用下面的命令(sdX是对应的tf卡读卡器的设备)往tf卡里烧入u-boot-sunxi-with-spl.bin:

```
sudo dd if=u-boot-sunxi-with-spl.bin of=/dev/sdX bs=1024 seek=8
```

Note: * 上面的命令是什么意思？

Note: ** 为什么seek=8？

然后插卡到荔枝派的tf口，上电看看UART1的启动输出

4.2.5 四.结语

开发环境就此搭建好了，也初次尝试编译了下，对于linux熟悉的朋友来说这一节很短 对于linux新手来说，这一节内容 还是挺多的，完全消化还是得多多搜索才行。下节会讲述uboot是怎样一步步启动CPU的。

4.3 u-boot-sunxi走读

4.3.1 .背景

本文所属目录层次为：

```
-> 2.系统移植
    -> 1.基于linux-sunxi SDK的系统移植
        -> 2. u-boot-sunxi走读
```

经过前面一节的开发环境搭建，大家应该能够初次编译uboot了，但是如果没有思考过2星问题的朋友还是对uboot没有什么印象，本节就带大家走读uboot，了解其工程结构和工作原理。

4.3.2 一.u-boot-sunxi目录走读

大致查看下u-boot-sunxi每个目录的文件（仅列出了在调试DDR时比较重要的文件，其它目录请自行打开浏览）

```
.
├── api                //封装一些平台无关的操作，如字符串打印，显示，网络，内存
├── arch                //
│   ├── arm
│   │   ├── cpu
│   │   │   ├── armv7
│   │   │   │   ├── sunxi    //cpu相关的一些操作，如定时器读取
│   │   │   │   └── u-boot-spl.lds //spl的放置方法
│   │   └── dts
│   │       ├── sun5i-a13.dtsi    //sun5i系列芯片的一些配置
│   │       ├── sun5i-a13-lichee.dts
│   │       ├── sun5i-r8-chip.dts
│   │       ├── sun5i-r8.dtsi
│   │       └── sun5i.dtsi
│   ├── lib            //一些库文件
│   └── mach-sunxi
│       ├── board.c        //board_init_f
│       ├── dram_sun4i.c    //ddr的操作，复位，时钟，延时，odt，etc.
│       └── dram_helpers.c  //ddr的设置及读写测试
└── board
```




如果需要移植新的版型，如上文所示，主要修改的就是dts文件和configs目录下的默认配置文件。具体dts文件的语法请搜索相关资料。

4.3.3 二.u-boot-sunxi启动流程走读

要让芯片跑起系统，对其bootloader必须非常了解，不然移植初期出现的问题将无从下手。笔者调试荔枝板的时候犯了个错误，在没有确认DDR正常运行的时候就贸然进行调试，结果出现的问题是如同堆栈溢出，或者指针指飞后的诡异情形，每次出错的位置总是不同，导致多次跟踪徒劳无返。

不过也正是因为犯了前面这个错误，导致笔者翻遍了u-boot-sunxi的代码，特别是SPL部分代码，使得调试稳定DDR后，移植中出现的问题马上就能找到原因，很快完成了bootloader的适配。

由于启动部分内容较多，现将A13的bootloader启动流程分成三个流程图将以展示，可以在github的资源文件目录下找到。

A13整体启动流程：简明介绍A13启动的几个过程
 uboot-sunxi spl部分流程：详细介绍spl部分启动流程
 uboot-sunxi uboot部分流程：详细介绍uboot部分启动流程

流程图只是起到一个路线图的作用，当你不太熟悉启动流程，而启动失败的时候，可以根据流程查看目前的启动进度，以及是在哪里卡住，可以快速定位出错的大致位置。

4.3.4 三.关于A13的裸机用法

群里有朋友问A13的裸机用法，其实在SPL或Uboot下就是A13的裸机使用。SPL下未初始化DDR，只能使用片内48KB SRAM，就完全是普通单片机用法，只是程序是需要从tf卡载入到内存里运行而已。这样使用可以不加DDR，只是内部SRAM较小。Uboot下已经初始化了DDR，而且对多数外设已经有了驱动，使用起来更方便，内存也可以充分利用。（像RTT之类的小型RTOS应该可以直接编译进Uboot了吧。。）

感兴趣的朋友可以尝试下在SPL或者Uboot下编程，可以对启动过程更有了解。

4.3.5 四.小结

对于多数linux开发者来说，此部分内容只作了解即可。此部分内容主要是BSP工程师的工作。下节打算讲下移植中遇到的最大的坑，也就是DDR的调试问题，这是关系到系统能否跑起来的关键问题，也是软硬件结合最紧密的问题，所以单独写一节来总结。

4.4 tf镜像烧录解读

4.4.1 .背景

本文所属目录层次为：

```
-> 2. 系统移植
    -> 1. 基于linux-sunxi SDK的系统移植
        -> 3. tf镜像烧录解读
```

这里对荔枝派的镜像烧录进行解读

4.4.2 一.系统镜像组成

TF卡或者其他存储介质中的系统镜像组成如下：

1. boot部分，含boot0和boot1/uboot
2. linux内核部分，含启动参数和fex配置
3. linux根文件系统

Note: ** 回想下在启动过程中这三个部分是如何串联起来的？

4.4.3 二.烧录boot部分镜像

将前面编译出的u-boot-sunxi-with-spl.bin烧录到tf 8KB偏移处（sdb换成你自己的tf卡读卡器设备名）

```
sudo dd if=u-boot-sunxi-with-spl.bin of=/dev/sdb bs=1024 seek=8
sudo sync
```

Tip: ** 学习下dd命令的详细用法

4.4.4 三.烧录linux内核镜像

需要准备uImage，script.bin,boot.scr三个文件

Note: ** 回想下这三个文件的作用？

1. uImage

(a) 使用linux-sunxi编译后可得到uImage

2. script.bin

(a) 使用下面命令将fex转为bin

(b) `fex2bin a13-lichee.fex script.bin`

(c) 需要根据使用的摄像头型号修改对应的fex字段

3. boot.scr

(a) 使用下面命令将boot.cmd转为boot.scr

(b) `mkimage -C none -A arm -T script -d boot.cmd boot.scr`

Tip: ** 查看fex文件和boot.cmd文件，理解各字段含义

这三个文件在uboot启动时会在第一分区搜索，所以需要新建一个TF卡分区（FAT格式），需要使用fdisk命令。

Tip: *请自行学习fdisk命令

如果原来tf卡已经有分区表，请先清除分区。

烧录相关的脚本已上传至“资源文件”目录，可以查看其中的脚本命令。

一般来说，第一分区设为16MB大小就足够了。

然后格式化为FAT格式：`sudo mkfs.vfat /dev/sdb1`

最后放入前面提到的三个文件即可。

4.4.5 四.烧录linux根文件系统

linux内核在启动时候会根据启动参数中的root参数来获得根文件系统分区号，然后加载之。

一般就按顺序把根文件系统设为第二分区，使用EXT3或者EXT4文件系统。

整个根文件系统有两种打包方式，dd方式和tar方式。

dd方式是直接备份整个分区信息，tar方式是直接打包整个分区。

dd出来的分区镜像可以直接使用mount命令在本机上加载查看，烧写速度也稍快些，所以荔枝派使用dd出来的镜像发布。

不过由于dd是备份了整个分区信息，所以连分区大小都是一致的，如果tf较大，可以自行扩容分区和文件系统

Tip: * 自学linux下扩容分区和文件系统的方法

```
dd if=fs2.img of=/dev/sdb2 bs=64K
sync
```

4.4.6 五.启动尝试

按上述方法准备好tf卡后，就可以尝试启动了！把tf卡插入背面卡槽，有屏幕的插入屏幕（注意方向），有摄像头的插入摄像头（注意方向），再连上UART1（默认系统串口），上电。正常情况就可以启动到debian系统了~

Tip: ** 给出的镜像出厂测试镜像，屏蔽了桌面系统，想开机启动桌面系统的可以尝试修改下

提供的镜像有两个用户lichee和root，密码均为lichee。在github“资源文件”目录下，有烧写相关的文件和脚本（脚本还未整理，比较乱，不过也可以参考使用）

4.5 linux-sunxi简明编译指南

1. 从本项目的ConfigFiles里下载a13_linux_defconfig配置文件，该文件是a13的默认配置文件
2. 将上面的配置文件拷贝到arch/arm/configs/目录下，写入默认配置：`make ARCH=arm a13_linux_defconfig`
3. 可以视情况修改一下menuconfig配置：`make ARCH=arm menuconfig`
4. 编译linux内核镜像（注意下面的j24是你的开发机有多少线程就写多少）`make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- -j24 uImage`
5. 编译安装内核模块

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- -j24 INSTALL_MOD_PATH=out modules
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- -j24 INSTALL_MOD_PATH=out modules_
↪install
```

可以在linux-sunxi/out/lib/modules/3.4.104下找到编译好的内核模块

4.6 LCD的参数适配

LCD的参数和其他外设一样在fex文件里修改

像分辨率和画布偏移之类的，都在以下字段：

```
lcd_x = 800
lcd_y = 480
lcd_dclk_freq = 33
lcd_hv_hspw = 23
lcd_hbp = 46
lcd_ht = 1055
lcd_hv_vspw = 1
lcd_vbp = 10
lcd_vt = 1050
```

例如改成1024*768

```
lcd_x = 1024
lcd_y = 768
lcd_dclk_freq = 65
lcd_hv_vspw = 6
```

```

lcd_hv_hspw =      136
lcd_hbp =          180
lcd_ht =          1344
lcd_vbp =           29
lcd_vt =          1612

```

在超分辨率使用时，注意以下代码（具体查看源码）的限制

```

/* hv panel, CPU panel and ttl panel */
if (info->lcd_if == 0 || info->lcd_if == 1 || info->lcd_if == 2) {
    /* MHz */
    if (lcd_dclk_freq > 2000000 && lcd_dclk_freq <= 297000000) {
        /* divider for dclk in tcon0 */
        *divider = 297000000 / (lcd_dclk_freq);
        pll_freq = lcd_dclk_freq * (*divider);
    } else {
        return -1;
    }
}
}

```

以下是未整理的移植时的手记，可以参考：

drivers/video/sunxi_display.c video_hw_init

drivers/video/videomodes.c

```

int video_get_params (struct ctfb_res_modes *pPar, char *penv)
mode: 0~9    640*480~1920*1200,默认0
x:
y:
refresh:
le:left_margin
ri:right_margin
up:upper_margin
lo:lower_margin
hs:hsync_len
vs:vsync_len
sync:sync
    #define FB_SYNC_HOR_HIGH_ACT    1        /* horizontal sync high active */
    #define FB_SYNC_VERT_HIGH_ACT    2        /* vertical sync high active */
    #define FB_SYNC_EXT              4        /* external sync */
    #define FB_SYNC_COMP_HIGH_ACT    8        /* composite sync high active */
    #define FB_SYNC_BROADCAST        16       /* broadcast video timings */
        /* vttotal = 144d/288n/576i => PAL */
        /* vttotal = 121d/242n/484i => NTSC */
    #define FB_SYNC_ON_GREEN          32       /* sync on green */
vmode:vmode
    #define FB_VMODE_NONINTERLACED    0        /* non interlaced */
    #define FB_VMODE_INTERLACED      1        /* interlaced */
    #define FB_VMODE_DOUBLE           2        /* double scan */
    #define FB_VMODE_ODD_FLD_FIRST   4        /* interlaced: top line first */
pclk:pixclock
pclk_khz:pixclock_khz
depth: 色深

bp = mode->hsync_len + mode->left_margin;
    10                      (43-10)=33
total = mode->xres + mode->right_margin + bp;
    480                      8

```

43


```

bp = mode->vsync_len + mode->upper_margin;
    10                                (12-10)=2
total = mode->yres + mode->lower_margin + bp;
    272                                4                                12
x:480,y:272,depth:18,pclk_khz:9000,le:33,ri:8,up:2,lo:4,hs:10,vs:10,sync:0,vmode:0
x:800,y:480,depth:18,pclk_khz:33000,le:100,ri:170,up:35,lo:2,hs:10,vs:10,sync:0,
↪vmode:0

Setting up a 480x272 lcd console (overscan 0x0) 背光闪了下
drivers/video/sunxi_display.c
    video_hw_init
        sunxi_mode_set(mode, fb_dma_addr);
            static void sunxi_lcdc_backlight_enable(void)
hsync + hbp(back porch) + X len + hfp(front porch)
hb(H blanking)

```

hbp相当于画面在画布中的偏移,

屏幕在左边和上边都有一部分不能显示的等待区域, 要调整hbp移出这部分,

否则会有一部分在屏幕外不显示

hsync值只要大于1即可

5.1 安卓编译简记

qq群里 @随风破浪 编译安卓一星期未果，我之前的开发机上的lichee目录又不小心被覆盖了，所以抽空简单记录下如何在原始sdk的基础上 修改适配，成功编译出可以跑在荔枝派上的安卓系统。

5.1.1 1. 编译环境准备

开发机系统要求

编译安卓需要64位linux系统，推荐**ubuntu 1404或者1604**，开发机至少需要**4GB内存**，**40GB硬盘**；推荐**8GB内存**，**100GB硬盘**以上。以下是开发过程中目录大小示意：

```
zp@ubuntu:~/develop$ du -lh --max-depth=1 v1.5.0/
26G v1.5.0/android
6.8G v1.5.0/lichee
33G v1.5.0/

zp@ubuntu:~/develop$ du -lh --max-depth=1 a13_android4.1_v1.2/
5.2G a13_android4.1_v1.2/lichee
23G a13_android4.1_v1.2/android4.1
28G a13_android4.1_v1.2/
```

开发机的CPU配置尽量高，笔者12线程并行编译，配以ssd，首次编译需要45分钟
网友双线程编译，耗时4小时左右。

开发机软件环境

首先安装开发所需要的软件包和一些库。下面的一些安装包可能有些过时的，遇到版本问题请自行解决。

在线安装 JDK6.0

```
sudo add-apt-repository "deb http://archive.canonical.com/ lucid partner"
sudo add-apt-repository ppa:ferramroberto/java
sudo apt-get update
sudo apt-get install sun-java6-jdk
sudo update-alternatives --c onfig java
```

在线安装编译库

```
sudo apt-get install git-core gnupg flex bison gperf build-essential zip curl zlibg-
↳ dev libc6-dev
lib32ncurses5-dev ia32-libs x11proto-core-dev libx11-dev lib32z1-dev libgl1-mesa-dev
g++-multilib mingw32 tofrodos python-markdown libxml2-utils
```

SDK下载

链接: <http://pan.baidu.com/s/1c4gztvE>

在SDK目录下 下载以下两个文件

```
lichee-4.1-v1.2.tar.gz
android4.1-v1.2.tar.gz
```

这是安卓4.1的SDK, 前面的是4.2的SDK,

我手上的4.2 SDK的mali驱动ko和应用层库so版本对不上, 换了从4.1里抠出的对应版本的so后, 可以勉强启动系统, 但是会不定时地出现缓冲队列满的问题导致ANR, 所以暂时无法使用

在开发机上新建目录, 如a13_android4.1 然后把两个sdk拷入, 解压

把lichee对应的目录名改成lichee (即去掉版本号), 目录如下所示

```
zp@ubuntu:~/develop/a13_android4.1_v1.2$ ls
android4.1  lichee
```

5.1.2 2. lichee编译

lichee是安卓系统的linux内核部分, 需要首先编译, 进入lichee目录执行:

```
./build.sh -p a13_nuclear -k 3.0
./build.sh pack
```

编译错误解析 编译过程中会出错, 每个人由于其开发环境以及执行步骤的不同, 错误都可能不同, 下面以我编译时遇到的错误为例进行解析

- mali驱动编译出错

```
make: Entering directory `/home/zp/develop/a13_android4.1_v1.2/lichee/linux-3.0/
↳ modules/mali'
/home/zp/develop/a13_android4.1_v1.2/lichee/linux-3.0
make -C DX910-SW-99002-r3p1-01rel0/driver/src/devicedrv/ump CONFIG=ca8-virtex820-m400-
↳ 1 BUILD=release KDIR=/home/zp/develop/a13_android4.1_v1.2/lichee/linux-3.0
```

```
make[1]: Entering directory `/home/zp/develop/a13_android4.1_v1.2/lichee/linux-3.0/
↳modules/mali/DX910-SW-99002-r3p1-01rel0/driver/src/devicedrv/ump'
make -C /home/zp/develop/a13_android4.1_v1.2/lichee/linux-3.0 M=/home/zp/develop/a13_
↳android4.1_v1.2/lichee/linux-3.0/modules/mali/DX910-SW-99002-r3p1-01rel0/driver/src/
↳devicedrv/ump modules
make[2]: Entering directory `/home/zp/develop/a13_android4.1_v1.2/lichee/linux-3.0'
  CC [M] /home/zp/develop/a13_android4.1_v1.2/lichee/linux-3.0/modules/mali/DX910-SW-
↳99002-r3p1-01rel0/driver/src/devicedrv/ump/common/ump_kernel_common.o
arm-none-linux-gnueabi-gcc: directory: No such file or directory
arm-none-linux-gnueabi-gcc: directory": No such file or directory
<command-line>:0:16: warning: missing terminating " character
make[3]: *** [/home/zp/develop/a13_android4.1_v1.2/lichee/linux-3.0/modules/mali/
↳DX910-SW-99002-r3p1-01rel0/driver/src/devicedrv/ump/common/ump_kernel_common.o]
↳Error 1
make[2]: *** [_module_/home/zp/develop/a13_android4.1_v1.2/lichee/linux-3.0/modules/
↳mali/DX910-SW-99002-r3p1-01rel0/driver/src/devicedrv/ump] Error 2
make[2]: Leaving directory `/home/zp/develop/a13_android4.1_v1.2/lichee/linux-3.0'
make[1]: *** [all] Error 2
make[1]: Leaving directory `/home/zp/develop/a13_android4.1_v1.2/lichee/linux-3.0/
↳modules/mali/DX910-SW-99002-r3p1-01rel0/driver/src/devicedrv/ump'
make: *** [build] Error 2
make: Leaving directory `/home/zp/develop/a13_android4.1_v1.2/lichee/linux-3.0/
↳modules/mali'

real    2m35.400s
user    15m36.416s
sys     0m35.740s
```

进入排除可知是脚本里检查了SVN版本的问题，我们直接忽略SVN版本，将linux-3.0/modules/mali/DX910-SW-99002-r3p1-01rel0/driver/src/devicedrv/ump 目录下的SVN_REV直接赋值为0（在Kbuild和Makefile.common中）

重新编译，仍然是类似的错误，只是换了个目录，于是直接在mali驱动目录下全局搜索SVN_REV，将赋值的地方直接改为0

再重新编译成功

```
mkdir: created directory `/home/zp/develop/a13_android4.1_v1.2/lichee/out'
`/home/zp/develop/a13_android4.1_v1.2/lichee/u-boot/u-boot.bin' -> `/home/zp/develop/
↳a13_android4.1_v1.2/lichee/out/android/u-boot.bin'
#####
#          compile success          #
#####

real    0m51.177s
user    1m1.324s
sys     0m4.736s
```

5.1.3 3. 安卓编译

进入安卓目录，执行以下命令序列：

```
source build/envsetup.sh          #导入一些环境变量
lunch                               #选择板型，直接选择10. nuclear_evb-eng，以后也直接在上面改
extract-bsp                         #将lichee中编译得到的内核镜像和模块拷贝过来
time make -j12 2>&1 | tee log.txt   #编译system.img，这里-j12换成你电脑的线程数（核
数*2）。12线程耗时约
```



```
pack      #打包成刷机镜像
```

编译system.img成功的提示:

```
Creating filesystem with parameters:
  Size: 536870912
  Block size: 4096
  Blocks per group: 32768
  Inodes per group: 8192
  Inode size: 256
  Journal blocks: 2048
  Label:
  Blocks: 131072
  Block groups: 4
  Reserved block group size: 31
Created filesystem with 1443/32768 inodes and 102168/131072 blocks
+ '[' 0 -ne 0 ']'
Running: mkuserimg.sh -s out/target/product/nuclear-evb/system out/target/product/
↳nuclear-evb/obj/PACKAGING/systemimage_intermediates/system.img ext4 system 536870912
Install system fs image: out/target/product/nuclear-evb/system.img
out/target/product/nuclear-evb/system.img+out/target/product/nuclear-evb/obj/
↳PACKAGING/recovery_patch_intermediates/recovery_from_boot.p maxsize=548110464
↳blocksize=4224 total=412271984 reserve=5537664

real    36m1.948s
user    350m4.772s
sys     14m16.936s
```

pack打包成功的提示:

```
/home/zp/develop/a13_android4.1_v1.2/lichee/tools/pack/pctools/linux/eDragonEx//home/
↳zp/develop/a13_android4.1_v1.2/lichee/tools/pack/outBuildImg 0
Dragon execute image.cfg SUCCESS !
CPlugin Free lib
CPlugin Free lib
-----image is at-----

/home/zp/develop/a13_android4.1_v1.2/lichee/tools/pack/sun5i_android_a13-evb.img
```

编译错误解析

1. 注意如果编译时突然中断, 则可能出现文件截断的情况, 需要先清空out下子目录中的的obj目录, 才好继续编译

例如下面错误就是:

```
target thumb C++: camera. <= device/softwinner/common/hardware/camera/
↳HALCameraFactory.cpp
In file included from device/softwinner/common/hardware/camera/CameraHardwareDevice.
↳h:26:0,
                        from device/softwinner/common/hardware/camera/HALCameraFactory.
↳cpp:30:
device/softwinner/common/hardware/camera/CameraHardware.h:29:23: fatal error:
↳videodev2.h: No such file or directory
compilation terminated.
```

```
make: *** [out/target/product/generic/obj/SHARED_LIBRARIES/camera._intermediates/
↳HALCameraFactory.o] Error 1
make: *** Waiting for unfinished jobs....
```

1. 在pack时遇到以下错误（虽然不导致编译失败，但是启动时会造成错误）

```
fail:/home/zp/develop/a13_android4.1_v1.2/lichee/tools/pack/out/bootfs/sprite 0
disk : c
CopyRootToFS(/home/zp/develop/a13_android4.1_v1.2/lichee/tools/pack/out/bootfs)
```

检查可以发现是lichee/tools/pack/pack中的打包脚本的这句出错：fsbuild bootfs.ini split_xxxx.fex

这个fsbuild在运行时貌似无法打包bootfs的二级目录，不知为何。。有些人却没有反馈有这个问题，应该是个人的环境不同。

由于fsbuild是二进制提供的，只能通过其作用揣测实际功能，经过试验，上面那句可以用以下脚本代替，基本作用就是构造一个fat文件系统镜像。

```
dd if=/dev/zero of=bootfs.fex bs=1M count=12
mkfs.vfat bootfs.fex
sudo mount bootfs.fex /mnt
sudo cp -r bootfs/* /mnt
sync
sudo umount /mnt
cat split_xxxx.fex >> bootfs.fex
```

1. 分区大小的问题（虽然不导致编译失败，但是启动时会造成错误） 如果往system.img里加入了太多东西，可能导致超出了默认的分區大小（512M），则需要修改默认分区规划

验证system.img的实际大小：simg2img system.img system.bin

如果生成的system.bin大小刚好是512M，并且可以挂载，那就是正常的。如果大小超过了512M，或者挂载时出错，那么就需要修正分区配置。

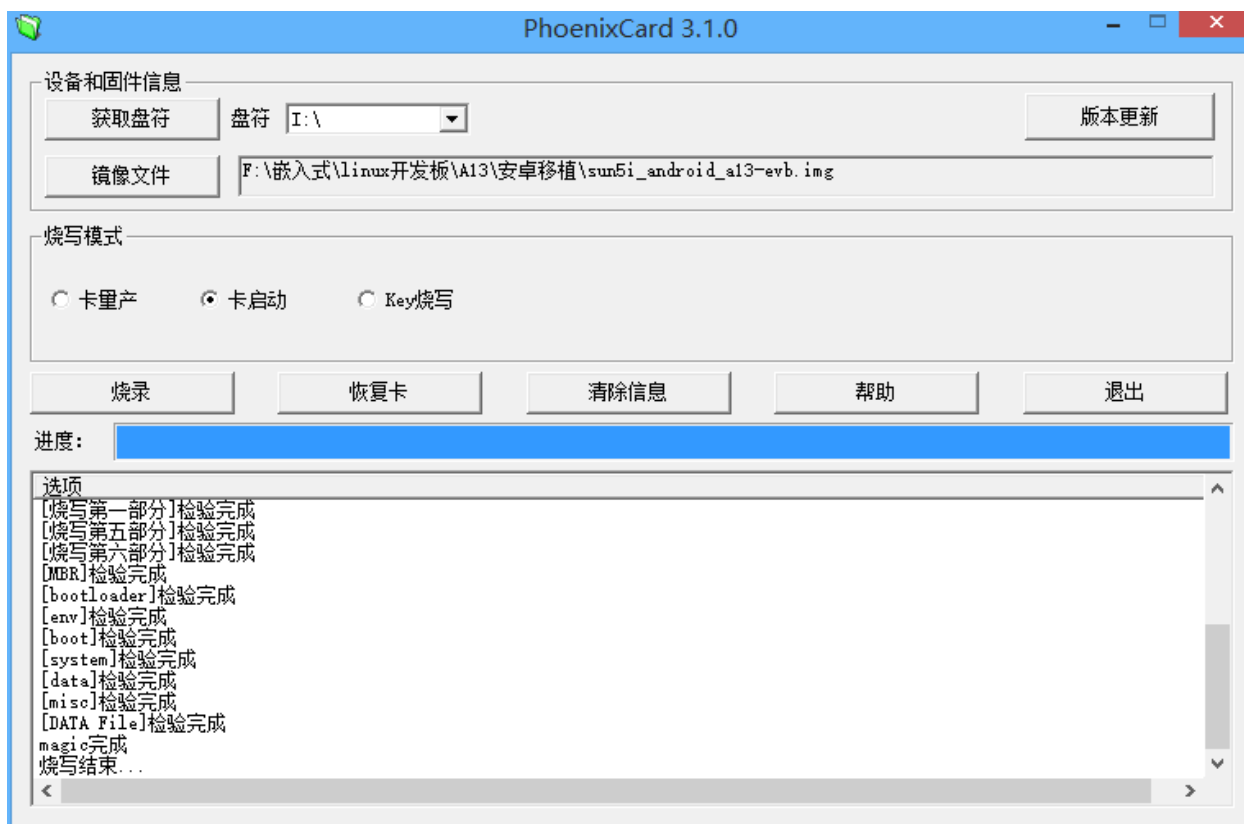
配置文件在lichee/tools/pack/chips/sun5i/configs/android/a13-evb/sys_config.fex
修改大小即可。

```
;----->nandd, android real rootfs
[partition3]
    class_name = DISK
    name       = system
    size_hi    = 0
    size_lo    = 524288
    user_type  = 1
    ro        = 0
```

5.1.4 4. 镜像下载

前面pack打包完成后，会生成sun5i_android_a13-evb.img，这就是刷机镜像

镜像烧录工具是网盘中的PhoenixCard，打开后选择tf卡对应盘符，以及待烧录镜像，选择卡启动，进行烧录



常见烧录错误

偶尔会出现‘处理出错’的提示，这时候一般只要重新烧录即可

5.1.5 5. 启动系统的适配过程

通过以上步骤编译的img还是不能在荔枝派上启动的，前面只能算是编译过一遍evb的安卓，需要在荔枝派上启动，还得进行一系列适配工作。

第一次启动（boot1跳转失败）

系统默认串口为UART1，接串口查看信息：

```
dram size =512
0xffffffff,0xffffffff
super_standby_flag = 0
HELLO! BOOT0 is starting!
boot0 version : 1.5.2
The size of Boot1 is 0x00036000.
Fail in checking boot1.
Ready to disable icache.
Fail in loading Boot1.
Jump to Fel.
```

可见DDR已被识别，但是加载boot1失败。

这其中的原因是DDR虽然被识别，但是参数配置错误，导致运行出错，所以我们需要先配置正确的DDR参数。

DDR参数记录在sys_config1.fex中，这里不过多解释sys_config1.fex的字段，直接使用之前在debian中适配好的fex文件覆盖

```
lichee/tools/pack/chips/sun5i/configs/android/al3-evb/sys_config1.fex
```

然后重新进行安卓编译过程。

编译完成后，可以直接重新烧写完整镜像，也可以只更新uboot和bootfs，后者需要对编译系统比较了解，在此不详细展开。

重新烧录后，可以看到闪过了两个开机画面，说明已经可以启动到linux内核了。

第二次启动（启动介质错误）

按前面修改后，启动会卡在第二张开机画面，查看串口信息：

```
[ 16.009615] init: buffer : /dev/block/nande
[ 16.014564] init: do_umount: /data
[ 16.018071] init: do_umount error = Invalid argument
[ 17.390045] usb 2-1: device not accepting address 2, error -110
[ 17.450200] ehci_irq: port change detect
[ 17.454145] ehci_irq: port change detect
[ 21.007468] init: buffer : /dev/block/nandh
[ 21.012161] init: do_umount: /cache
[ 21.015755] init: do_umount error = Invalid argument
[ 21.022007] init: open device error :No such file or directory
[ 31.008863] init: buffer : /dev/block/nandi
[ 31.013556] init: do_umount: /databk
[ 31.017237] init: do_umount error = Invalid argument
[ 31.029268] init: cannot find '/system/bin/sh', disabling 'console'
[ 31.035766] init: cannot find '/system/bin/servicemanager', disabling
↪ 'servicemanager'
[ 31.043742] init: cannot find '/system/bin/vold', disabling 'vold'
[ 31.049968] init: cannot find '/system/bin/netd', disabling 'netd'
[ 31.056192] init: cannot find '/system/bin/debuggerd', disabling 'debuggerd'
[ 31.063282] init: cannot find '/system/bin/rild', disabling 'ril-daemon'
[ 31.069987] init: cannot find '/system/bin/surfaceflinger', disabling
↪ 'surfaceflinger'
[ 31.077943] init: cannot find '/system/bin/app_process', disabling 'zygote'
[ 31.084939] init: cannot find '/system/bin/drmserver', disabling 'drm'
[ 31.091502] init: cannot find '/system/bin/mediaserver', disabling 'media'
[ 31.098387] init: cannot find '/system/bin/dbus-daemon', disabling 'dbus'
[ 31.105211] init: cannot find '/system/bin/installd', disabling 'installd'
[ 31.112125] init: cannot find '/system/etc/install-recovery.sh', disabling 'flash_
↪ recovery'
[ 31.120541] init: cannot find '/system/bin/keystore', disabling 'keystore'
[ 31.127432] init: cannot find '/system/bin/u3gmonitor', disabling 'u3gmonitor'
[ 31.135780] init: cannot find '/system/bin/sh', disabling 'console'
```

可见是tf卡启动与nand启动的不同造成linux内核挂载根文件系统出错，所以需要修改开机启动脚本，即*.rc文件

```
device/softwinner/nuclear-evb/init.sun5i.rc
device/softwinner/nuclear-evb/ueventd.sun5i.rc
```

将其中的nandX按下面对应关系修改

```
nanda ---- mmcblk0p2
nandb ---- mmcblk0p5
nandc ---- mmcblk0p6
nandd ---- mmcblk0p7
nande ---- mmcblk0p8
nandf ---- mmcblk0p9
nandg ---- mmcblk0p10
nandh ---- mmcblk0p11
nandi ---- mmcblk0p12
```

其中有一句挂载剩余空间的替换也对应替换:

```
#format_userdata /dev/block/nandj NUCLEAR
exec /system/bin/busybox mount -t vfat /dev/block/mmcblk0p1 /mnt/sdcard
```

再修改vold.fstab, 开机挂载方式, 前面两行改成以下, 即sd卡0作为sdcard, sd卡2作为外置sd卡

| | | | | |
|-----------|--------|-------------|------|--|
| dev_mount | sdcard | /mnt/sdcard | auto | /devices/platform/sunxi-mmc.0/mmc_host |
| dev_mount | extsd | /mnt/extsd | auto | /devices/platform/sunxi-mmc.2/mmc_host |

如果需要修改成sd卡2启动, 则还需要修改uboot代码

lichee\u-boot\include\configs中sun5i.a13.h文件中的卡启动定义

```
#define CONFIG_MMC_SUNXI_SLOT 2
```

最后修改uboot的环境变量env.cfg(lichee/tools/pack/chips/sun5i/configs/android/default/env.cfg)

将mmc作为启动介质

```
bootdelay=1
#default bootcmd, will change at runtime according to key press
bootcmd=run setargs_mmc boot_normal#default nand boot
#kernel command arguments
console=ttyS0,115200
nand_root=/dev/system
mmc_root=/dev/mmcblk0p7
init=/init
loglevel=6
#set kernel cmdline if boot.img or recovery.img has no cmdline we will use this
setargs_nand=setenv bootargs console=${console} root=${nand_root} init=${init}
↪loglevel=${loglevel} partitions=${partitions}
setargs_mmc=setenv bootargs console=${console} root=${mmc_root} init=${init} loglevel=
↪${loglevel} partitions=${partitions}
#nand command syntax: sunxi_flash read address partition_name read_bytes
#0x40007800 = 0x40008000(kernel entry) - 0x800(boot.img header 2k)
boot_normal=sunxi_flash read 40007800 boot;boota 40007800
boot_recovery=sunxi_flash read 40007800 recovery;boota 40007800
boot_fastboot=fastboot
#recovery key
recovery_key_value_max=0x13
recovery_key_value_min=0x10
#fastboot key
fastboot_key_value_max=0x8
fastboot_key_value_min=0x2
```

修改完成后重新编译下安卓

为了省事, 可以不下载整个镜像, 只更新boot.fex和env.fex (lichee/tools/pack/out下)

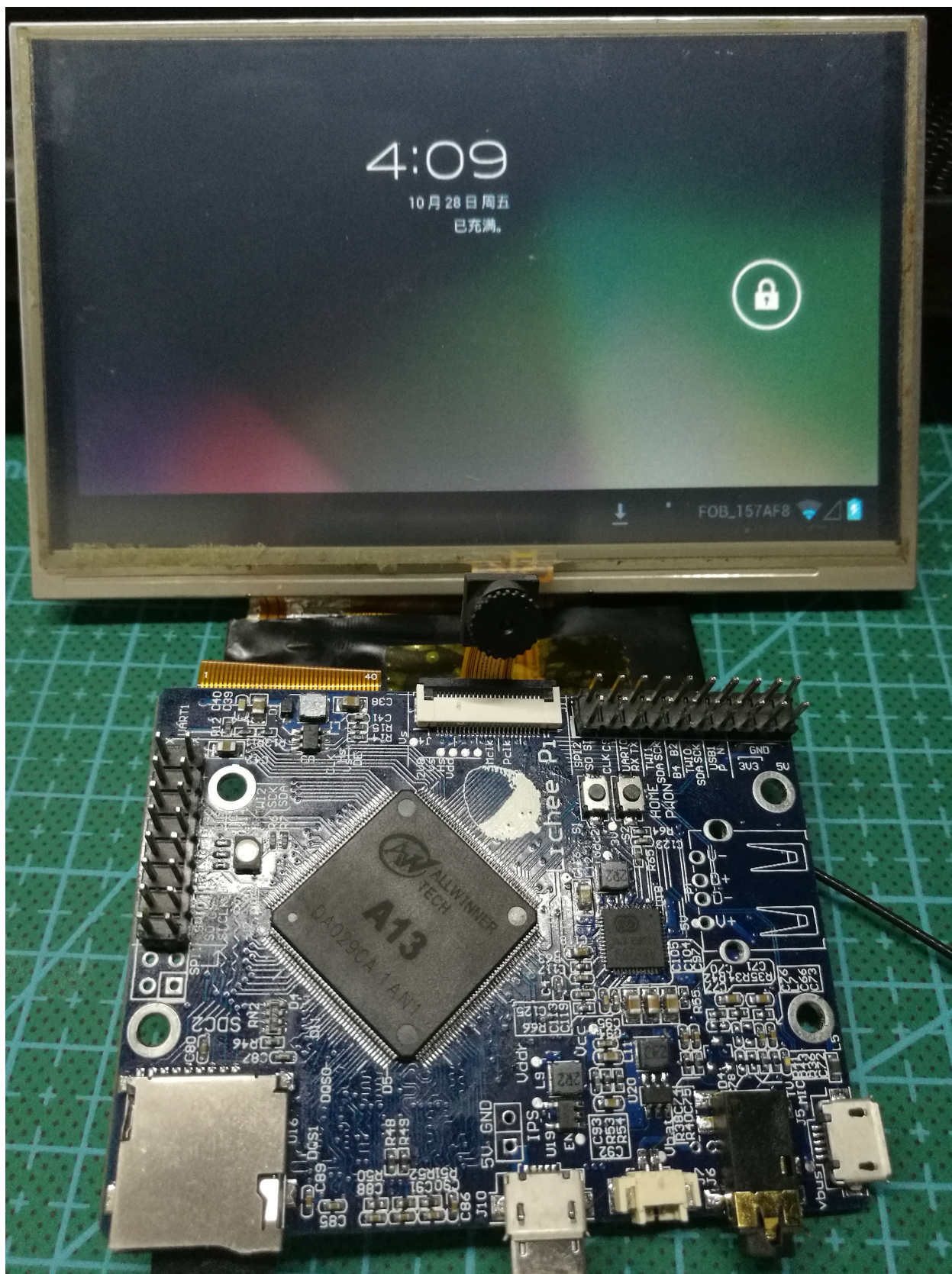
在linux下，使用fdisk -l查看tf卡分区：

| Device | Boot | Start | End | Blocks | Id | System |
|------------|------|---------|----------|----------|----|-----------------------------------|
| /dev/sdb1 | | 3448832 | 15595518 | 6073343+ | b | W95 FAT32 //剩余空间作为u盘 |
| /dev/sdb2 | * | 73728 | 106495 | 16384 | 6 | FAT16 //bootfs,含uboot |
| /dev/sdb3 | | 1 | 3448832 | 1724416 | 85 | Linux extended |
| /dev/sdb5 | | 106496 | 139263 | 16384 | 83 | Linux //env.fex |
| /dev/sdb6 | | 139264 | 204799 | 32768 | 83 | Linux //boot.fex,含linux内核及ramdisk |
| /dev/sdb7 | | 204800 | 1253375 | 524288 | 83 | Linux //system分区 |
| /dev/sdb8 | | 1253376 | 2301951 | 524288 | 83 | Linux //data分区 |
| /dev/sdb9 | | 2301952 | 2334719 | 16384 | 83 | Linux //misc分区 |
| /dev/sdb10 | | 2334720 | 2400255 | 32768 | 83 | Linux //recovery分区 |
| /dev/sdb11 | | 2400256 | 2924543 | 262144 | 83 | Linux //cache分区 |
| /dev/sdb12 | | 2924544 | 3448831 | 262144 | 83 | Linux //datbak分区 |

所以只需更新两个分区：

```
root@ubuntu64:/home/zp# dd if=env.fex of=/dev/sdb5
256+0 records in
256+0 records out
131072 bytes (131 kB) copied, 0.097856 s, 1.3 MB/s
root@ubuntu64:/home/zp# dd if=boot.fex of=/dev/sdb6
19916+0 records in
19916+0 records out
10196992 bytes (10 MB) copied, 8.54376 s, 1.2 MB/s
root@ubuntu64:/home/zp# sync
```

更新之后即可成功进入到安卓系统~



虽然此时可以进入系统，但是很多外设不能运行，这些外设的驱动适配见后面的外设适配解析。

5.2 codec适配及Audio系统解析

5.2.1 codec适配

启动后耳机及mic无反应，开始绕了个大圈看了遍安卓的Audio系统，结果从最上层查到最底层，发现原因不过是fex没写全而已：

```
[audio_para]
audio_used = 1
audio_playback_used = 1
capture_used = 1
audio_lr_change      = 0
audio_pa_ctrl        = port:PG0<1><default><default><0>
```

加上audio_pa_ctrl的配置即可，这里使用了空闲引脚PG0，实际上没有接PA，也可以直接改驱动源码来支持不接PA，不过比较麻烦，就直接拿个空闲引脚凑数了。

5.2.2 Audio系统解析

暂待更新

5.3 RTL8723BU wifi模块适配

5.3.1 linux驱动添加

安卓下使能外设驱动都需要先适配好linux下驱动，所以首先来编译linux驱动模块

1. 拷贝官方驱动（github的资源文件目录下下载）到lichee的linux驱动目录下

```
zp@ubuntu:~/develop/al3_android4.1_v1.2/lichee$ ls linux-3.0/drivers/net/wireless/
↪ rtl8723bu/
clean core hal ifcfg-wlan0 include Kconfig Makefile os_dep platform runwpa ↪
↪ wlan0dhcpc
```

1. 配置驱动Makefile以及Kconfig，然后在menuconfig里选中刚加入的驱动

```
CONFIG_PLATFORM_ARM_SUNXI = y
obj-$(CONFIG_RTL8723BU) += rtl8723bu/
source "drivers/net/wireless/rtl8723bu/Kconfig"
<M> Realtek 8723B USB WiFi
```

1. 重新编译lichee，获得内核模块

```
./build.sh -p al3_nuclear -k 3.0
zp@ubuntu:~/develop/al3_android4.1_v1.2/lichee$ ls out/android/lib/modules/3.0.8+/87*
out/android/lib/modules/3.0.8+/8723bu.ko
```

1. 修改wifi驱动，隐藏过多调试信息

前面这样生成的ko带有很多调试信息输出，以至于开启wifi模块时太慢，导致打开wifi超时，所以需要隐藏多余的信息。编辑linux-3.0/drivers/net/wireless/rtl8723bu/include/autoconf.h，去掉DEBUG的相关宏定义。

5.3.2 安卓配置文件

1. 修改板级配置文件device/softwinner/nuclear-evb/BoardConfig.mk

```
BOARD_WIFI_VENDOR := realtek
ifeq ($(BOARD_WIFI_VENDOR), realtek)
    WPA_SUPPLICANT_VERSION := VER_0_8_X
    BOARD_WPA_SUPPLICANT_DRIVER := NL80211
    BOARD_WPA_SUPPLICANT_PRIVATE_LIB := lib_driver_cmd_rtl
    BOARD_HOSTAPD_DRIVER := NL80211
    BOARD_HOSTAPD_PRIVATE_LIB := lib_driver_cmd_rtl

    SW_BOARD_USR_WIFI := rtl8723bu
    BOARD_WLAN_DEVICE := rtl8723bu
endif
```

1. 修改init启动脚本init.sun5i.rc

```
# 1.1 realtek wifi sta service
service wpa_supplicant /system/bin/wpa_supplicant -iwlan0 -Dnl80211 -c/data/misc/wifi/
↪wpa_supplicant.conf -e/data/misc/wifi/entropy.bin
    class main
    socket wpa_wlan0 dgram 660 wifi wifi
    disabled
    oneshot

# 1.2 realtek wifi sta p2p concurrent service
service p2p_supplicant /system/bin/wpa_supplicant \
    -ip2p0 -Dnl80211 -c/data/misc/wifi/p2p_supplicant.conf -e/data/misc/wifi/
↪entropy.bin -N \
    -iwlan0 -Dnl80211 -c/data/misc/wifi/wpa_supplicant.conf
    class main
    socket wpa_wlan0 dgram 660 wifi wifi
    disabled
    oneshot
```

1. 修改安卓板级mk文件nuclear_evb.mk（蓝牙需要，wifi不用改）
2. 修改安卓hardware相关代码hardware/libhardware_legacy/wifi/wifi.c

```
#elif defined RTL_8723BU_WIFI_USED
/* rtl8723bu usb wifi */
#ifndef WIFI_DRIVER_MODULE_PATH
#define WIFI_DRIVER_MODULE_PATH    "/system/vendor/modules/8723bu.ko"
#endif
#ifndef WIFI_DRIVER_MODULE_NAME
#define WIFI_DRIVER_MODULE_NAME    "8723bu"
#endif

#ifndef WIFI_DRIVER_MODULE_ARG
#define WIFI_DRIVER_MODULE_ARG    "ifname=wlan0 if2name=p2p0"
```

1. wifi的安卓mk修改

```
hardware/libhardware_legacy/wifi/Android.mk
ifeq ($(SW_BOARD_USR_WIFI), rtl8723bu)
LOCAL_CFLAGS += -DRTL_8723BU_WIFI_USED
LOCAL_CFLAGS += -DRTL_WIFI_VENDOR
endif
```

1. wpa的安卓mk修改

```
external/wpa_supplicant_8/wpa_supplicant/Android.mk
ifeq ($(SW_BOARD_USR_WIFI), rtl8188eu)
L_CFLAGS += -DCONFIG_WFD
endif
```

1. 重新编译安卓

实际需要更新的就只有libhardware_legacy.so和8723bu.ko，当然也可以打包整个镜像重新烧写。

以局部更新为例，挂载tf卡的QQA，拷贝so到lib/libhardware_legacy.so

拷贝ko到vendor/modules

sync后启动即可

5.4 usb适配简记

5.4.1 usb调试信息太多，删减之

```
[ 232.061924] [sw_hcd]: sw_hcd_urb_dequeue, sw_hcd(df8a28ec, 0x0, 0x3f), urb(dbd34f00,
→ 1514, 0), dev = 3, ep = 2, dir = in
[ 232.071179] sw_hcd_cleanup_urb: qh(0xd0df6e40, 0x2, 0x2), urb(0xdbd34f00, 1514, 0),
→ ep(0xdf8a2a00, 3, 0xd0df6e40, 0x (null))
```

出处在

```
./linux-3.0/drivers/usb/sun5i_usb/hcd/core/sw_hcd_host.c: DMSG_INFO("[sw_hcd]: sw_
→ hcd_urb_dequeue, sw_hcd(%p, 0x%d, 0x%x), "
./linux-3.0/drivers/usb/sun5i_usb/hcd/core/sw_hcd_host.c: DMSG_INFO("sw_hcd_cleanup_
→ urb: qh(0x%p, 0x%x, 0x%x), urb(0x%p, %d, %d), ep(0x%p, %d, 0x%p, 0x%p) \n",
```

修改linux-3.0/drivers/usb/sun5i_usb/include/sw_usb_debug.h

```
/* 普通信息打印 */
#if 0
#define DMSG_INFO DMSG_PRINT
#else
#define DMSG_INFO(...)
#endif
```

5.4.2 3g网卡适配

安卓下开启3g网卡主要步骤是

1. 在驱动中勾选usb串口，pppd等
2. 调试通过usb_modeswitch, 模式切换后自动或者手动出现3个ttyUSBx设备
3. evdo脚本拨号

5.4.3 在内核配置里开启串口驱动, pppd驱动等

Device Drivers -> USB support -> USB Serial Converter support USB driver for GSM and CDMA modems

```
<*> PPP (point-to-point protocol) support
[*]   PPP multilink support (EXPERIMENTAL)
[*]   PPP filtering
<*>   PPP support for async serial ports
<*>   PPP support for sync tty ports
<*>   PPP Deflate compression
<*>   PPP BSD-Compress compression
<*>   PPP MPPE compression (encryption) (EXPERIMENTAL)
<*>   PPP over Ethernet (EXPERIMENTAL)
<*>   PPP over IPv4 (PPTP) (EXPERIMENTAL)
<*>   PPP over L2TP (EXPERIMENTAL)
<*>   PPP on L2TP Access Concentrator
<*>   PPP on PPTP Network Server
```

先在电脑上试着驱动3g网卡;

插上后查看内核信息:

```
[ 8497.458906] usb 2-2.1: new full-speed USB device number 12 using uhci_hcd
[ 8497.726318] usb 2-2.1: New USB device found, idVendor=1d09, idProduct=1000
[ 8497.726323] usb 2-2.1: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[ 8497.726325] usb 2-2.1: Product: USB MMC Storage
[ 8497.726327] usb 2-2.1: Manufacturer: Qualcomm, Incorporated
[ 8497.726329] usb 2-2.1: SerialNumber: 000000000002
[ 8497.729026] usb-storage 2-2.1:1.0: USB Mass Storage device detected
[ 8497.729192] scsi host43: usb-storage 2-2.1:1.0
[ 8498.732291] scsi 43:0:0:0: Direct-Access      Qualcomm MMC Storage      2.31 PQ: 0
↪ANSI: 2
[ 8498.738204] scsi 43:0:0:1: CD-ROM           Qualcomm MMC Storage      2.31 PQ: 0
↪ANSI: 2
[ 8498.741198] sd 43:0:0:0: Attached scsi generic sg3 type 0
[ 8498.835612] sd 43:0:0:0: [sdb] Attached SCSI removable disk
[ 8500.089041] sr 43:0:0:1: [sr2] scsi3-mmc drive: 357x/308x writer cdda
[ 8500.089184] sr 43:0:0:1: Attached scsi CD-ROM sr2
[ 8500.089287] sr 43:0:0:1: Attached scsi generic sg4 type 5
[ 8500.361385] sr 43:0:0:1: [sr2] CDRom (ioctl) error, command:
[ 8500.361391] Xpwrite, Read disk info 51 00 00 00 00 00 00 00 02 00
[ 8500.361395] sr 43:0:0:1: [sr2] Sense Key : Hardware Error [current]
[ 8500.361397] sr 43:0:0:1: [sr2] Add. Sense: No additional sense information
```

lsusb看网卡型号:

Bus 002 Device 007: ID 1d09:1000 TechFaith Wireless Technology Limited

可以看到被识别成了安装光盘, sr2,手动弹出光盘后就变为正常的设备: eject sr2

//有些网卡需要发送特别的指令, 需要使用usb_modeswitch切换模式

Bus 002 Device 013: ID 1d09:e003 TechFaith Wireless Technology Limited

这就和usb_modeswitch的效果一样

```
[ 8662.455084] usb 2-2.1: USB disconnect, device number 12
[ 8666.948103] usb 2-2.1: new full-speed USB device number 13 using uhci_hcd
[ 8667.220302] usb 2-2.1: New USB device found, idVendor=1d09, idProduct=e003
```

```
[ 8667.220305] usb 2-2.1: New USB device strings: Mfr=1, Product=2, SerialNumber=4
[ 8667.220307] usb 2-2.1: Product: CDMA 1x/EVDO Rev.A Device
[ 8667.220309] usb 2-2.1: Manufacturer: Co.,Ltd
[ 8667.220310] usb 2-2.1: SerialNumber: 000000000002
[ 8667.240512] usb-storage 2-2.1:1.3: USB Mass Storage device detected
[ 8667.246943] scsi host44: usb-storage 2-2.1:1.3
[ 8668.253167] scsi 44:0:0:0: Direct-Access      Qualcomm MMC Storage      2.31 PQ: 0
↳ANSI: 2
[ 8668.255704] sd 44:0:0:0: Attached scsi generic sg3 type 0
[ 8668.271240] sd 44:0:0:0: [sdb] Attached SCSI removable disk
```

Tip: PID=0x1D09, VID=0xe0003

在drivers/usb/serial/option.c里加入PID,VID (转换后的)

```
//发现华为一个型号的PID一样。。
#define HUAWEI_PRODUCT_ET128          0x1D09
#define HUAWEI_VENDOR_ID              0x12D1
//自己加一个吧
595 static const struct usb_device_id option_ids[] = {
596     { USB_DEVICE(0xe0003, 0x1d09) }, //ID 1d09:1000 TechFaith Wireless
↳Technology Limited
```

Device Monitoring Studio抓取3G网卡MessageContent

<http://blog.chinaunix.net/uid-29764914-id-5181529.html>

usb_modeswitch的使用 (新版下的规则文件下载)

http://blog.csdn.net/yang1982_0907/article/details/45969179

但是切换后仍然没有ttyUSBx, 需要手工增加:

```
modprobe usb_wwan
modprobe option
echo "1d09 e003" > /sys/bus/usb-serial/drivers/option1/new_id
```

运行后出现了ttyUSB0~2 可以将它加入到udev规则中, 在/lib/udev/rules.d/50-udev-default.rules(也可能再etc目录中)后面添加

```
ACTION=="add", SUBSYSTEM=="usb",SYSFS{idVendor}=="1d09", SYSFS{idProduct}=="1000",
RUN+="/usr/sbin/usb_modeswitch -c /etc/usb_modeswitch.conf"
RUN+="echo '1d09 e003' > /sys/bus/usb-serial/drivers/option1/new_id"
```

//安卓下已经自带

```
vim /etc/usb_modeswitch.d/1d09_1000
DefaultVendor= 0x1d09
DefaultProduct= 0x1000

TargetVendor= 0x1d09
TargetProduct= 0xe003

StandardEject=1
CheckSuccess= 10      #wait 10s
```

然后进行拨号, 在/etc/ppp/peers目录下创建新文件evdo:

```
mkdir /etc/ppp/peers/
busybox vi /etc/ppp/peers/evdo
/dev/ttyUSB0
115200
nodetach
lock
user "ctnet@mycdma.cn"
password "vnet.mobi"
crtscts
show-password
usepeerdns
noauth
noipdefault
novj
novjccomp
nocc
defaultroute
ipcp-accept-local
ipcp-accept-remote
connect '/usr/sbin/chat -s -v -f /etc/ppp/peers/evdo-connect-chat'
#connect '/system/bin/chat -s -v -f /etc/ppp/peers/evdo-connect-chat' #android
```

再创建evdo-connect-chat

```
busybox vi /etc/ppp/peers/evdo-connect-chat
```

```
TIMEOUT 2
ABORT 'NO CARRIER'
ABORT 'ERROR'
ABORT 'NO DIALTONE'
ABORT 'BUSY'
ABORT 'NO ANSWER'
"" ATE1
"" "AT+CFUN=1"
OK-AT-OK ATD#777
CONNECT ''
```

输入命令**pppd call evdo&**就可以上网了，断开网络就输入poff。

输出的拨号信息:

```
pppd call evdo
timeout set to 2 seconds
abort on (NO CARRIER)
abort on (ERROR)
abort on (NO DIALTONE)
abort on (BUSY)
abort on (NO ANSWER)
send (ATE1^M)
send (AT+CFUN=1^M)
expect (OK)
^M
OK
-- got it

send (ATD#777^M)
expect (CONNECT)
^M
AT+CFUN=1^M^M
```

```

OK^M
ATD#777^M^M
CONNECT
-- got it

send (^M)
Serial connection established.
Using interface ppp0
Connect: ppp0 <--> /dev/ttyUSB0
CHAP authentication succeeded: OK
CHAP authentication succeeded
local  IP address 10.100.35.16
remote IP address 125.88.103.85
primary  DNS address 114.114.114.114
secondary DNS address 223.5.5.5

^CTerminating on signal 2
Connect time 0.2 minutes.
Sent 355 bytes, received 126 bytes.
Connection terminated.

```

安卓上驱动

安卓上串口和dmesg的信息有限，主要看logcat

首次尝试pppd call evdo，log信息如下：

```
pppd ( 1230): Can't create lock file /var/lock/LCK..ttyUSB0
```

排查发现根本没有/var/lock目录，于是新建/var/run和/var/lock目录

```

mkdir /var
mkdir /var/run
mkdir /var/lock

```

重新拨号，dmesg提示：

```

timeout set to 2 seconds
abort on (NO CARRIER)
abort on (ERROR)
abort on (NO DIALTONE)
abort on (BUSY)
abort on (NO ANSWER)
send (ATE1^M)
send (AT+CFUN=1^M)
expect (OK)
ATE1^M^M
OK
-- got it

send (ATD#777^M)
expect (CONNECT)
^M
AT+CFUN=1^M^M
OK^M
ATD#777^M^M

```

```
CONNECT
-- got it

send (^M)
```

logcat -v time提示

```
01-02 08:01:51.419 I/pppd      ( 1195): Serial connection established.
01-02 08:01:51.429 D/pppd      ( 1195): using channel 1
01-02 08:01:51.479 I/pppd      ( 1195): Using interface ppp0
01-02 08:01:51.479 I/pppd      ( 1195): Connect: ppp0 <--> /dev/ttyUSB0
01-02 08:01:52.179 I/pppd      ( 1195): CHAP authentication succeeded: OK
01-02 08:01:52.299 I/pppd      ( 1195): local  IP address 10.100.34.51
01-02 08:01:52.299 I/pppd      ( 1195): remote IP address 125.88.103.85
01-02 08:01:52.299 I/pppd      ( 1195): primary  DNS address 114.114.114.114
01-02 08:01:52.299 I/pppd      ( 1195): secondary DNS address 223.5.5.5
```

ifconfig可见ppp0设备:

```
ppp0      Link encap:Point-to-Point Protocol
          inet addr:10.100.46.132  P-t-P:125.88.103.85  Mask:255.255.255.255
          UP POINTOPOINT RUNNING NOARP MULTICAST  MTU:1430  Metric:1
          RX packets:3 errors:0 dropped:0 overruns:0 frame:0
          TX packets:3 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:3
          RX bytes:54 (54.0 B)  TX bytes:54 (54.0 B)
```

此时已经可以ping 通114.114.114.114, 但无法解析域名

```
busybox vi init.sun5i.rc
setprop "net.dns1" "8.8.8.8"
setprop "net.dns2" "8.8.4.4"
```

安卓上3g网卡自动加载总结

1. 切换上网卡状态 (/etc/usb_modeswitch.d/1d09_1000自动完成)

创建ttyUSB0等串口

```
echo "1d09 e003" > /sys/bus/usb-serial/drivers/option1/new_id
/etc/ppp/peers/evdo /etc/ppp/peers/evdo-connect-chat 拨号脚本 (创建一次即可)
```

mkdir /var /var/run /var/lock 创建临时目录

pppd call evdo& 拨号上网, 并设置dns

0已经自动完成

1. 需要在init.sun5i.rc里加上初始化

```
on boot #echo "1d09 e003" > /sys/bus/usb-serial/drivers/option1/new_id //太早执行无效 exec /system/bin/sh
/system/etc/ppp.sh //这里延时到开机执行
```

2. 在sdk里加上对应的脚本 相关文件在device/softwinner/common/rild里, 需要修改上层的sw-common.mk来拷贝文件到system分区

```
PRODUCT_COPY_FILES += \
    device/softwinner/common/rild/ip-down:system/etc/ppp/ip-down \
```



```
device/softwinner/common/rild/ip-up:system/etc/ppp/ip-up \
device/softwinner/common/rild/call-pppd:system/etc/ppp/call-pppd \
device/softwinner/common/rild/peers/evdo:system/etc/ppp/peers/evdo\
device/softwinner/common/rild/peers/evdo-connect-chat:system/etc/ppp/peers/
↪evdo-connect-chat\
```

1. 需要在init.sun5i.rc里加上初始化

```
mkdir /var 0770 root system
mount tmpfs none /var mode=0770,uid=0,gid=1000
mkdir /var/run 0750 root system
mkdir /var/lock 0750 root system
```

1. 需要在init.sun5i.rc里加上初始化

```
service ppp /system/bin/pppd call evdo
    user root
    group system radio
    disabled
    oneshot
setprop "net.dns1" "8.8.8.8"
setprop "net.dns2" "8.8.4.4"
```

5.5 UART适配

5.5.1 linux 下适配

使用默认编译的安卓系统，ls /dev可以看到 ttyS0~3

尝试echo 'test' > /dev/ttyS0可以在串口终端看到test字符，说明uart1就是ttyS0

荔枝派上除了默认的uart1是系统log输出口外，uart0与sd卡复用，不方便使用，就剩uart3可以使用。

linux下使用stty设置串口参数：

```
stty -F /dev/ttyS1 ispeed 115200 ospeed 115200
```

但是发现对ttyS0之外的串口操作会返回：stty: /dev/ttyS1: Input/output error

在linux的驱动目录下找到相关文件：tty/serial/8250_sunxi.c

```
struct platform_device sw_uart_dev[] = {
    [0] = {.name = "sunxi-uart", .id = 0, .num_resources = ARRAY_SIZE(sw_uart_res[0]),
    ↪ .resource = &sw_uart_res[0][0], .dev = {}},
    .....
};

static int __init sw_serial_init(void)
{
    int ret;
    int i;
    int used = 0;
    char uart_para[16];

    memset(sw_serial, 0, sizeof(sw_serial));
    uart_used = 0;
```

```

    for (i=0; i<MAX_PORTS; i++, used=0) {
        sprintf(uart_para, "uart_para%d", i);
        ret = script_parser_fetch(uart_para, "uart_used", &used, sizeof(int));
        if (ret)
            UART_MSG("failed to get uart%d's used information\n", i);
        if (used) {
            uart_used |= 1 << i;
            platform_device_register(&sw_uart_dev[i]);          //这里注册平台设备，往下
        }
    }

    if (uart_used) {
        UART_MSG("used uart info.: 0x%02x\n", uart_used);
        ret = platform_driver_register(&sw_serial_driver);
        return ret;
    }

    return 0;
}

static int __devinit
sw_serial_probe(struct platform_device *dev)
{
    struct sw_serial_port *sport;
    int ret;
    UART_MSG("this coming sw_serial_probe\n");
    sport = kzalloc(sizeof(struct sw_serial_port), GFP_KERNEL);
    if (!sport)
        return -ENOMEM;
    sport->port_no = dev->id;
    sport->pdev = dev;

    ret = sw_serial_get_config(sport, dev->id);
    if (ret) {
        UART_MSG(KERN_ERR "Failed to get config information\n");
        goto free_dev;
    }

    ret = sw_serial_get_resource(sport);
    if (ret) {
        UART_MSG(KERN_ERR "Failed to get resource\n");
        goto free_dev;
    }
    platform_set_drvdata(dev, sport);

    sport->port.irq = sport->irq;
    sport->port.fifosize = 64;
    sport->port.regshift = 2;
    sport->port.iotype = UPIO_DWAPB32;
    sport->port.flags = UPF_IOREMAP | UPF_BOOT_AUTOCONF;
    sport->port.uartclk = sport->sclk;
    sport->port.pm = sw_serial_pm;
    sport->port.dev = &dev->dev;

    sport->port.mapbase = sport->mmres->start;
    sw_serial[sport->port_no] = serial8250_register_port(&sport->port); //注册端口
    UART_MSG("serial probe %d, membase %p irq %d mapbase 0x%08x\n",

```

```

        dev->id, sport->port.membase, sport->port.irq, sport->port.mapbase);
    UART_MSG("sport->pdev is %x \n &sport->pdev is %x", sport->pdev, &sport->pdev);
    UART_MSG("pdev.dev is %x \n &pdev.dev is %x", sport->pdev->dev, &sport->pdev->
    ↪dev);
    UART_MSG("dev.dev is %x \n &dev.dev is %x", dev->dev, &dev->dev);
    return 0;
free_dev:
    kfree(sport);
    sport = NULL;
    return ret;
}

```

.config

```

CONFIG_SERIAL_8250=y
CONFIG_SERIAL_8250_CONSOLE=y
CONFIG_SERIAL_8250_SUNXI=y
CONFIG_SERIAL_8250_NR_UARTS=4
CONFIG_SERIAL_8250_RUNTIME_UARTS=4

```

8250.c

```

#define UART_NR CONFIG_SERIAL_8250_NR_UARTS
static unsigned int nr_uarts = CONFIG_SERIAL_8250_RUNTIME_UARTS;

int serial8250_register_port(struct uart_port *port)
{
    struct uart_8250_port *uart;
    int ret = -ENOSPC;

    if (port->uartclk == 0)
        return -EINVAL;

    mutex_lock(&serial_mutex);

    uart = serial8250_find_match_or_unused(port);
    if (uart) {
        uart_remove_one_port(&serial8250_reg, &uart->port);
        uart->port.iobase      = port->iobase;
        uart->port.membase     = port->membase;
        uart->port.irq         = port->irq;
        uart->port.irqflags    = port->irqflags;
        uart->port.uartclk     = port->uartclk;
        uart->port.fifosize    = port->fifosize;
        uart->port.regshift    = port->regshift;
        uart->port.iotype      = port->iotype;
        uart->port.flags       = port->flags | UPF_BOOT_AUTOCONF;
        uart->port.mapbase     = port->mapbase;
        uart->port.private_data = port->private_data;
        if (port->dev)
            uart->port.dev = port->dev;

        if (port->flags & UPF_FIXED_TYPE)
            serial8250_init_fixed_type_port(uart, port->type);

        set_io_from_upio(&uart->port);
        /* Possibly override default I/O functions. */
        uart->port.flags      = port->flags | UPF_BOOT_AUTOCONF;
    }
}

```

```

        uart->port.mapbase      = port->mapbase;
        uart->port.private_data = port->private_data;
        if (port->dev)
            uart->port.dev = port->dev;

        if (port->flags & UPF_FIXED_TYPE)
            serial8250_init_fixed_type_port(uart, port->type);

        set_io_from_upio(&uart->port);
        /* Possibly override default I/O functions. */
        if (port->serial_in)
            uart->port.serial_in = port->serial_in;
        if (port->serial_out)
            uart->port.serial_out = port->serial_out;
        /* Possibly override set_termios call */
        if (port->set_termios)
            uart->port.set_termios = port->set_termios;
        if (port->pm)
            uart->port.pm = port->pm;

        if (serial8250_isa_config != NULL)
            serial8250_isa_config(0, &uart->port,
                                   &uart->capabilities);

        ret = uart_add_one_port(&serial8250_reg, &uart->port);
        if (ret == 0)
            ret = uart->port.line;
    }
    mutex_unlock(&serial_mutex);

    return ret;
}

```

serial_core.c

```

/**
 * uart_add_one_port - attach a driver-defined port structure
 * @drv: pointer to the uart low level driver structure for this port
 * @uport: uart port structure to use for this port.
 *
 * This allows the driver to register its own uart_port structure
 * with the core driver. The main purpose is to allow the low
 * level uart drivers to expand uart_port, rather than having yet
 * more levels of structures.
 */
int uart_add_one_port(struct uart_driver *drv, struct uart_port *uport)
{
    struct uart_state *state;
    struct tty_port *port;
    int ret = 0;
    struct device *tty_dev;

    BUG_ON(in_interrupt());

    if (uport->line >= drv->nr)
        return -EINVAL;

    state = drv->state + uport->line;

```

```

    port = &state->port;

    mutex_lock(&port_mutex);
    mutex_lock(&port->mutex);
    if (state->uart_port) {
        ret = -EINVAL;
        goto out;
    }
    state->uart_port = uport;
    state->pm_state = -1;

    uport->cons = drv->cons;
    uport->state = state;

    /*
     * If this port is a console, then the spinlock is already
     * initialised.
     */
    if (!(uart_console(uport) && (uport->cons->flags & CON_ENABLED))) {
        spin_lock_init(&uport->lock);
        lockdep_set_class(&uport->lock, &port_lock_key);
    }

    uart_configure_port(drv, state, uport);

    /*
     * Register the port whether it's detected or not. This allows
     * setserial to be used to alter this ports parameters.
     */
    tty_dev = tty_register_device(drv->tty_driver, uport->line, uport->dev);    //
    ↪最终注册设备
    if (likely(!IS_ERR(tty_dev))) {
        device_init_wakeup(tty_dev, 1);
        device_set_wakeup_enable(tty_dev, 0);
    } else
        printk(KERN_ERR "Cannot register tty device on line %d\n",
               uport->line);
    /*
     * Ensure UPF_DEAD is not set.
     */
    uport->flags &= ~UPF_DEAD;

out:
    mutex_unlock(&port->mutex);
    mutex_unlock(&port_mutex);

    return ret;
}

tty_io.c
```c
struct device *tty_register_device(struct tty_driver *driver, unsigned index,
 struct device *device)
{
 char name[64];
 dev_t dev = MKDEV(driver->major, driver->minor_start) + index;

```



```

 if (index >= driver->num) {
 printk(KERN_ERR "Attempt to register invalid tty line number "
 " (%d).\n", index);
 return ERR_PTR(-EINVAL);
 }

 if (driver->type == TTY_DRIVER_TYPE_PTY)
 pty_line_name(driver, index, name);
 else
 tty_line_name(driver, index, name);

 return device_create(tty_class, device, dev, NULL, name);
 }
EXPORT_SYMBOL(tty_register_device);

static void tty_line_name(struct tty_driver *driver, int index, char *p)
{
 sprintf(p, "%s%d", driver->name, index + driver->name_base); //最终的ttySx
}

```

大致的驱动加载过程就是上面这样

下面看下log信息： [ 0.158533] [uart]: used uart info.: 0x06 [ 0.158563] [uart]: this coming sw\_serial\_probe [ 0.288179] [uart]: serial probe 1, membase (null) irq 2 mapbase 0x01c28400 [ 0.295378] [uart]: sport->pdev is c08c4a08 [ 0.295383] &sport->pdev is df04c2c8[uart]: pdev.dev is c08c4ad0 [ 0.303155] &pdev.dev is c08c5680[uart]: dev.dev is c08c4ad0 [ 0.309079] &dev.dev is c08c5680[uart]: this coming sw\_serial\_probe [ 0.318097] [uart]: <3>Failed to get config information [ 0.323344] sunxi-uart: probe of sunxi-uart.2 failed with error -1

可见uart1和uart3在获取fex时都被识别出来了，uart1成功probe，但是uart3 “Failed to get config information”  
uart3对应sw\_uart\_dev[2],即

```

ret = sw_serial_get_config(sport, dev->id);
if (ret) {
 UART_MSG(KERN_ERR "Failed to get config information\n");
 goto free_dev;
}

static int sw_serial_get_config(struct sw_serial_port *sport, u32 uart_id)
{
 char uart_para[16] = {0};
 int ret;

 sprintf(uart_para, "uart_para%d", uart_id);
 ret = script_parser_fetch(uart_para, "uart_port", &sport->port_no, sizeof(int));
 if (ret)
 return -1;
 if (sport->port_no != uart_id)
 return -1;
 ret = script_parser_fetch(uart_para, "uart_type", &sport->pin_num, sizeof(int));
 if (ret)
 return -1;

 return 0;
}

```

查看可知fex里的uart\_port和uart\_parax必须一一对应，所以就是fex里之前写错了，改正回来

这时查看ttyS1参数就有了:

```
busybox stty -F /dev/ttyS1
speed 9600 baud;
intr = ^C; quit = ^\; erase = ^?; kill = ^U; eof = ^D; eol = <undef>;
eol2 = <undef>; start = ^Q; stop = ^S; susp = ^Z; rprnt = ^R; werase = ^W;
lnext = ^V; flush = ^O; min = 1; time = 0;
-brkint -imaxbel
```

设置下波特率, 并测试

```
busybox stty -F /dev/ttyS1 ispeed 115200 ospeed 115200
echo 'test' > /dev/ttyS1
```

成功在uart3上收到数据~



## 6.1 camdriod编译过程走读

### 6.1.1 camdroid编译

```
source build/envsetup.sh #导入环境变量设置，如下面的这些命令
lunch #选择平台型号，在build/envsetup.sh，包含了device/softwinner/common/
 ↪ vendorsetup.sh
mklichee #编译BootLoader和内核，模块
extract-bsp #拷贝前面的结果
make -j12 #编译camdroid
pack #打包镜像
```

### 6.1.2 lichee编译

mklichee在device/softwinner/common/vendorsetup.sh里：

```
function mklichee()
{
 mksetting
 mk_info "build lichee ..."
 mkbr && mkkernel
mkbr && mkkernel && mkuboot
 [$? -ne 0] && return 1
 return 0
}
```

## mksetting打印配置信息

```
function mksetting()
{
 printf "\n"
 printf "mkscript current setting:\n"
 printf " Chip: ${LICHEE_CHIP}\n"
 printf " Platform: ${LICHEE_PLATFORM}\n"
 printf " Board: ${LICHEE_BOARD}\n"
 printf " Output Dir: ${LICHEE_PLAT_OUT}\n"
 printf "\n"
}
```

实际打印结果:

```
> mkscript current setting:
> Chip: sun8iw8p1
> Platform:
> Board:
> Output Dir: /home/zp/develop/lichee_git/lichee_zero/camdroid/../../lichee/out/
↪ sun8iw8p1/linux/common
```

## mkbr编译buildroot

```
function mkbr()
{
 mk_info "build buildroot ..."
 local build_script
 build_script="scripts/build.sh"
 LICHEE_PLATFORM="linux"

 (cd ${LICHEE_BR_DIR} && [-x ${build_script}] && ./${build_script} "buildroot" $
↪ ${LICHEE_PLATFORM} ${LICHEE_CHIP})
 [$? -ne 0] && mk_error "build buildroot Failed" && return 1
 mk_info "build buildroot OK."
}
```

执行结果:

```
INFO: build buildroot ...
external toolchain has been installed
INFO: build buildroot OK.
```

---

**Tip:** export LICHEE\_BR\_DIR=\${LICHEE\_DIR}/buildroot

---

所以先进入lichee/buildroot, -x表示进入跟踪模式, 执行 scripts/build.sh, 这里是导入了一些路径变量

```
EXTERNAL_DIR=${LICHEE_BR_DIR}/external-packages
DESTDIR=${LICHEE_BR_DIR}/images
STAGING_DIR=${LICHEE_BR_OUT}/staging
INCDIR=${STAGING_DIR}/usr/include
TARGET_DIR=${LICHEE_BR_OUT}/target
TARGET_SYSROOT_OPT="--sysroot=${STAGING_DIR}"
```



然后执行 `./${build_script} "buildroot" ${LICHEE_PLATFORM} ${LICHEE_CHIP}` 也就是：  
`./scripts/build.sh "buildroot" linux sun8iw8p1` 解析命令里没有对后面的参数进行解析。。

```
case "$1" in
 clean)
 rm -rf ${LICHEE_BR_OUT}
 ;;
 *)
 if ["x${LICHEE_PLATFORM}" = "xlinux"] ; then #根本没有对这个赋值
 build_buildroot
 export PATH=${LICHEE_BR_OUT}/external-toolchain/bin:${PATH}
 build_external
 else
 build_toolchain
 fi
 ;;
esac
```

这里执行了后面的else，也就是build\_toolchain(其实是解压外部工具链)

```
build_toolchain()
{
 local tooldir="${LICHEE_BR_OUT}/external-toolchain"
 mkdir -p ${tooldir} #out/sun8iw8p1/linux/common/buildroot/external-
 ↪toolchain
 if [-f ${tooldir}/.installed] ; then
 printf "external toolchain has been installed\n"
 else
 printf "installing external toolchain\n"
 printf "please wait for a few minutes ...\n"
 tar --strip-components=1 \
 -jxf ${LICHEE_BR_DIR}/dl/gcc-linaro.tar.bz2 \
 -C ${tooldir}
 [$? -eq 0] && touch ${tooldir}/.installed
 fi
 export PATH=${tooldir}/bin:${PATH}
}
```

简单地说就是把 buildroot/dl/gcc-linaro.tar.bz2 解压到 out/sun8iw8p1/linux/common/buildroot/external-toolchain

另一路选择执行的两个函数：

```
build_buildroot() //编译buildroot
{
 if [! -f ${LICHEE_BR_OUT}/.config] ; then #如果没有配置过，则使用默认配置
 printf "\nUsing default config ...\n\n"
 make O=${LICHEE_BR_OUT} -C ${LICHEE_BR_DIR} ${LICHEE_BR_DEFCONF}
 fi

 make O=${LICHEE_BR_OUT} -C ${LICHEE_BR_DIR} LICHEE_GEN_ROOTFS=n \
 BR2_JLEVEL=${LICHEE_JLEVEL}
}

build_external() //external-packages,指buildroot里的所有外部包
{
 for dir in ${EXTERNAL_DIR}/* ; do
 if [-f ${dir}/Makefile] ; then
 BUILD_COMMAND="make -C ${dir} ${BUILD_OPTIONS} all"
```

```

 eval $BUILD_COMMAND
 BUILD_COMMAND="make -C ${dir} ${BUILD_OPTIONS} install"
 eval $BUILD_COMMAND
 fi
done
}

```

即执行了: `make O=out/sun8iw8p1/linux/common/buildroot -C ./ LICHEE_GEN_ROOTFS=n BR2_JLEVEL=${LICHEE_JLEVEL}`

即编译br目录, 但不生成rootfs

## mkkernel编译内核

```

function mkkernel()
{
 local platformdef=$tdevice

 if [! -n $tdevice]; then
 echo "Please lunch device"
 return 1
 fi

 echo "Make the kernel"
 echo "platformdef=${platformdef}"
 (cd ${LICHEE_KERN_DIR}/; ./build.sh -p ${platformdef})
 [$? -ne 0] && mk_error "build mkkernel fail" && return 1
 echo "Make the kernel finish"
 return 0
}

```

执行显示结果:

```

Make the kernel
platformdef=tiger-cdr

```

即执行了: `cd lichee/linux3.4; ./build.sh -p tiger-cdr`

```

#这里首先导入了PLATFORM=tiger-cdr
while getopts hp:m: OPTION
do
 case $OPTION in
 h) show_help
 ;;
 p) PLATFORM=$OPTARG
 ;;
 m) MODULE=$OPTARG
 ;;
 *) show_help
 ;;
 esac
done

#没有指定platform则退出
if [-z "$PLATFORM"]; then
 show_help
 exit 1

```

```

fi

#没有指定模块则默认编译所有
if [-z "$MODULE"]; then
 MODULE="all"
fi

#执行scripts/build_tiger-cdr.sh
if [-x ./scripts/build_${PLATFORM}.sh]; then
 ./scripts/build_${PLATFORM}.sh $MODULE
else
 printf "\nERROR: Invalid Platform\nonly sun6i sun6i_fiber or sun6i_
↳dragonboard support\n"
 show_help
 exit 1
fi

```

对应目录里脚本为:

```

LICHEE_ROOT=`(cd ${LICHEE_KDIR}/..; pwd)`
export PATH=${LICHEE_ROOT}/out/sun8iw8pl/linux/common/buildroot/external-toolchain/
↳bin:${LICHEE_ROOT}/tools/pack/pctools/linux/android:$PATH

case "$1" in
kernel)
 build_kernel
 ;;
modules)
 build_modules
 ;;
clean)
 clean_kernel
 clean_modules
 ;;
*)
 build_kernel
 build_modules
 # build_ramfs #这里可以生成boot.img
 # gen_output
 echo -e "\n\033[0;31;1m${LICHEE_CHIP} compile Kernel successful\033[0m\n\n"
 ;;
esac

```

总之就是默认编译了内核和模块。。可以看到build\_kernel其实编译了uImage和modules, 并把bImage和zImage拷到了output目录 把ko文件拷到了 lichee/linux-3.4/output/lib/modules/3.4.39 下

```

build_kernel()
{
 echo "Building kernel"

 cd ${LICHEE_KDIR}

 rm -rf output/
 echo "${LICHEE_MOD_DIR}"
 mkdir -p ${LICHEE_MOD_DIR}
 # echo "build_kernel LICHEE_KERN_DEFCONF" ${LICHEE_KERN_DEFCONF}
 # We need to copy rootfs files to compile kernel for linux image

```

```
cp -f rootfs.cpio.gz output/

if [! -f .config] ; then
printf "\n\033[0;31;1mUsing default config ${LICHEE_KERN_DEFCONF} ...
↪\033[0m\n\n"
 printf "\n\033[0;31;1mUsing default config sun8iw8p1smp_tiger_cdr_
↪defconfig ...\033[0m\n\n"
cp arch/arm/configs/${LICHEE_KERN_DEFCONF} .config
 cp arch/arm/configs/sun8iw8p1smp_tiger_cdr_defconfig .config
fi

make ARCH=arm CROSS_COMPILE=${CROSS_COMPILE} -j${LICHEE_JLEVEL} uImage modules

update_kern_ver

#The Image is origin binary from vmlinux.
cp -vf arch/arm/boot/Image output/bImage
cp -vf arch/arm/boot/[zu]Image output/

cp .config output/

tar -jcf output/vmlinux.tar.bz2 vmlinux

if [! -f ./drivers/arisc/binary/arisc]; then
 echo "arisc" > ./drivers/arisc/binary/arisc
fi
cp ./drivers/arisc/binary/arisc output/

for file in $(find drivers sound crypto block fs security net -name "*.ko");
↪do
 cp $file ${LICHEE_MOD_DIR}
done
cp -f Module.symvers ${LICHEE_MOD_DIR}

}
```

build\_modules部分就没做事了

## mkuboot编译uboot

(这部分代码没有放出)

```
function mkuboot ()
{
 (cd ${LICHEE_UBOOT_DIR}; ./build.sh -p sun8iw8p1_nor)
 [$? -ne 0] && echo "build u-boot Failed" && return 1
 (cd ${LICHEE_UBOOT_DIR}; ./build.sh -p sun8iw8p1)
 [$? -ne 0] && echo "build u-boot Failed" && return 1
 return 0
}
```

## 6.1.3 extract-bsp

拷贝zimage和modules

```

CURDIR=$PWD
cd $DEVICE

#extract kernel
if [-f kernel]; then
 rm kernel
fi
cp -rf $LICHEE_KERN_OUTDIR/zImage kernel
echo "$DEVICE/zImage copied!"

#extract linux modules
if [-d modules]; then
 rm -rf modules
fi
mkdir -p modules/modules
cp -rf $LINUXOUT_MODULE_DIR modules/modules
echo "$DEVICE/modules copied!"
chmod 0755 modules/modules/*

cd $CURDIR

```

### 6.1.4 pack打包镜像

```

function pack()
{
 if ["-d" == $1]; then
 echo "pack card"
 pack_card
 else
 echo "pack_normal"
 pack_normal
 fi

 return 0
}

function pack_normal()
{
 local platformdef=$tdevice
 echo "Pack to image....." ${platformdef}

 export CAMLINUX_IMAGE_OUT="$CAMLINUX_BUILD_TOP/out/target/product/${platformdef}"
 if ["tiger-ipc" == ${platformdef}]; then
 echo "copy tiger-ipc uboot bin files"
 cp -rf ${LICHEE_TOOLS_DIR}/pack/chips/sun8iw8p1/configs/tiger-ipc/
 ↪bin ${LICHEE_TOOLS_DIR}/pack/chips/sun8iw8p1/
 fi
 (cd ${LICHEE_TOOLS_DIR}/pack; ./pack -c sun8iw8p1 -p camdroid -b $
 ↪{platformdef})
 [$? -ne 0] && echo "pack Failed" && return 0
 return 0
}

function pack_card()
{
 ...
 (cd ${LICHEE_TOOLS_DIR}/pack; ./pack -c sun8iw8p1 -p camdroid -b ${platformdef} -d_
 ↪card0)
}

```



```
...
}
```

打包过程:

```
./pack -c sun8iw8p1 -p camdroid -b ${platformdef}
```

```
do_prepare
do_common
echo "CAMLINUX_IMAGE_OUT=${CAMLINUX_IMAGE_OUT}
do_pack_${PACK_PLATFORM}
do_finish
```

## do\_prepare

```
tools_file_list=(
common/tools/split_xxxx.fex
chips/${PACK_CHIP}/tools/split_xxxx.fex
common/tools/usbttool_test.fex
chips/${PACK_CHIP}/tools/usbttool_test.fex
common/tools/cardscrip.fex
chips/${PACK_CHIP}/tools/cardscrip.fex
common/tools/cardtool.fex
chips/${PACK_CHIP}/tools/cardtool.fex
common/tools/usbttool.fex
chips/${PACK_CHIP}/tools/usbttool.fex
common/tools/aultls32.fex
chips/${PACK_CHIP}/tools/aultls32.fex
common/tools/aultools.fex
chips/${PACK_CHIP}/tools/aultools.fex
)

configs_file_list=(
common/toc/toc1.fex
common/toc/toc0.fex
common/imagecfg/image_linux.cfg
common/partition/sys_partition_dump.fex
common/partition/sys_partition_private.fex
chips/${PACK_CHIP}/configs/default/*
chips/${PACK_CHIP}/configs/${PACK_BOARD}/*.fex
chips/${PACK_CHIP}/configs/${PACK_BOARD}/*.cfg
)

boot_resource_list=(
chips/${PACK_CHIP}/boot-resource/boot-resource.out/
chips/${PACK_CHIP}/boot-resource/boot-resource.ini.out/
chips/${PACK_CHIP}/configs/${PACK_BOARD}/bootlogo.bmp.out/boot-resource/
)

boot_file_list=(
chips/${PACK_CHIP}/bin/boot0_nand_${PACK_CHIP}.bin.out/boot0_nand.fex
chips/${PACK_CHIP}/bin/boot0_sdcard_${PACK_CHIP}.bin.out/boot0_sdcard.fex
chips/${PACK_CHIP}/bin/boot0_spinor_${PACK_CHIP}.bin.out/boot0_spinor.fex
chips/${PACK_CHIP}/bin/fes1_${PACK_CHIP}.bin.out/fes1.fex
chips/${PACK_CHIP}/bin/u-boot-${PACK_CHIP}.bin.out/u-boot.fex
chips/${PACK_CHIP}/bin/u-boot-spinor-${PACK_CHIP}.bin.out/u-boot-spinor.fex
```

```

)

boot_file_secure=(
chips/${PACK_CHIP}/bin/semelis.bin:out/semelis.bin
chips/${PACK_CHIP}/bin/sboot_${PACK_CHIP}.bin:out/sboot.bin
)

function do_prepare()
{
...
 # Cleanup
 rm -rf out/
 mkdir -p out/

 printf "copying tools file\n"
 for file in ${tools_file_list[@]} ; do
 cp -f $file out/ 2> /dev/null
 done
...
#拷贝各种fex到out下, 包含开机画面等

```

## do\_common

转换格式, 通过fex更新boot

```

function do_common()
{
 cd out/

 busybox unix2dos sys_config.fex
 busybox unix2dos sys_partition.fex
 script sys_config.fex > /dev/null
 script sys_partition.fex > /dev/null
 cp -f sys_config.bin config.fex

 if ["x${PACK_PLATFORM}" = "xdragonboard"] ; then
 busybox dos2unix test_config.fex
 cp test_config.fex boot-resource/
 busybox unix2dos test_config.fex
 script test_config.fex > /dev/null
 cp test_config.bin boot-resource/
 fi

 # Those files for SpiNor. We will try to find sys_partition_nor.fex
 if [-f sys_partition_nor.fex -o \
 -f sys_partition_nor_${PACK_PLATFORM}.fex]; then

 mv -f sys_partition_nor_${PACK_PLATFORM}.fex \
 sys_partition_nor.fex >/dev/null 2>&1

 # Here, will create sys_partition_nor.bin
 busybox unix2dos sys_partition_nor.fex
 script sys_partition_nor.fex > /dev/null
 update_boot0 boot0_spinor.fex sys_config.bin SDMMC_CARD > /dev/null
 update_uboot u-boot-spinor.fex sys_config.bin >/dev/null
 fi
}

```

```

Those files for Nand or Card
update_boot0 boot0_nand.fex sys_config.bin NAND > /dev/null
update_boot0 boot0_sdcard.fex sys_config.bin SDMMC_CARD > /dev/null
update_uboot u-boot.fex sys_config.bin > /dev/null
update_fesl fesl.fex sys_config.bin > /dev/null
fsbuild boot-resource.ini split_xxxx.fex > /dev/null

if ["x${PACK_FUNC}" = "xprvt"] ; then
 u_boot_env_gen env_burn.cfg env.fex > /dev/null
else
 u_boot_env_gen env.cfg env.fex > /dev/null
fi

if [-f "${LICHEE_OUT}/arisc"] ; then
 ln -s $LICHEE_OUT/arisc arisc.fex
fi
}

```

## do\_pack\_\${PACK\_PLATFORM}

```

function do_pack_android()
{
 printf "packing for android\n"

 if [-z "${ANDROID_IMAGE_OUT}"] ; then
 pack_error "please specify ANDROID_IMAGE_OUT env"
 exit 1
 fi

 ln -s ${ANDROID_IMAGE_OUT}/boot.img boot.fex
 ln -s ${ANDROID_IMAGE_OUT}/system.img system.fex
 ln -s ${ANDROID_IMAGE_OUT}/recovery.img recovery.fex

 if [-f ${ANDROID_IMAGE_OUT}/userdata.img] ; then
 ln -s ${ANDROID_IMAGE_OUT}/userdata.img userdata.fex
 fi

 if ["x${PACK_SIG}" = "xsig"] ; then
 echo "signature sunxi mbr"
 signature sunxi_mbr.fex dlinfex
 echo "signature over"
 elif ["x${PACK_SIG}" = "xsecure"] ; then
 do_signature
 else
 echo "normal"
 fi
}

function do_pack_camdroid()
{
 printf "packing for camdroid\n"

 if [-z "${CAMLINUX_IMAGE_OUT}"] ; then
 pack_error "please specify CAMLINUX_IMAGE_OUT env"
 exit 1
 fi
}

```

```

fi
ln -s ${CAMLINUX_IMAGE_OUT}/boot.img boot.fex
ln -s ${CAMLINUX_IMAGE_OUT}/system.img rootfs.fex #使用前面打包的system.img作为
根文件系统
}
function do_pack_dragonboard()
{
 printf "packing for dragonboard\n"

 ln -s ${LICHEE_OUT}/boot.img boot.fex
 ln -s ${LICHEE_OUT}/rootfs.ext4 rootfs.fex
}
function do_pack_linux()
{
 printf "packing for linux\n"
#输出目录是linux-3.4/output/
 ln -s ${LICHEE_OUT}/vmlinux.tar.bz2 vmlinux.fex
 ln -s ${LICHEE_OUT}/boot.img boot.fex #
 ln -s ${LICHEE_OUT}/rootfs.ext4 rootfs.fex

 if ["x${PACK_SIG}" = "xsecure"] ; then
 do_signature
 else
 echo "normal"
 fi
}

```

## do\_finish

```

function do_finish()
{
 # Yeah, it should contain all files into full_img.fex for spinor
 # Because, as usually, spinor image size is very small.
 # If fail to create full_img.fex, we should fake it empty.

 # WTF, it is so ugly! It must be sunxi_mbr.fex, not sunxi_mbr_xxx.fex
 # Check whether sys_partition_nor.bin is exist, and create sunxi_mbr.fex
 # for Nor.
 if [-f sys_partition_nor.bin]; then
 mv -f sys_partition.bin sys_partition.bin_back
 cp -f sys_partition_nor.bin sys_partition.bin
 update_mbr sys_partition.bin 1 > /dev/null

 merge_package full_img.fex boot0_spinor.fex \
 u-boot-spinor.fex sunxi_mbr.fex sys_partition.bin

 mv -f sys_partition.bin_back sys_partition.bin
 fi
 if [! -f full_img.fex]; then
 echo "full_img.fex is empty" > full_img.fex
 fi

 update_mbr sys_partition_nor.bin 1 > /dev/null
 dragon image.cfg sys_partition_nor.fex

 if [-e ${IMG_NAME}]; then

```

```

mv ${IMG_NAME} ../${IMG_NAME}
echo '-----image is at-----'
echo -e '\033[0;31;1m'
echo ${ROOT_DIR}/${IMG_NAME}
echo -e '\033[0m'

fi

cd ..
printf "pack finish\n"
}

```

### 6.1.5 打包完成后的布局

Device	Boot	Start	End	Blocks	Id	System
/dev/sdf1		56992	1939454	941231+	b	W95 FAT32
/dev/sdf2	*	40992	46111	2560	6	FAT16
/dev/sdf3		1	56992	28496	85	Linux extended
/dev/sdf5		46112	55199	4544	83	Linux
/dev/sdf6		55200	56223	512	83	Linux
/dev/sdf7		56224	56479	128	83	Linux
/dev/sdf8		56480	56735	128	83	Linux
/dev/sdf9		56736	56863	64	83	Linux
/dev/sdf10		56864	56991	64	83	Linux

配置里的分区表:

```

[partition]
name = boot
size = 5120
downloadfile = "boot.fex" #内核ramfs
[partition]
name = system
size = 9088
downloadfile = "rootfs.fex" #根文件系统
[partition]
name = cfg
size = 1024
downloadfile = "cfg.fex" #jffs2的cfg, 用于保存可变的配置字段
[partition]
name = boot_logo
size = 256
downloadfile = "boot_logo.fex"
[partition]
name = shutdown_logo
size = 256
downloadfile = "shutdown_logo.fex"
[partition]
name = env
size = 128
downloadfile = "env.fex" #uboot启动的环境变量
[partition]
name = private
size = 128

```

## cfg.fex的生成

device/softwinner/tiger-cdr/res/make\_cfg\_fex.sh

```
CFG_PATH="/pack/chips/sun8iw8pl/configs/CDR/cfg.fex"
DEST=$LICHEE_TOOLS_DIR$CFG_PATH

echo "./mkfs.jffs2 -d ./data -o cfg.fex"
#-p total size
./mkfs.jffs2 -d ./cfg -p 0x80000 -o cfg.fex

echo "move cfg.fex to $DEST"
mv cfg.fex $DEST
```

## boot.fex(boot.img)生成

```
INSTALLED_BOOTIMAGE_TARGET := $(PRODUCT_OUT)/boot.img
$(INSTALLED_BOOTIMAGE_TARGET): $(MKBOOTIMG) $(INTERNAL_BOOTIMAGE_FILES)
 $(call pretty,"Target boot image: $@")
 $(hide) $(MKBOOTIMG) $(INTERNAL_BOOTIMAGE_ARGS) $(BOARD_MKBOOTIMG_ARGS) --
 ↪output $@
 $(hide) $(call assert-max-image-size,$@,$(BOARD_BOOTIMAGE_PARTITION_SIZE),raw)
```

```
regen_rootfs_cpio()
{
 echo "regenerate rootfs cpio"

 cd ${LICHEE_KDIR}/output
 echo "###"
 if [-x "../scripts/build_rootfs.sh"]; then
 ../scripts/build_rootfs.sh e ./rootfs.cpio.gz > /dev/null
 else
 echo "No such file: scripts/build_rootfs.sh"
 exit 1
 fi
 echo "###"

 mkdir -p ./skel/lib/modules/${KERNEL_VERSION}
 echo "###"

 if [-e ${LICHEE_MOD_DIR}/nand.ko]; then
 cp ${LICHEE_MOD_DIR}/nand.ko ./skel/lib/modules/${KERNEL_VERSION}
 if [$? -ne 0]; then
 echo "copy nand module error: $?"
 exit 1
 fi
 fi
 echo "####ttt"

 ko_file=`find ./skel/lib/modules/${KERNEL_VERSION}/ -name *.ko`
 if [! -z "$ko_file"]; then
 ${STRIP} -d ./skel/lib/modules/${KERNEL_VERSION}/*.ko
 fi
 echo "####ttt"

 rm -f rootfs.cpio.gz
```



```
../scripts/build_rootfs.sh c rootfs.cpio.gz > /dev/null
rm -rf skel
echo "###ttt"

cd - > /dev/null
}
```

---

**Tip:** TARGET\_ROOT\_OUT := \$(PRODUCT\_OUT)/\$(TARGET\_COPY\_OUT\_ROOT)

---

### 6.1.6 修改配置生成linux镜像

由上面的走读可知，需要生成boot分区，linux根文件系统分区，及对应的分区表

内核配置: linux-3.4/arch/arm/configs/sun8iw8plsmpl\_defconfig linux-3.4/arch/arm/configs/sun8iw8plsmpl\_tiger\_cdr\_defconfig

配置内核: make ARCH=arm menuconfig

板级配置和开机logo ls tools/pack/chips/sun8iw8pl/configs/tiger-cdr/

boot-resource cfg.fex sys\_config.fex sys\_partition\_nor\_camdroid.fex